

Term paper

Energy Demand Forecaster

Author 1: Karrar Al-Ameeri
Matriculation No.1: 7027181

Author 2: Aravind Lakshmi Narayanan
Matriculation No.2: 7027887

Author 3: Abhilash Nayak
Matriculation No.3: 7027894

Author 4: Pranoti Patil
Matriculation No.4: 7027601

Course of Studies: Business Intelligence and Data Analytics

First examiner: Prof. Dr. Elmar Wings
Submission date: May 24, 2026

Contents

Contents	3
List of Figures	9
List of Listings	11
I. Introduction	15
1. Introduction	17
1.1. Background and Motivation	17
1.2. Problem Description	17
1.3. Challenges in Electricity Load Forecasting	18
1.4. Proposed Solutions	18
1.5. Objectives of the Study	19
1.6. Structure of the Report	19
II. Domain Knowledge - Application	21
2. Domain Knowledge	23
2.1. Application	23
2.2. Domain Problem	23
2.3. Data Acquisition	24
2.4. Data Quantity	25
2.5. Data Quality	25
2.6. Data Relevance	26
2.7. Outliers and Anomalies	26
III. Domain Knowledge - Machine Learning/Algorithms	29
3. Packages	31
3.1. Introduction	31
3.2. Pandas	31
3.2.1. Introduction	31
3.2.2. Description	32
3.2.3. Installation	32
3.2.4. Example — Description	33
3.2.5. Example — Manual	34
3.2.6. Conclusion	37
3.2.7. Further Readings	37
3.3. NumPy	37
3.3.1. Introduction	37
3.3.2. Description	38
3.3.3. Installation	38
3.3.4. Example — Description	39

3.3.5. Example — Manual	40
3.3.6. Conclusion	41
3.3.7. Further Readings	42
3.4. Statsmodels	42
3.4.1. Introduction	42
3.4.2. Description	42
3.4.3. Installation	43
3.4.4. Example — Description	44
3.4.5. Example — Manual	45
3.4.6. Conclusion	48
3.4.7. Further Readings	48
4. Data Mining Algorithm	49
4.1. Introduction	49
4.2. SARIMA (Seasonal AutoRegressive Integrated Moving Average)	49
4.3. LSTM	53
IV. Domain Knowledge - Tools	61
5. Development Environment	63
5.0.1. Python Version	63
5.1. Python Description	63
5.2. Python Installation	63
5.3. Python Configuration	64
5.4. Python First Steps	66
5.5. Python Program: “Hello, World!”	67
V. Methodology	69
6. Methodology	71
6.1. KDD Process	71
6.1.1. Selection	71
6.1.2. Preprocessing (Cleaning)	72
6.1.3. Transformation	72
6.1.4. Data Mining	73
6.1.5. Interpretation and Evaluation	73
6.2. Justification for Using KDD Process	74
6.2.1. Data-Centric Nature Fits Time Series Projects	74
6.2.2. Structured Transformation Emphasis	74
6.2.3. Iterative, Yet Linear Flow Works Well	75
6.2.4. Interpretability of Results	75
6.2.5. Ideal for Academic & Research Projects	75
6.2.6. Final Justification Statement	75
VI. Developmenmt	77
7. Database	79
7.1. Data Selection	79
7.1.1. Origin	79
7.1.2. Features	80
7.1.3. Data Types	82
7.1.4. Quality	83

7.1.5.	Quantity	84
7.1.6.	Fairness/Bias	85
7.2.	Data Processing	86
7.2.1.	One Database	86
7.2.2.	Properties	87
7.2.3.	Outliers	88
7.2.4.	Anomalies	88
7.2.5.	Augmentation	89
7.2.6.	Data Version Control	90
8.	KDD: Data Transformation	93
8.1.	Role in the KDD Process	93
8.2.	Application	93
8.3.	Input	94
8.4.	Hyperparameters (Feature Extraction)	94
8.5.	Training Integration	95
8.6.	Interpretation	96
8.7.	Output	96
8.8.	Flowchart (Transformation)	96
9.	KDD: Data Mining	99
9.1.	Role in the KDD Process	99
9.2.	Application	99
9.3.	Input	100
9.4.	Hyperparameters (Model and Training)	101
9.5.	Training	102
9.6.	Interpretation (Model Selection and Conversion)	102
9.7.	Output (End of KDD in This Project)	103
9.8.	Flowchart (Data Mining)	104
VII.	Development to Deployment	105
10.	Development To Deployment	107
10.1.	Data structure	107
10.2.	Tools	108
10.3.	File Structure for Model Exchangement	109
10.4.	Description of File Types	110
10.5.	Saving Models	111
10.6.	Loading	112
11.	Evaluation	113
11.1.	Concepts	113
11.1.1.	Understanding Electricity Demand as a Time Series Problem	113
11.1.2.	Building an Automated ML Pipeline for Forecasting	113
11.1.3.	Making Forecasting Accessible with a GUI	114
11.1.4.	Additional Conceptual Elements	114
VIII.	Deployment	115
12.	Application Deployment	117
12.1.	Deployment Strategy	117
12.1.1.	Local-First Architecture	117
12.1.2.	Comparison of Deployment Options	117

12.1.3. Streamlit as the Deployment Framework	118
12.2. Portable Bundle Pipeline	118
12.2.1. Overview	118
12.2.2. Build Steps	118
12.2.3. Build Pipeline Flowchart	118
12.2.4. Runtime Packages	118
12.3. Bundle Directory Structure	118
12.4. Platform Deployment	118
12.4.1. macOS Deployment	118
12.4.2. Windows Deployment	121
12.5. Shipped Model Artefacts	122
12.5.1. LSTM Checkpoint	122
12.5.2. Scaler Parameters	122
12.5.3. Sample Dataset	122
12.6. Developer Deployment	122
12.7. Full Inference Script – The INO file	123

IX. Monitoring 125

13. Monitoring	127
13.1. Idea	127
13.2. Plan/Description	127
13.3. Getting New Data	128
13.4. Updating Data in ML Pipeline	128
13.5. Checks	130
13.6. Code: Functions	131
13.7. Code: Tests and Validation	132
13.8. Privacy	133
13.9. Robustness	134
13.10 Process	134

X. Verification - Evaluation - Conclusion 137

14. Evaluation	139
14.1. Concept	139
14.1.1. Understanding Electricity Demand as a Time Series Problem	139
14.1.2. Exploratory Time Series Understanding	140
14.1.3. Forecasting Pipeline Concept	141
14.1.4. Model Choice: SARIMA and LSTM	141
14.2. Application	142
14.2.1. Application Architecture	142
14.2.2. Application Flow	142
14.2.3. Input and Output	143
14.2.4. Tools Used in the Application	143
14.2.5. Code Example: Data Loading and Feature Preparation	144
14.2.6. Use in the Project	144
14.3. Results	144
14.3.1. Forecast Output	145
14.3.2. LSTM Forecast Result	145
14.3.3. LSTM Training Behaviour	146
14.3.4. Model Performance Metrics	146
14.3.5. Code Example: Evaluation Metrics	146
14.3.6. SARIMA Result Interpretation	147

14.3.7. LSTM Result Interpretation	147
14.3.8. Residual Diagnostics	148
14.3.9. Comparison of SARIMA and LSTM	148
14.3.10. Result Value	148
14.3.11. Summary of Results	149
15. Validation	151
15.1. General	151
15.1.1. Chronological Train/Test Split	151
15.1.2. Validation Metrics	151
15.1.3. Visual Validation	151
15.1.4. Seasonal Decomposition Validation	152
15.1.5. Model Comparison Validation	152
15.1.6. Practical Validation	153
15.2. Validation – 3 Ideas	153
15.2.1. Idea 1: Rolling Forecast Validation	153
15.2.2. Idea 2: Benchmark Against Simple Baseline Models	154
15.2.3. Idea 3: Peak Demand Validation	154
16. Conclusion	155
16.1. Conclusion	155
16.1.1. Self-Critical Reflection	155
16.1.2. Unanswered Points	156
16.1.3. Next Steps	156
16.1.4. Final Reflection	157
XI. Appendix	159
17. Bill of Materials	161
17.1. Material List and Hardware Bill of Materials	161
17.1.1. Laptop	161
17.1.2. Mouse	162
17.1.3. Keyboard	162
17.1.4. Additional Recommendations	163
17.2. Software Requirement and Bill Of Materials	163
17.2.1. Visual Studio Code (VS Code)	163
18. requirements.txt	165
18.1. Purpose of requirements.txt	165
18.2. Dependencies Used	165
18.3. Explanation of Packages	166
18.4. How to Use requirements.txt	166
18.5. Conclusion	167
19. Personal Findings and Learnings – Karrar Al-Ameeri	169
19.1. Overview of My Contribution	169
19.2. Team Leadership and Project Coordination	169
19.2.1. Structuring the Team and the Work	169
19.3. Building the Streamlit Application	169
19.3.1. Design Goals	170
19.4. Cross-Platform Portable Deployment	170
19.4.1. The Problem	170
19.4.2. The Solution	170
19.5. The Quantile Loss Innovation	171

19.6. Reflection	171
20. Personal Findings and Learnings – Aravind Lakshmi Narayanan	173
20.1. Overview of My Contribution	173
20.2. Dataset Selection	173
20.3. Data Validation and Quality Assessment	173
20.4. Literature Review and Citation Integrity	174
20.5. Reflection	174
21. Personal Findings and Learnings – Abhilash Nayak and Pranoti Patil	177
21.1. Personal Findings and Learnings – Abhilash Nayak	177
21.1.1. Overview of My Contribution	177
21.2. KDD, Database, and Data Understanding	177
21.3. SARIMA and Forecast Evaluation	178
21.4. Literature Review and Academic Direction	178
21.5. Poster and Communication	178
21.6. Reflection	178
21.7. Personal Findings and Learnings – Pranoti Patil	179
21.7.1. Overview of My Contribution	179
21.8. Application Development Data Documentation	179
21.9. Package Documentation and Reproducibility	179
21.10. Literature Review and Model Understanding	180
21.11. Poster and Visual Presentation	180
21.12. Reflection	180
21.13. Shared Collaboration	181
Literature	183

List of Figures

3.1. Role of each Python package within the KDD pipeline for hourly electricity load forecasting. Packages covered in this chapter are highlighted; PyTorch (LSTM) is discussed in Chapter 4.	31
3.2. Structure of the PJME hourly DataFrame after loading with Pandas. The Datetime column is converted to a DatetimeIndex , enabling time-series operations such as resample() , asfreq() , and rolling() throughout the preprocessing pipeline.	33
3.3. PJME daily mean energy consumption (2002–2018) with a 7-day rolling average overlay (orange dashed). The moving average reduces intra-week noise and reveals clear seasonal cycles: summer cooling peaks in July–August and winter heating surges in January–February. Generated by scripts/generate_report_figures.py	35
3.4. Cyclical encoding maps discrete hour integers onto a unit circle via $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$. Hours 0 and 23 are only 15° apart on the circle (b), preserving the temporal adjacency that the raw integer representation (a) destroys.	39
3.5. Statsmodels SARIMA workflow used in this project. A SARIMAX object is instantiated with the model order, fitted via Kalman-filter MLE, and the resulting SARIMAXResults object provides both the 720-step point forecast with confidence interval and the residual diagnostic statistics.	43
3.6. First 7 days (168 hours) of the SARIMA(1,1,1)(1,1,1,24) 720-hour forecast (blue) versus actual PJME demand (black), with a 95 % confidence interval (shaded). Zooming into the opening week reveals that SARIMA captures the daily consumption cycle and broad load level, even though accuracy deteriorates substantially over the full 30-day horizon (MAE = 11,662 MW). Generated by scripts/generate_report_figures.py	45
3.7. Residual ACF (left) and PACF (right) of the SARIMA(1,1,1)(1,1,1,24) model at lags 1–48. Lag 0 is omitted so the y-axis rescales to show the actual residual structure. Bars exceeding the shaded 95 % confidence bands indicate residual autocorrelation that the model has not fully absorbed; any spikes near lags 24 or 48 would signal unresolved daily seasonality. Generated by scripts/generate_report_figures.py	48
4.1. Layered transformations applied by SARIMA(1, 1, 1)(1, 1, 1) ₂₄ . Each differencing step removes a distinct source of non-stationarity before the AR and MA terms model the remaining stochastic structure.	51
4.2. End-to-end SARIMA forecasting pipeline from raw CSV ingestion to quantitative evaluation.	53
4.3. Architecture of the LSTMForecaster model. A 168-hour input window of 9 features is processed sequentially by two stacked LSTM layers. The final hidden state h_T is projected by a linear head to produce 720 simultaneous output values — one scaled load prediction per forecast hour.	55

4.4. End-to-end LSTM-q70 forecasting pipeline. Feature engineering and min-max scaling precede windowed dataset construction; the trained model produces a 720-step normalised forecast that is inverse-scaled before evaluation against the test series.	58
5.1. Python 3.12 Installation Flow on Windows 11	64
5.2. Python Virtual Environment Configuration Workflow	65
5.3. Initial Verification and Testing of Python Environment	66
5.4. Execution Flow of a Basic Python Script	67
6.1. The five KDD stages applied in this project, with the corresponding source module for each stage. The pipeline flows from raw CSV ingestion at the top to quantitative model evaluation at the bottom.	71
7.1. KDD Data Processing Flow for PJME Electricity Forecasting	86
8.1. Transformation pipeline from the cleaned single-column series to the windowed, scaled (168×9) feature matrices consumed by the LSTM DataLoader . SARIMA bypasses this pipeline and reads the cleaned series directly.	97
9.1. Data mining pipeline showing the parallel SARIMA and LSTM-q70 branches converging at evaluation. Both models are scored on the same 720-hour held-out test window before their artefacts are saved to disk.	104
10.1. Forecasting Data Input and Output Flow	108
12.1. Portable bundle build pipeline. The script is executed once on a developer Mac; the resulting dist/portable/ folder can be copied to any distribution medium.	120
12.2. macOS deployment sequence. Step 2 (Gatekeeper setup) is required only once per bundle copy.	121
13.1. Energy Demand Forecasting Pipeline: SARIMA vs LSTM	130
13.2. Energy Demand Forecasting Pipeline: From Data Input to Export	136
14.1. PJME hourly electricity load over the full historical period	140
14.2. Average electricity load by hour of day, day of week, and month	140
14.3. ACF and PACF plots for PJME load up to 72 hourly lags	141
14.4. SARIMA forecast compared with actual PJME electricity demand	145
14.5. LSTM forecast compared with actual PJME electricity demand	145
14.6. LSTM training and validation loss curves	146
14.7. Residual ACF plot after SARIMA forecasting	148
15.1. Seasonal decomposition of PJME hourly electricity demand	152

List of Listings

Acronyms

Part I.

Introduction

1. Introduction

1.1. Background and Motivation

Accurate electricity load forecasting is crucial for utility companies, grid operators, and policymakers operating in the energy sector, where the inability to store electricity on a massive scale requires supply to continuously and precisely match consumer demand. Forecasting supports critical operations such as grid stability management, dynamic pricing, energy trading, and infrastructure planning [GG+24].

Utility providers and regional transmission organizations such as PJM Interconnection, which oversees one of the largest electricity markets in North America generate massive volumes of high-frequency consumption data daily. With the advent of smart grids and scalable analytics tools, analyzing large-scale time series data for forecasting energy loads has become not just feasible but essential for operational efficiency [Rob20].

The digitalization of the energy sector through smart meters and Internet of Things (IoT) devices has led to the generation of large volumes of high-frequency data. Classical time series methods such as ARIMA (Autoregressive Integrated Moving Average) and its seasonal extension SARIMA offer reliable forecasting under linear and stationary conditions. However, electricity consumption is often highly non-linear, influenced by a multitude of internal and external factors including time of day, day of the week, weather conditions, and macroeconomic indicators [Sha16].

Recent years have witnessed the rise of Deep Learning (DL) models in complex forecasting tasks. These techniques have demonstrated improved accuracy by uncovering non-linear relationships within high-frequency datasets. Long Short-Term Memory (LSTM) networks an advanced variant of Recurrent Neural Networks (RNNs) specifically designed to overcome the vanishing gradient problem are increasingly being adopted in research and industry to handle long-term dependencies in sequential data [MSSJ22a].

1.2. Problem Description

Grid operators face the challenge of forecasting demand across various time intervals, from minute-by-minute load balancing to multi-year capacity planning. Inaccurate forecasts can disrupt generation schedules and increase reliance on expensive, high-emission backup power plants. The volatile nature of electricity demand—shaped by daily human routines, sudden weather changes, and industrial operational shifts—poses a persistent challenge, as many traditional models accumulate compounding errors over extended forecast horizons such as the 720-hour (30-day) window targeted in this study [HA21a].

Moreover, external factors introduce sudden demand shifts, making the forecasting problem highly complex and uncertain. Traditional time series models often struggle to capture rapid, non-linear spikes effectively. Conversely, over-reliance on black-box neural networks without proper validation, baseline comparisons, and interpretability may reduce trust among grid decision-makers [GG+24].

1.3. Challenges in Electricity Load Forecasting

Electricity consumption forecasting presents multiple technical, operational, and data-related challenges:

- **Multiple Seasonalities:** Energy data exhibits complex, overlapping seasonal components—such as intra-day peaks (morning and evening) and weekly trends (weekdays vs. weekends)—that require adaptive modeling techniques [Sha16].
- **Non-linear and Non-stationary Behavior:** Electricity demand frequently experiences non-linear spikes due to unexpected temperature drops or industrial surges. Deep learning networks often perform better in capturing these rapid shifts compared to traditional linear models [MSSJ22a].
- **Data Anomalies and Preprocessing:** Dealing with real-world energy data involves handling missing values, sensor dropouts, and anomalies such as duplicate or missing hours caused by Daylight Saving Time (DST) transitions [HA21a].
- **Model Interpretability vs. Accuracy:** Deep learning architectures like LSTMs offer high accuracy for complex patterns but provide less transparency than classical statistical models like SARIMA, which can affect business adoption and trust [GG+24].
- **Feature Engineering Complexity:** Extracting meaningful temporal and calendar-based features from raw timestamps is critical for model performance but requires domain knowledge and careful integration [MSSJ22a].
- **Loss Function Design:** Standard Mean Squared Error (MSE) penalizes under- and over-prediction symmetrically, which is inappropriate for electricity markets where underestimating peak demand carries higher operational costs. Asymmetric loss functions such as quantile loss can better align model training with real-world cost structures [HA21a].

1.4. Proposed Solutions

This project proposes a comparative evaluation framework leveraging both classical statistical techniques (SARIMA) and modern deep learning architectures (LSTM) to tackle the multifaceted nature of hourly electricity load forecasting. The approach emphasizes:

- **Comparative Methodology:** Establishing a rigorous baseline with SARIMA to capture linear trends and daily seasonal structure, evaluated directly against an LSTM network designed to uncover complex non-linear patterns [GG+24].
- **Peak-Aware Loss Function:** Training the LSTM with quantile loss at $\tau = 0.70$ rather than standard MSE, to systematically reduce under-prediction of consumption peaks that carry the highest operational cost [HA21a].
- **Feature Engineering:** Comprehensive extraction of temporal features (hour of day, day of week, month) to provide contextual signals to the machine learning models [MSSJ22a].
- **Model Evaluation:** Utilizing robust error metrics—Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE)—to quantitatively compare model performance over the 720-hour forecast horizon [HA21a].
- **Knowledge Discovery in Databases (KDD):** Following a structured KDD pipeline to ensure data is systematically selected, cleaned, transformed, and mined for accurate predictions [FPSS96b].

- **Deep Learning Implementation:** Utilizing LSTM networks to inherently address the vanishing gradient problem found in standard RNNs, making them highly effective for learning long-term dependencies in time series data [MSSJ22a].

1.5. Objectives of the Study

The primary objectives of this study are:

- To explore and systematically preprocess a multi-year hourly electricity consumption dataset (PJME, 2002–2018) sourced from PJM Interconnection [GG+24].
- To conduct a comparative analysis of a classical time series model (SARIMA) against a deep learning model (LSTM) over a 720-hour (30-day) forecast horizon.
- To develop a modular, reproducible data pipeline capable of training both models and generating forecasts.
- To assess whether an asymmetric quantile loss function improves peak-load prediction accuracy compared to a standard MSE-trained LSTM baseline.
- To identify best practices for temporal feature engineering in high-frequency time series data.
- To provide actionable insights and high-quality visualizations to aid decision-making for utility operators, delivered through an interactive forecasting application.

1.6. Structure of the Report

To provide a comprehensive overview of the project lifecycle, this report is structured into the following chapters:

- **Chapter 1: Introduction** – Outlines the project overview, problem description, challenges, solutions, and objectives.
- **Chapter 2: Domain Knowledge** – Explores the practical application and the specific domain problem of electricity load forecasting, detailing data acquisition, data quantity, data quality, data relevance, and the identification of outliers and anomalies.
- **Chapter 3: Domain Machine Learning - Packages** – Describes the primary Python packages and frameworks utilized for implementing the machine learning pipeline.
- **Chapter 4: Domain Machine Learning - Algorithms** – Details the theoretical foundations of the traditional (SARIMA) and deep learning (LSTM) models used in this project.
- **Chapter 5: Domain Tools** – Describes the software environment, essential libraries, and version control systems utilized for collaborative development.
- **Chapter 6: Methodology** – Explains the practical application of the Knowledge Discovery in Databases (KDD) process to the chosen dataset.
- **Chapter 7: Development - KDD Database** – Details the initial data loading, storage strategies, and exploratory data analysis (EDA) of the PJM dataset.
- **Chapter 8: Development - KDD Data Transformation** – Covers the critical feature engineering steps, including extracting temporal variables, handling missing hours, and scaling the data for neural networks.

- **Chapter 9: Development - KDD Data Mining** – Outlines the model training phase, hyperparameter tuning, and the fitting of both the SARIMA and LSTM architectures.
- **Chapter 10: Development to Deployment** – Bridges the gap between experimental Jupyter Notebooks and modularizing the Python codebase into executable scripts.
- **Chapter 11: Application Deployment** – Discusses how the forecasting tool is executed, hosted, and utilized for generating real-world predictions.
- **Chapter 12: Monitoring** – Proposes strategies for tracking model drift and performance degradation over time as new electricity consumption data becomes available.
- **Chapter 13: Evaluation** – Presents the quantitative results of the 720-hour forecast, comparing the MAE and RMSE metrics of the SARIMA baseline and the LSTM model.
- **Chapter 14: Validation** – Discusses residual analysis and validates whether the model outputs meet the operational business requirements of utility operators.
- **Chapter 15: Conclusion** – Summarizes the project findings, discusses limitations encountered during development, and provides recommendations for future work.
- **Chapter 16: Application Appendix** – Contains supplementary materials, extended data tables, and additional code snippets.

Part II.

Domain Knowledge - Application

2. Domain Knowledge

2.1. Application

Hourly electricity load forecasting is a critical operational task in modern power systems, enabling grid operators, energy traders, and utility planners to maintain the continuous balance between electricity supply and demand. Because electricity cannot be stored economically at scale, imbalances between generation and consumption result directly in operational costs, reserve penalties, and grid instability. Load forecasting has been a fundamental business problem in the electric power industry for over a century, and the increased market competition, aging infrastructure, and renewable energy integration requirements of recent decades have made accurate load forecasts progressively more essential for the planning and operation of energy systems[HF16].

This application develops and compares two forecasting approaches — a classical Seasonal Autoregressive Integrated Moving Average (SARIMA) model and a Long Short-Term Memory (LSTM) deep learning model applied to the PJM East (PJME) hourly electricity consumption dataset. The dataset is sourced from PJM Interconnection LLC, a regional transmission organisation coordinating the movement of wholesale electricity across 13 U.S. states and the District of Columbia, and is publicly available via the Kaggle platform [PJM24]. It covers hourly electricity demand in megawatts from 2002 to 2018, comprising approximately 145,000 observations, representing one of the most widely benchmarked time series in the energy load forecasting literature. Comparative studies have demonstrated that SARIMA effectively captures seasonal trends in electricity consumption data, while LSTM shows strong potential in modelling complex temporal dependencies though its performance is sensitive to hyperparameter settings and the availability of sufficient training data [PY24].

The PJME series is particularly well-suited to this comparative study due to its 16-year continuous span, its pronounced and stable daily, weekly, and annual seasonal cycles driven by summer cooling and winter heating demand, and its minimal structural breaks over the observation period. Electricity load time series are characterised by non-linearity, non-stationarity, and multiple seasonal patterns properties that make accurate short-term load forecasting a challenging task and motivate the use of both classical statistical and deep learning approaches [PY24]. By evaluating SARIMA and LSTM on identical training and test splits and reporting MAE and RMSE, this study produces reproducible results that are directly comparable to published baselines on the same dataset.

2.2. Domain Problem

Despite the widespread availability of historical electricity consumption data in modern power systems, accurately forecasting hourly load remains a significant operational challenge. Electricity demand is driven by a complex combination of interacting factors including time of day, day of week, seasonal temperature effects, public holidays, and broader socio-economic conditions that vary simultaneously and unpredictably across short and long time horizons. These complexities have made load forecasting a persistent challenge in the electric power industry, with growing renewable integration, aging infrastructure, and increased market competition further intensifying the difficulty of

producing reliable forecasts [HF16].

Classical statistical methods such as ARIMA and its seasonal variant SARIMA have long served as standard tools for this task, offering interpretability and solid performance under stable conditions. However, the main shortcoming of these model families is the linearity assumption for the system being observed — a hypothesis that does not generally hold for electricity load data, which is influenced by weather conditions, human behaviour, and seasonal characteristics that interact in highly non-linear ways [GLA22]. Deep learning methods such as LSTM were specifically proposed to overcome this limitation, but introduce their own challenges in terms of training complexity, data requirements, and interpretability.

This project seeks to bridge this gap by directly comparing SARIMA and LSTM on the PJME hourly dataset under identical experimental conditions. The core challenge lies in evaluating whether the added complexity of deep learning delivers meaningful accuracy gains over a well-configured classical model — producing findings that are honest, reproducible, and grounded in a real-world benchmark used across the energy forecasting literature.

2.3. Data Acquisition

The dataset used in this study is the PJM East hourly electricity consumption series (`PJME_hourly.csv`), sourced from PJM Interconnection LLC—a regional transmission organisation (RTO) coordinating the movement of wholesale electricity across all or parts of 13 U.S. states and the District of Columbia. The data were originally published on PJM’s operational website and are publicly available via the Kaggle platform [PJM24]. The file contains 145,366 observations recorded at hourly intervals from January 2002 to August 2018, structured as two columns: a datetime timestamp and `PJME_MW`, representing electricity demand in megawatts.

Prior to modelling, the dataset requires standard time series preprocessing steps including parsing and indexing the datetime column, verifying hourly frequency regularity, and identifying any missing or duplicate timestamps introduced by Daylight Saving Time transitions. Consistent temporal alignment and the elimination of data irregularities are essential prerequisites for both SARIMA, which assumes a regularly spaced and stationary series, and LSTM, which relies on fixed-length input sequences for training. Energy consumption forecasting for smart grid operations requires careful data preparation, as the quality and structure of the input time series directly influences the reliability of SARIMA, and LSTM model outputs [Dub+21].

The raw data is sourced directly from PJM’s operational monitoring systems and reflects actual metered electricity demand aggregated across the eastern PJM zone. This makes the dataset representative of real-world grid conditions, encompassing millions of residential, commercial, and industrial consumers across a large multi-state region. As a publicly available, well-documented benchmark, it supports reproducible research and enables direct comparison with published forecasting results on the same series[BP23].

For robust training of both SARIMA and LSTM models, a sufficiently long historical span is essential. SARIMA requires enough complete seasonal cycles to reliably estimate seasonal parameters, while LSTM networks benefit from large volumes of sequential data to learn temporal dependencies without overfitting. Historical consumption data spanning multiple years is critical for developing reliable electricity forecasting models, as both statistical and deep learning approaches require sufficient

observations to capture seasonal variation and long-term demand trends [BP23]. The PJME series, covering 16 full years of continuous hourly observations, satisfies this requirement for both modelling approaches.

The dataset was retrieved from the Kaggle platform and was most recently accessed on 03 May 2026: Kaggle Dataset

2.4. Data Quantity

The performance and reliability of time series forecasting models are strongly influenced by the quantity and granularity of the available input data. Effective pattern recognition-including the identification of daily, weekly, and annual seasonality, trend variations, and autocorrelation structures-requires a sufficiently large and temporally consistent dataset. In the context of electricity load forecasting, datasets spanning multiple years at hourly resolution are generally recommended to ensure adequate exposure to seasonal variations and long-term demand dynamics [HF16; BP23].

The PJME hourly dataset used in this study contains 145,366 observations recorded at hourly intervals from January 2002 to August 2018 - a span of approximately 16 years. This volume of data is well-suited to both modelling approaches employed. SARIMA requires multiple complete seasonal cycles to reliably estimate its seasonal parameters, while LSTM networks require large volumes of sequential training data to learn temporal dependencies and avoid overfitting. Comparative analysis of SARIMA and LSTM models on energy consumption data confirms that longer training sequences enable both models to better capture the underlying patterns of the time series, with LSTM in particular benefiting from the availability of extended historical records to learn long-range dependencies [Dub+21].

The hourly granularity of the dataset is equally important. A one-hour resolution preserves the intra-day consumption cycles such as morning ramp-ups and evening peaks that are central to short-term load forecasting. Coarser resolutions such as daily or weekly aggregates would obscure these patterns and reduce the effective information available to both SARIMA and LSTM during training. The absence of missing values and duplicate timestamps, confirmed during data inspection, further ensures that no imputation or gap-filling is required prior to modelling.

With over 145,000 observations spanning 16 full years of continuous hourly recordings, the PJME dataset provides a robust empirical foundation for this comparative study. The large sample size supports reliable parameter estimation for SARIMA and meaningful weight optimisation for the LSTM network, while the multi-year span ensures that both models are evaluated on patterns representative of real-world seasonal and long-term demand behaviour.

2.5. Data Quality

Data quality is a critical determinant of forecasting accuracy, as model performance is highly sensitive to inconsistencies, anomalies, and irregular temporal structures in the input series. High-quality time series data should be complete, consistently structured, and largely free from noise or formatting issues that may distort learned patterns or violate underlying model assumptions [NKHZ21; Li+21].

The PJME hourly dataset underwent a systematic quality assessment prior to modelling. No missing values were identified in either column, and no fully duplicate rows

were present. The Datetime column was successfully parsed into a valid datetime format and sorted chronologically, ensuring compatibility with time series indexing. Electricity demand values in **PJME_MW** range from 14,544 MW to 62,009 MW, with a mean of approximately 32,080 MW consistent with the expected demand profile of a large multi-state regional grid.

Two minor structural irregularities were identified. Four duplicate timestamps were detected, attributable to the autumn Daylight Saving Time (DST) transition, and 30 two-hour gaps were observed during the spring DST transition. Both are well-documented artefacts of DST in hourly energy datasets rather than data collection errors. An IQR-based outlier analysis additionally identified 3,455 observations (2.38%) outside the expected demand range, likely reflecting extreme weather-driven peaks or low-consumption periods such as public holidays.

These preprocessing steps are consistent with best practices in time series preprocessing for energy system data, which have been shown to significantly enhance downstream model reliability [NKHZ21]. Overall, the PJME dataset was determined to be of sufficient quality for training reliable forecasting models, with the identified irregularities being well-understood and manageable within standard preprocessing workflows.

2.6. Data Relevance

Forecast accuracy depends not only on data quality but also on the relevance of the input variables [Arm01]. In this study, the dataset consists of two columns Datetime and **PJME_MW** forming a univariate time series of hourly electricity demand. This structure is standard in time series forecasting, where historical values of the target variable act as the primary input.

The Datetime column provides the temporal framework for modelling seasonality and cyclical behaviour. From it, features such as hour of day, day of week, and month can be derived, capturing recurring demand patterns. The **PJME_MW** column represents electricity demand (in megawatts) and exhibits strong autocorrelation across multiple time scales, making it suitable for both SARIMA and LSTM models.

These two variables are sufficient for the forecasting approaches adopted. SARIMA models the **PJME_MW** series directly, while LSTM utilises lagged sequences and derived temporal features to learn complex patterns [PY24]. Although no exogenous variables (e.g., weather or holidays) are included, the long historical span of the dataset implicitly reflects their aggregate influence, providing a reliable baseline within a univariate forecasting framework.

2.7. Outliers and Anomalies

Outliers and anomalies are observations that deviate significantly from the expected distribution of a time series. If left untreated, they can distort parameter estimation, increase forecast error, and reduce model reliability [BG+21; NKHZ21]. While some anomalies reflect genuine events (e.g., extreme weather or public holidays), others arise from structural issues such as Daylight Saving Time (DST) transitions.

An outlier analysis was conducted on the **PJME_MW** series using the interquartile range (IQR) method, where values outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ are flagged. With $Q1 = 27,573$ MW, $Q3 = 35,650$ MW, and $IQR = 8,077$ MW, the resulting bounds are 15,457.5 MW and 47,765.5 MW. A total of 3,455 observations (2.38%)

were identified as outliers, ranging from 14,544 MW to 62,009 MW. These values are consistent with extreme demand conditions, such as seasonal heating and cooling peaks, and were therefore retained.

In addition, timestamp anomalies were identified, including four duplicate timestamps (autumn DST transition) and 30 two-hour gaps (spring DST transition). These are known artefacts of hourly energy datasets and not measurement errors. No missing values were observed.

Overall, the dataset is of high quality. Outliers were preserved to maintain the full variability of demand patterns, while DST-related irregularities were addressed during preprocessing through timestamp correction and reindexing to a regular hourly frequency, ensuring temporal consistency prior to modelling [NKHZ21].

Part III.

Domain Knowledge - Machine Learning/Algorithms

3. Packages

3.1. Introduction

In this *Energy Demand Forecaster* project, Python packages form the foundation of the data analysis and modelling pipeline. Each library occupies a specific role in the Knowledge Discovery in Databases (KDD) workflow from ingesting and preprocessing high-frequency time series data, to engineering temporal features, training forecasting models, and computing evaluation metrics [Kum22].

Python was selected as the implementation language because of its rich scientific ecosystem, concise syntax, and strong community support in the data science and machine learning domains [Pyt25]. The three core libraries covered in this chapter are:

- **Pandas** — the primary tool for loading, cleaning, and transforming the hourly PJM electricity consumption dataset.
- **NumPy** — the numerical backbone used for vectorised computations, cyclical feature encoding, and metric calculation.
- **Statsmodels** — the statistical modelling library that implements the SARIMA baseline and provides residual diagnostic tools, covered in Section 3.4.

Together, these libraries support a coherent, end-to-end analytics pipeline that ensures data is systematically prepared and reliably consumed by both the SARIMA and LSTM models [FPSS96b].

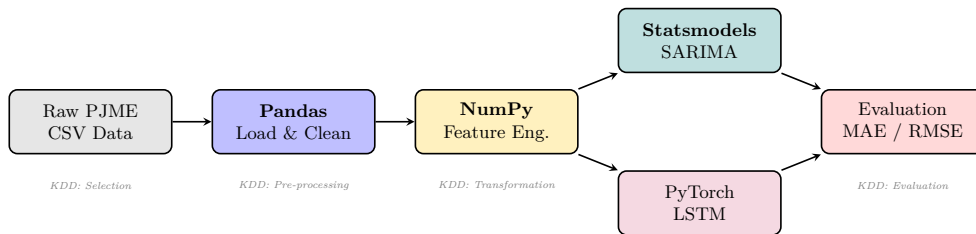


Figure 3.1.: Role of each Python package within the KDD pipeline for hourly electricity load forecasting. Packages covered in this chapter are highlighted; PyTorch (LSTM) is discussed in Chapter 4.

3.2. Pandas

3.2.1. Introduction

Pandas is a versatile and widely-adopted open-source Python library specifically designed for data manipulation and analysis. Developed originally in 2008, it has since become a fundamental tool for data scientists and analysts due to its ability to handle and process large volumes of structured data efficiently. Pandas provides flexible, high-performance data structures such as **Series** (one-dimensional) and **DataFrame** (two-dimensional), which simplify complex data operations and support a wide range

of data types. These features allow users to perform tasks such as filtering, grouping, merging, and pivoting data with ease.

The library is particularly valuable in energy analytics, where datasets often involve multiple dimensions and time-dependent variables. By offering seamless support for handling time series data, Pandas enables analysts to explore trends, seasonality, and cycles in electricity consumption, making it easier to generate meaningful insights. Its intuitive syntax and integration with other Python libraries such as NumPy, Matplotlib, and Scikit-learn create a robust ecosystem for end-to-end data analysis, from data cleaning and transformation to visualisation and predictive modelling.

In the context of electricity load forecasting, Pandas plays a crucial role in preparing and managing data before applying machine learning or statistical models. It helps in detecting missing data, correcting inconsistencies, and performing feature engineering, which directly impacts the quality of forecasts. Overall, Pandas serves as the backbone for handling complex datasets in time series analysis, empowering analysts to make informed decisions based on accurate and timely insights [tea24].

3.2.2. Description

Pandas provides a comprehensive set of tools to work with time series and other structured data efficiently. One of its core strengths lies in the **DataFrame** data structure, which allows data to be indexed and sliced by time, making temporal analysis straightforward. The library supports various datetime formats and offers utilities to convert strings to datetime objects, enabling precise handling of timestamps, time zones, and periods.

Key functionalities of Pandas relevant to this project include resampling, which allows data aggregation at different time frequencies (e.g., converting hourly electricity readings into daily averages), and rolling window calculations, which help in smoothing data or computing moving averages. Additionally, Pandas facilitates handling of missing or irregular time series data through forward filling, backward filling, or interpolation methods, which are crucial for maintaining continuity after Daylight Saving Time (DST) transitions [tea24].

Beyond time series manipulation, Pandas integrates seamlessly with NumPy for numerical operations, Matplotlib for visualisation, and statsmodels or Scikit-learn for statistical and machine learning modelling. This interoperability makes Pandas a central component in the analytics pipeline— from data ingestion and cleaning, through exploratory analysis and feature engineering, to model training and validation. In energy demand forecasting, these capabilities allow analysts to prepare datasets that capture seasonality, long-term trends, and demand anomalies, ultimately improving the accuracy and reliability of predictive models [tea24].

3.2.3. Installation

Pandas is a widely-used Python library for data analysis and manipulation, and installing it is straightforward using Python’s package managers. The most common way is via **pip**, Python’s standard package installer. Before installation, it is advisable to create and activate a virtual environment to isolate dependencies and avoid package conflicts between projects.

To install the latest stable release of Pandas using **pip**, run the following command:

Listing 3.1: Install Pandas using pip

```
pip install pandas
```

If you are using the Anaconda distribution, Pandas can be installed using **conda**:

Listing 3.2: Install Pandas using conda

Datetime (index)	PJME_MW
2002-01-01 01:00:00	32 087.0
2002-01-01 02:00:00	30 714.0
2002-01-01 03:00:00	28 768.0
2002-01-01 04:00:00	27 603.0
... 145,357 rows total ...	
2018-08-03 00:00:00	34 729.0

Figure 3.2.: Structure of the PJME hourly **DataFrame** after loading with Pandas. The **Datetime** column is converted to a **DatetimeIndex**, enabling time-series operations such as **resample()**, **asfreq()**, and **rolling()** throughout the preprocessing pipeline.

```
conda install pandas
```

After installing, import Pandas into your scripts using the standard alias:

Listing 3.3: Import Pandas

```
import pandas as pd
```

To verify the installation and check the version:

Listing 3.4: Check Pandas version

```
import pandas as pd
print(pd.__version__)
```

3.2.4. Example — Description

In this project, Pandas is used to load the PJME hourly dataset (a CSV file containing timestamped electricity consumption readings in megawatts), convert the **Datetime** column to a proper datetime type, and remove duplicate timestamps caused by DST transitions. Setting **Datetime** as the index enables powerful time-based operations such as resampling and frequency alignment using **asfreq()**.

Temporal features including hour of day, day of week, month, and year are extracted from the index to capture cyclical demand patterns. The hourly data is then aggregated to a daily level using **resample()**, and trends are smoothed with a 7-day moving average via **rolling()**, facilitating clear visualisation of weekly and seasonal consumption patterns.

The following code demonstrates the complete preprocessing workflow with Doxygen-style documentation:

Listing 3.5: Loading and Preparing Energy Consumption Data for Time Series Analysis

```
/**
 * @file pjme_analysis.py
 * @brief Perform time series analysis on PJME hourly energy data.
 *
 * Demonstrates loading, cleaning, and preprocessing the hourly
 * electricity demand dataset from PJM Interconnection for use
 * with SARIMA and LSTM forecasting pipelines.
 */
import pandas as pd
```

```

import matplotlib.pyplot as plt
import numpy as np

## @brief Load and preprocess the PJME hourly energy dataset.
# @param file_path Path to the CSV file.
# @return Cleaned DataFrame with a DatetimeIndex.
def load_and_preprocess_data(file_path):
    df = pd.read_csv(file_path)
    df['Datetime'] = pd.to_datetime(df['Datetime'])
    df = df.sort_values('Datetime').reset_index(drop=True)
    df.set_index('Datetime', inplace=True)
    df = df[~df.index.duplicated(keep='first')]
    df = df.asfreq('h', method=None)
    return df

## @brief Resample energy data to a specified frequency.
# @param df DataFrame with DatetimeIndex and PJME_MW column.
# @param freq Resampling frequency string (e.g., 'D' for daily).
# @return Resampled Series with mean energy consumption.
def resample_energy(df, freq='D'):
    return df['PJME_MW'].resample(freq).mean()

## @brief Apply a rolling moving average to smooth the energy series.
# @param series pandas Series of energy values.
# @param window Rolling window size in periods.
# @return Smoothed pandas Series.
def calculate_moving_average(series, window=7):
    return series.rolling(window=window).mean()

if __name__ == "__main__":
    df = load_and_preprocess_data('PJME_hourly.csv')
    daily_series = resample_energy(df, freq='D')
    smoothed = calculate_moving_average(daily_series, window=7)

    plt.figure(figsize=(12, 5))
    plt.plot(daily_series, label='Daily Demand (MW)', alpha=0.5)
    plt.plot(smoothed, label='7-Day Moving Average',
             linestyle='--', color='orange')
    plt.title('PJME Daily Energy Consumption with Moving Average')
    plt.xlabel('Datetime')
    plt.ylabel('Energy Consumption (MW)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

```

3.2.5. Example — Manual

Step 1: Loading the Dataset

The first task is to import the PJME CSV dataset using the `read_csv()` function.

```

import pandas as pd

df = pd.read_csv('PJME_hourly.csv')

```

Explanation:

- `read_csv()` reads a comma-separated values file into a **DataFrame**.

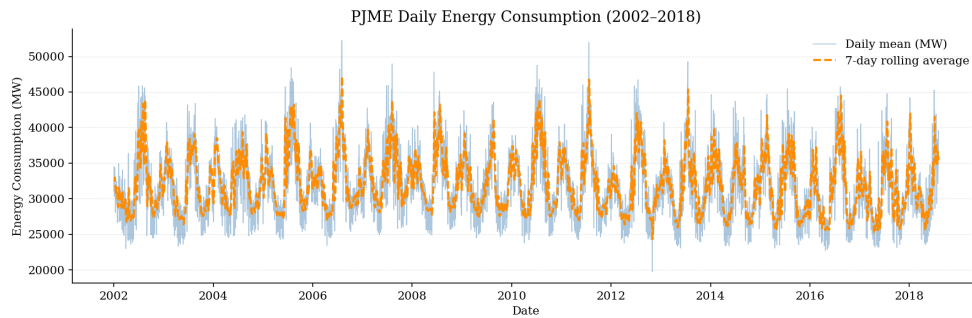


Figure 3.3.: PJME daily mean energy consumption (2002–2018) with a 7-day rolling average overlay (orange dashed). The moving average reduces intra-week noise and reveals clear seasonal cycles: summer cooling peaks in July–August and winter heating surges in January–February. Generated by `scripts/generate_report_figures.py`.

- The result contains a `Datetime` column and a `PJME_MW` column representing hourly consumption in megawatts.

Step 2: Inspecting the Data

Before any transformation, inspect the structure and integrity of the dataset.

```
print(df.head())
print(df.info())
print(df.isnull().sum())
```

Explanation:

- `head()` shows the first five rows for a quick sanity check.
- `info()` reveals column types and non-null counts.
- `isnull().sum()` identifies missing values that require imputation or removal.

Step 3: Datetime Conversion

Convert the date column to Pandas' datetime type to enable time-based operations.

```
df['Datetime'] = pd.to_datetime(df['Datetime'], errors='coerce')
```

Explanation:

- Converts string representations into datetime objects.
- `errors='coerce'` replaces malformed entries with `NaT` (Not a Time), preventing pipeline failures.

Step 4: Handling Missing Values

Drop rows where essential fields are absent.

```
df.dropna(subset=['Datetime', 'PJME_MW'], inplace=True)
```

Explanation:

- Missing timestamps or consumption values would distort time-based aggregations and model training.
- Only affected rows are removed to preserve the remaining data.

Step 5: Setting the Datetime Index

Set the datetime column as the DataFrame index to unlock Pandas' time series methods.

```
df.set_index('Datetime', inplace=True)
df = df[~df.index.duplicated(keep='first')]
df = df.asfreq('h')
```

Explanation:

- A `DatetimeIndex` enables `resample()`, `rolling()`, and date-slice indexing.
- `duplicated()` removes duplicate timestamps introduced by DST clock-back transitions.
- `asfreq('h')` enforces a strict hourly frequency, inserting `NaN` for any gaps.

Step 6: Filtering a Date Range

Select a representative subset for analysis or validation.

```
filtered_df = df['2017-01-01':'2018-08-03']
```

Explanation:

- Pandas date-string slicing is both concise and efficient.
- The final 20 months of data were reserved as the out-of-sample test window in this project.

Step 7: Resampling to Daily Frequency

Aggregate the hourly series to daily mean consumption.

```
daily_consumption = df['PJME_MW'].resample('D').mean()
```

Explanation:

- `resample('D')` groups data by calendar day.
- `mean()` computes the average hourly demand in megawatts, smoothing intra-day noise for trend analysis.

Step 8: Applying a Moving Average

Smooth the daily series to reveal underlying seasonal trends.

```
moving_avg = daily_consumption.rolling(window=7).mean()
```

Explanation:

- `rolling(window=7)` creates a 7-day sliding window.
- `mean()` returns the weekly average demand, which dampens day-to-day volatility and highlights seasonal patterns.

Step 9: Visualisation

Plot the original and smoothed series to support exploratory data analysis.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.plot(daily_consumption, label='Daily Consumption (MW)', alpha=0.5)
plt.plot(moving_avg, label='7-Day Moving Average',
         linestyle='--', color='orange')
plt.xlabel('Datetime')
plt.ylabel('Energy Consumption (MW)')
plt.title('PJME Hourly Energy Consumption Over Time')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Explanation:

- The plot enables visual inspection of daily, weekly, and seasonal demand patterns.
- Analysts can identify summer cooling peaks, winter heating surges, and long-term consumption trends more easily with the smoothed overlay.

3.2.6. Conclusion

This walkthrough demonstrates how Pandas supports every phase of time series preprocessing for energy demand forecasting:

- Loading raw CSV data and converting timestamps.
- Removing DST-induced duplicate hours and enforcing a regular frequency.
- Resampling and smoothing to expose seasonal demand structure.
- Filtering date ranges for train/test splitting.

Mastering these operations is essential for building a reliable forecasting pipeline, as data quality directly constrains model accuracy.

3.2.7. Further Readings

- McKinney, W., 2022. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. 3rd ed. Sebastopol, CA: O'Reilly Media.
- The Pandas Development Team, 2024. *Pandas Documentation*. [online] Available at: <https://pandas.pydata.org/docs/> [Accessed 17 Jun. 2025].
- Hyndman, R.J. and Athanasopoulos, G., 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts [HA21a].

3.3. NumPy

3.3.1. Introduction

NumPy is the fundamental package for scientific computing in Python. It provides the `ndarray` data structure—an N-dimensional, typed array that supports fast,

memory-efficient operations—alongside a comprehensive library of mathematical functions covering linear algebra, Fourier analysis, random number generation, and more [Har+20].

In the context of this energy demand forecasting project, NumPy underpins virtually every numerical operation in the pipeline. It is used to encode temporal features as cyclical signals, to cast data into the typed arrays required by PyTorch, to compute RMSE from prediction arrays, and to ensure reproducibility through seeded random number generation. Because NumPy operations are implemented in optimised C code and leverage vectorisation rather than Python-level loops, they handle the scale of the PJME dataset—over 145,000 hourly readings spanning 2002 to 2018—with negligible overhead [Har+20; Idr15].

3.3.2. Description

At the heart of NumPy is the `ndarray` object, which stores homogeneous data in a contiguous block of memory. Unlike Python lists, `ndarray` objects consume less memory, support broadcasting (element-wise operations between arrays of compatible but different shapes), and benefit from single-instruction multiple-data (SIMD) CPU optimisations. These properties are critical in machine learning pipelines where large tensors must be constructed and manipulated efficiently [Har+20].

The following NumPy capabilities are directly exercised in this project:

- **Trigonometric functions** (`np.sin`, `np.cos`, `np.pi`): used in `feature_engineering.py` to encode the hour of day, day of week, and month of year as cyclical sin/cos pairs, eliminating the artificial discontinuity between hour 23 and hour 0 that a raw integer representation would create.
- **Type casting** (`np.float32`, `np.float64`, `np.int64`): PyTorch expects `float32` tensors; NumPy's `astype(np.float32)` converts the Pandas DataFrame columns into the correct dtype before the data is consumed by the LSTM training loop.
- **Mathematical functions** (`np.sqrt`, `np.mean`, `np.where`): `evaluation.py` computes RMSE as `np.sqrt(mean_squared_error(y_true, y_pred))`, while `np.where` guards the min-max scaler against division by zero on degenerate constant columns.
- **Index generation** (`np.arange`): the sliding-window dataset class in `lstm_model.py` uses `np.arange` to precompute the array of valid start indices for each input window, enabling efficient random access during mini-batch training.
- **Reproducibility** (`np.random.seed`): seeding NumPy's random number generator alongside PyTorch and Python's built-in `random` module ensures that weight initialisation and data shuffling are fully deterministic across runs.

3.3.3. Installation

NumPy can be installed independently or as part of a scientific Python distribution such as Anaconda. It is typically listed as a dependency of higher-level libraries such as Pandas and Scikit-learn and is therefore often already present in the project environment.

Listing 3.6: Install NumPy using pip

```
pip install numpy
```

To confirm installation and verify the version:

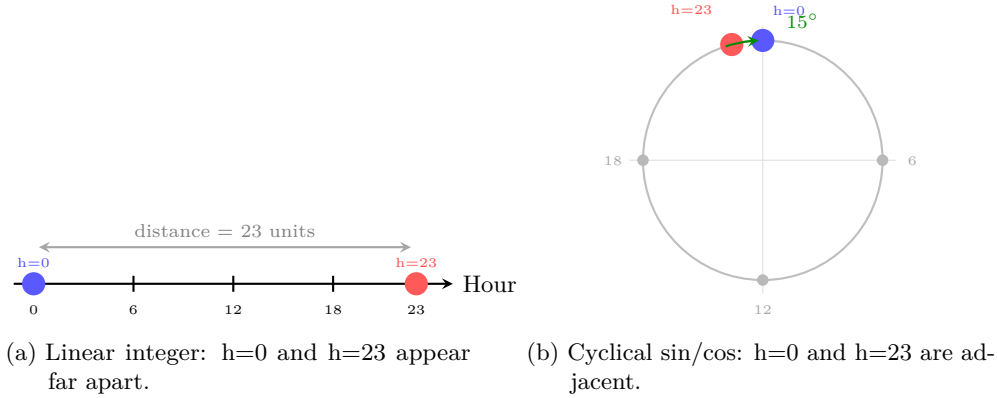


Figure 3.4.: Cyclical encoding maps discrete hour integers onto a unit circle via $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$. Hours 0 and 23 are only 15° apart on the circle (b), preserving the temporal adjacency that the raw integer representation (a) destroys.

Listing 3.7: Check NumPy version

```
import numpy as np
print(np.__version__)
```

This ensures consistency across development and deployment environments, particularly important when reproducibility of numerical results is required.

3.3.4. Example — Description

The most distinctive NumPy contribution in this project is the *cyclical encoding* of temporal features implemented in `src/feature_engineering.py`. Raw integer representations of time (e.g., hour 0–23) are discontinuous: the network would perceive midnight (hour 0) and 11 pm (hour 23) as maximally far apart, even though they are one hour from each other. Encoding each period as a pair of sine and cosine values maps it onto a unit circle, so the representation wraps smoothly and the angular distance between any two hours reflects their true temporal proximity.

The same technique is applied to day of week (period 7) and month of year (period 12), giving the LSTM a continuous and geometrically correct representation of the calendar structure of electricity demand [MSSJ22a].

The following code replicates the exact logic from `feature_engineering.py`:

Listing 3.8: Cyclical Temporal Feature Encoding with NumPy

```
import numpy as np
import pandas as pd

## @brief Encode a periodic integer feature as a sin/cos pair.
# @param values Array of integer values (e.g., hour 0-23).
# @param period The cycle length (24 for hours, 7 for days, 12 for months).
# @return Tuple of (sin_values, cos_values) arrays.
def encode_cyclical(values, period):
    radians = 2 * np.pi * values / period
    return np.sin(radians), np.cos(radians)

# Load hourly data and extract temporal components
df = pd.read_csv('PJME_hourly.csv', parse_dates=['Datetime'],
                 index_col='Datetime')
```

```

hour      = df.index.hour.values          # 0 - 23
dayofweek = df.index.dayofweek.values     # 0 (Mon) - 6 (Sun)
month      = df.index.month.values - 1    # zero-based: 0 - 11

df['hour_sin'], df['hour_cos'] = encode_cyclical(hour, 24)
df['dayofweek_sin'], df['dayofweek_cos'] = encode_cyclical(dayofweek, 7)
df['month_sin'], df['month_cos'] = encode_cyclical(month, 12)

print(df[['hour_sin', 'hour_cos',
           'dayofweek_sin', 'dayofweek_cos',
           'month_sin', 'month_cos']].head())

```

3.3.5. Example — Manual

Step 1: Import NumPy

```
import numpy as np
```

The alias `np` is the community standard and is expected by most downstream libraries.

Step 2: Create a Sample Consumption Array

```

# Simulated hourly PJME consumption (MW) for one day
consumption = np.array([
    28000, 27500, 26800, 26200, 26000, 26500,
    28000, 31000, 34000, 35500, 35000, 34500,
    34000, 34200, 34800, 35200, 36000, 37500,
    38000, 36500, 34000, 32000, 30500, 29000
], dtype=np.float64)

```

Specifying `dtype=np.float64` ensures consistent precision and compatibility with downstream numerical operations.

Step 3: Compute Descriptive Statistics

```

print(f"Mean:    {np.mean(consumption):.1f} MW")
print(f"Std:     {np.std(consumption):.1f} MW")
print(f"Min:     {np.min(consumption):.1f} MW")
print(f"Max:     {np.max(consumption):.1f} MW")

```

Explanation:

- NumPy's vectorised statistics operate on the entire array in a single pass, significantly faster than Python-loop equivalents.

Step 4: Cyclical Encoding of the Hour

```

hours = np.arange(24)          # 0 to 23
radians = 2 * np.pi * hours / 24
hour_sin = np.sin(radians)
hour_cos = np.cos(radians)

print("Hour 0 -> sin:", round(hour_sin[0], 4),
      "cos:", round(hour_cos[0], 4))
print("Hour 12 -> sin:", round(hour_sin[12], 4),
      "cos:", round(hour_cos[12], 4))

```

```
print("Hour 23 -> sin:", round(hour_sin[23], 4),
      "cos:", round(hour_cos[23], 4))
```

Explanation:

- Hours 0 and 23 produce similar cosine values, reflecting their true proximity in time—something a raw integer would not capture.
- `np.arange(24)` generates the integer sequence without a Python loop, matching the vectorised style expected throughout the pipeline.

Step 5: Min-Max Normalisation

```
data_min = np.min(consumption)
data_max = np.max(consumption)
data_rng = data_max - data_min

# Guard against zero-range columns (constant features)
safe_rng = np.where(data_rng == 0, 1.0, data_rng)

normalised = (consumption - data_min) / safe_rng
print("Normalised range:", normalised.min(), "to", normalised.max())
```

Explanation:

- `np.where` applies a conditional element-wise replacement, preventing division by zero on any column where all values are identical—a pattern adopted directly from `lstm_model.py`.
- The result maps every value to $[0, 1]$, which is the expected input range of the LSTM network.

Step 6: RMSE Computation

```
# Simulated actual and predicted demand (MW)
actual = np.array([35000, 36000, 34500, 33000, 37000], dtype=np.float64)
predicted = np.array([34200, 35500, 35000, 33800, 36200], dtype=np.float64)

rmse = np.sqrt(np.mean((actual - predicted) ** 2))
mae = np.mean(np.abs(actual - predicted))

print(f"RMSE: {rmse:.1f} MW")
print(f"MAE: {mae:.1f} MW")
```

Explanation:

- The subtraction, squaring, and mean are all vectorised—NumPy applies them across the entire array with a single C-level loop.
- This matches the pattern in `evaluation.py`, where `np.sqrt(mean_squared_error(y_true, y_pred))` is used to compute RMSE after inverse-scaling the LSTM outputs.

3.3.6. Conclusion

NumPy is the numerical backbone of the Energy Demand Forecaster. Whether encoding temporal features as cyclical signals to preserve periodicity, normalising consumption values to the $[0, 1]$ range required by the LSTM, computing evaluation metrics such as MAE and RMSE, or seeding the random number generator for

reproducibility—every operation relies on NumPy’s fast, vectorised array primitives. Its tight integration with Pandas, PyTorch, and Scikit-learn makes it an indispensable component of any serious time series analysis pipeline [Har+20].

3.3.7. Further Readings

- Harris, C.R. et al., 2020. Array programming with NumPy. *Nature*, 585, pp.357–362 [Har+20].
- Idris, I., 2015. *NumPy: Beginner’s Guide*. Packt Publishing [Idr15].
- NumPy Development Team, 2025. *NumPy User Guide*. [online] Available at: <https://numpy.org/doc/stable/user/> [Accessed May 2026].

3.4. Statsmodels

3.4.1. Introduction

Statsmodels is an open-source Python library dedicated to the estimation, testing, and diagnostic analysis of statistical models. First presented at the SciPy conference in 2010, it has since become a widely used reference library for econometric and time series modelling in Python. It provides a comprehensive set of classical statistical procedures that complement the machine learning tools available in Scikit-learn [SP10]. In this *Energy Demand Forecaster* project, Statsmodels is the main library used for the SARIMA baseline model. Its `tsa.statespace.sarimax` submodule implements SARIMA through a stable state-space representation. The model parameters are estimated using maximum likelihood estimation (MLE), supported internally by the Kalman filter.

Beyond model fitting, Statsmodels also supports diagnostic analysis. In this project, the Ljung-Box residual diagnostic test is used through `statsmodels.stats.diagnostic.acorr_ljungbox`. The ACF plot utility is used through `statsmodels.graphics.tsaplots.plot_acf`. Both tools are used in `evaluation.py` to validate whether the fitted SARIMA model has adequately captured the autocorrelation structure of the PJME electricity demand series [SP10].

The choice of Statsmodels over a manual ARIMA implementation is justified by the reliability of its Kalman-filter backend, its built-in support for out-of-sample forecasting with confidence intervals, and its native integration with Pandas `Series` and `DatetimeIndex` objects.

3.4.2. Description

The core of Statsmodels’ time series functionality relevant to this project is the `SARIMAX` class, which implements the general Seasonal AutoRegressive Integrated Moving Average with exogenous regressors model. When no exogenous variables are supplied, it reduces to a pure SARIMA model. The class accepts two key parameter tuples [HA21a; SP10]:

- `order = (p, d, q)` — the non-seasonal AR order p , differencing degree d , and MA order q .
- `seasonal_order = (P, D, Q, s)` — the seasonal AR order P , seasonal differencing D , seasonal MA order Q , and the seasonal period s (set to 24 for hourly data with daily seasonality).

Once instantiated, calling `model.fit()` returns a `SARIMAXResults` object that encapsulates:

- **Fitted parameters** accessible via `fit.params`.
- **Residuals** accessible via `fit.resid`, used to check whether the model has absorbed all systematic autocorrelation.
- **A model summary** via `fit.summary()`, reporting parameter estimates, standard errors, AIC, BIC, and Ljung-Box statistics.
- **Out-of-sample forecasts** via `fit.get_forecast(steps=720)`, returning a `PredictionResults` object from which the point forecast (`predicted_mean`) and confidence interval (`conf_int(alpha=0.05)`) can be extracted as Pandas objects.

Two additional flags used in `sarima_model.py` deserve explanation: `enforce_stationarity=False` and `enforce_invertibility=False`. By default, Statsmodels constrains parameter estimation to the stationary and invertible regions of the parameter space. On hourly load data these constraints cause the MLE optimizer to fail to converge; disabling them allows the optimizer to explore a wider region and consistently reach a solution.

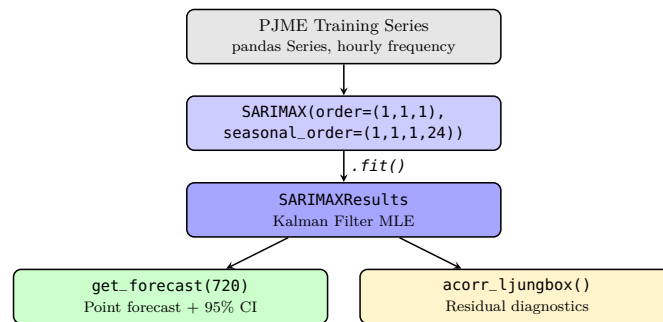


Figure 3.5.: Statsmodels SARIMA workflow used in this project. A `SARIMAX` object is instantiated with the model order, fitted via Kalman-filter MLE, and the resulting `SARIMAXResults` object provides both the 720-step point forecast with confidence interval and the residual diagnostic statistics.

3.4.3. Installation

Statsmodels can be installed via `pip` or `conda`. It depends on NumPy, SciPy, and Pandas, which are typically already present in a data science environment.

Listing 3.9: Install Statsmodels using `pip`

```
pip install statsmodels
```

Listing 3.10: Install Statsmodels using `conda`

```
conda install statsmodels
```

To verify the installation:

Listing 3.11: Check Statsmodels version

```
import statsmodels
print(statsmodels.__version__)
```

3.4.4. Example — Description

The following code replicates the exact SARIMA pipeline used in `src/sarima_model.py` and `src/evaluation.py`. It covers model fitting, 720-step (30-day) forecasting with a 95% confidence interval, and residual diagnostics via the Ljung-Box test.

Listing 3.12: SARIMA Fitting and Forecasting with Statsmodels

```
/**
 * @file sarima_pipeline.py
 * @brief Fit a SARIMA model to hourly PJME data and generate a 30-day
 *        forecast.
 *
 * Replicates the pipeline in src/sarima_model.py and src/evaluation.py.
 * Order (1,1,1)(1,1,1,24) was selected by minimising AIC on a held-out
 * validation window.
 */

import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX, SARIMAXResults
from statsmodels.stats.diagnostic import acorr_ljungbox

## @brief Fit SARIMA(1,1,1)(1,1,1,24) on a training series.
# @param train pandas Series with a DatetimeIndex at hourly frequency.
# @return      Fitted SARIMAXResults object.
def fit_sarima(train):
    model = SARIMAX(
        train,
        order=(1, 1, 1),
        seasonal_order=(1, 1, 1, 24),
        enforce_stationarity=False,
        enforce_invertibility=False,
    )
    return model.fit(dispatch=False, maxiter=200)

## @brief Generate a point forecast and 95% confidence interval.
# @param fit      Fitted SARIMAXResults object.
# @param steps    Number of hourly steps to forecast (default 720 = 30
#                 days).
# @return         Tuple of (point_forecast Series, conf_int DataFrame).
def forecast(fit, steps=720):
    result = fit.get_forecast(steps=steps)
    point = result.predicted_mean
    conf_int = result.conf_int(alpha=0.05)
    return point, conf_int

## @brief Run the Ljung-Box test on model residuals.
# @param fit      Fitted SARIMAXResults object.
# @param lags     Number of lags to test (default 24 = one daily cycle).
# @return         Dictionary with test statistic and p-value.
def residual_diagnostics(fit, lags=24):
    residuals = pd.Series(fit.resid).dropna()
    lb = acorr_ljungbox(residuals, lags=[lags], return_df=True)
    return {
        "ljung_box_stat": float(lb["lb_stat"].iloc[0]),
        "ljung_box_pvalue": float(lb["lb_pvalue"].iloc[0]),
    }

# ===== Main Workflow =====
```

```

if __name__ == "__main__":
    # Load and prepare the PJME series
    df = pd.read_csv("PJME_hourly.csv", parse_dates=["Datetime"],
                    index_col="Datetime")
    df = df[~df.index.duplicated(keep="first")].asfreq("h")

    # Train/test split: last 720 hours as the test window
    train = df["PJME_MW"].iloc[:-720]
    test = df["PJME_MW"].iloc[-720:]

    # Fit model
    fit = fit_sarima(train)
    print(fit.summary())

    # Forecast
    point, conf_int = forecast(fit, steps=720)
    print(point.head())

    # Residual diagnostics
    diag = residual_diagnostics(fit, lags=24)
    print(f"Ljung-Box stat: {diag['ljung_box_stat']:.3f}")
    print(f"Ljung-Box p-value: {diag['ljung_box_pvalue']:.4f}")
    # p > 0.05 -> residuals are indistinguishable from white noise

```

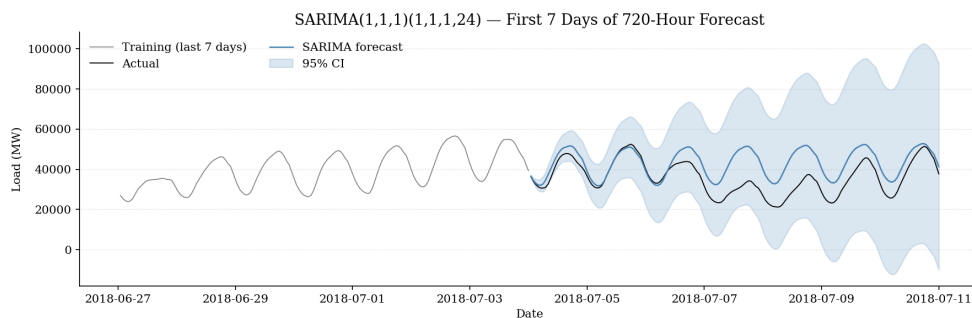


Figure 3.6.: First 7 days (168 hours) of the SARIMA(1,1,1)(1,1,1,24) 720-hour forecast (blue) versus actual PJME demand (black), with a 95 % confidence interval (shaded). Zooming into the opening week reveals that SARIMA captures the daily consumption cycle and broad load level, even though accuracy deteriorates substantially over the full 30-day horizon (MAE = 11,662 MW). Generated by `scripts/generate_report_figures.py`.

3.4.5. Example — Manual

Step 1: Import the SARIMAX Class

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

Explanation:

- SARIMAX is the unified class for ARIMA, SARIMA, and their exogenous-regressor extensions within Statsmodels' state-space framework.

Step 2: Prepare the Training Series

```
import pandas as pd

df = pd.read_csv("PJME_hourly.csv", parse_dates=["Datetime"],
                 index_col="Datetime")
df = df[~df.index.duplicated(keep="first")].asfreq("h")

train = df["PJME_MW"].iloc[:-720]  # everything except the last 30 days
```

Explanation:

- Statsmodels requires a regular `DatetimeIndex`; the `asfreq("h")` call enforces hourly regularity and surfaces any gaps as `NaN` before fitting.
- DST duplicate removal (`duplicated`) prevents ambiguous timestamps from corrupting the Kalman filter.

Step 3: Specify and Fit the Model

```
model = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 24),
    enforce_stationarity=False,
    enforce_invertibility=False,
)
fit = model.fit(dispatch=False, maxiter=200)
```

Explanation:

- `order=(1,1,1)` specifies one AR term, one differencing pass, and one MA term.
- `seasonal_order=(1,1,1,24)` adds a seasonal layer with period $s = 24$ hours, capturing the dominant daily consumption cycle.
- `enforce_stationarity=False` and `enforce_invertibility=False` are required on hourly load data to prevent premature convergence failures in the MLE optimizer.
- `dispatch=False` suppresses per-iteration solver output for cleaner logs.

Step 4: Inspect the Fitted Model

```
print(fit.summary())
print("AIC:", fit.aic)
print("BIC:", fit.bic)
```

Explanation:

- `summary()` prints a comprehensive table including estimated coefficients, standard errors, z -statistics, and information criteria.
- AIC and BIC are used for model selection: lower values indicate a better balance between fit quality and model complexity.

Step 5: Generate a 720-Hour Forecast

```
result = fit.get_forecast(steps=720)
point = result.predicted_mean # pandas Series
conf_int = result.conf_int(alpha=0.05) # DataFrame with lower_ci / upper_ci
```

Explanation:

- `get_forecast(steps=720)` performs a true out-of-sample forecast for 720 hours (30 days) beyond the training window.
- `predicted_mean` is a Pandas Series with the forecast index aligned to the original `DatetimeIndex`, making plotting and metric computation straightforward.
- `conf_int(alpha=0.05)` returns the 95% confidence interval as a two-column DataFrame, used to communicate forecast uncertainty to stakeholders.

Step 6: Residual Diagnostics — Ljung-Box Test

```
from statsmodels.stats.diagnostic import acorr_ljungbox

residuals = pd.Series(fit.resid).dropna()
lb = acorr_ljungbox(residuals, lags=[24], return_df=True)

print(f"Ljung-Box stat : {lb['lb_stat'].iloc[0]:.3f}")
print(f"Ljung-Box p-value: {lb['lb_pvalue'].iloc[0]:.4f}")
```

Explanation:

- The Ljung-Box test checks whether residual autocorrelations at the specified lags are jointly zero.
- A p-value above 0.05 indicates that residuals are statistically indistinguishable from white noise, confirming that the model has captured all systematic temporal structure in the data [HA21a].
- Testing at lag 24 is natural for hourly data because it probes whether any daily seasonal pattern remains unexplained.

Step 7: ACF Plot of Residuals

```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

fig, ax = plt.subplots(figsize=(10, 4))
plot_acf(residuals, lags=48, ax=ax)
ax.set_title("Residual ACF -- SARIMA(1,1,1)(1,1,1,24)")
fig.tight_layout()
plt.show()
```

Explanation:

- `plot_acf` visualises residual autocorrelations at each lag, with shaded 95% confidence bands.
- Spikes outside the bands at lag 24 or 48 would indicate that the seasonal component is inadequately modelled, prompting a revision of the seasonal order P or Q .

- Testing 48 lags (two full daily cycles) is standard practice for hourly energy data to confirm that neither the first nor the second harmonic of the daily pattern persists in the residuals.

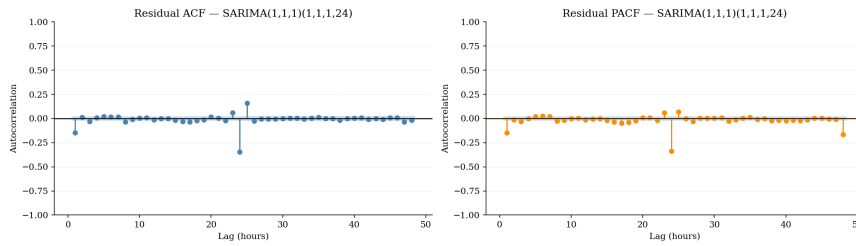


Figure 3.7.: Residual ACF (left) and PACF (right) of the SARIMA(1,1,1)(1,1,1,24) model at lags 1–48. Lag0 is omitted so the y-axis rescales to show the actual residual structure. Bars exceeding the shaded 95 % confidence bands indicate residual autocorrelation that the model has not fully absorbed; any spikes near lags 24 or 48 would signal unresolved daily seasonality. Generated by `scripts/generate_report_figures.py`.

3.4.6. Conclusion

Statsmodels provides the complete statistical infrastructure for the SARIMA baseline in this project. The `SARIMAX` class handles model specification, Kalman-filter MLE fitting, and out-of-sample forecasting with confidence intervals through a single, coherent interface. The diagnostic tools — Ljung-Box test and ACF plot — close the modelling loop by verifying that residuals are free of autocorrelation, a necessary condition for the confidence intervals to be statistically valid [SP10]. Together, these capabilities make Statsmodels the natural choice for the classical time series baseline against which the deep learning model is benchmarked.

3.4.7. Further Readings

- Seabold, S. and Perktold, J., 2010. Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*, pp.92–96 [SP10].
- Hyndman, R.J. and Athanasopoulos, G., 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts. Available at: <https://otexts.com/fpp3/> [HA21a].
- Statsmodels Development Team, 2025. *Statsmodels Documentation — Time Series Analysis*. [online] Available at: <https://www.statsmodels.org/stable/tsa.html> [Accessed May 2026].

4. Data Mining Algorithm

4.1. Introduction

The data mining phase of the KDD process applies predictive models to the prepared, transformed dataset in order to extract patterns and generate actionable forecasts [FPSS96b]. In this project, two complementary algorithms address the 720-hour (30-day) electricity load forecasting task on the PJME dataset: SARIMA, a classical statistical model that exploits the explicit autocorrelation structure of the load series, and LSTM, a deep learning model that learns non-linear temporal dependencies from a rich set of engineered temporal features.

Together they form a comparative framework aligned with established best practice in energy forecasting research: a well-fitted statistical baseline is compared against a deep learning challenger to determine whether the additional modelling complexity translates into improved forecast accuracy [GG+24]. Both models are evaluated on identical train/test splits using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). Table 4.1 summarises the principal characteristics of each approach; detailed results appear in Chapter 14.

Table 4.1.: High-level comparison of the two forecasting models used in this project.

Property	SARIMA(1,1,1)(1,1,1,24)	LSTM-q70
Model family	Classical statistical	Deep learning
Parameter estimation	Kalman filter MLE	Adam back-propagation
Input representation	Raw univariate series	168-h window, 9 features
Output	720-h forecast + 95 % CI	720-h forecast (scaled)
Loss function	Log-likelihood (MLE)	Pinball ($\tau = 0.70$)
Interpretability	High (explicit coefficients)	Low (weight matrices)
Test MAE	11,662 MW	3,307 MW
Test RMSE	12,932 MW	4,156 MW

4.2. SARIMA (Seasonal AutoRegressive Integrated Moving Average)

1. Description

SARIMA is a classical parametric model for stationary, seasonally structured univariate time series. It generalises the ARIMA model by adding a seasonal layer of autoregressive and moving-average terms at lag multiples of the seasonal period s , making it well-suited to data with periodic patterns such as the daily electricity consumption cycle present in the PJME dataset [HA21a].

A SARIMA(p, d, q)(P, D, Q) $_s$ model is characterised by six integer orders and one period:

- p : non-seasonal AR order — number of lagged values of y_t in the model.

- d : non-seasonal differencing degree — number of times the series is first-differenced to remove a trend.
- q : non-seasonal MA order — number of lagged white-noise errors ε_{t-i} in the model.
- P, D, Q : seasonal counterparts of the above, operating at lag multiples of s .
- s : the seasonal period (set to $s = 24$ for hourly data with a dominant daily consumption cycle).

Using the backshift operator B (where $By_t = y_{t-1}$ and $B^s y_t = y_{t-s}$), the general SARIMA(p, d, q)(P, D, Q) $_s$ model is written compactly as [HA21a]:

$$\underbrace{\phi_p(B) \Phi_P(B^s)}_{\text{AR components}} \underbrace{(1-B)^d (1-B^s)^D}_{\text{differencing}} y_t = \underbrace{\theta_q(B) \Theta_Q(B^s)}_{\text{MA components}} \varepsilon_t \quad (4.1)$$

where $\phi_p(B) = 1 - \phi_1 B - \dots - \phi_p B^p$, $\Phi_P(B^s) = 1 - \Phi_1 B^s - \dots - \Phi_P B^{Ps}$, and $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$ is a white-noise innovation.

For the specific configuration SARIMA(1, 1, 1)(1, 1, 1) $_{24}$ deployed in this project, Equation (4.1) reduces to:

$$(1 - \phi_1 B)(1 - \Phi_1 B^{24})(1 - B)(1 - B^{24}) y_t = (1 + \theta_1 B)(1 + \Theta_1 B^{24}) \varepsilon_t \quad (4.2)$$

One non-seasonal differencing pass removes the slowly varying trend; one seasonal differencing pass at lag 24 removes the systematic daily pattern; single AR and MA terms at both scales then model the residual stochastic structure [HA21a]. Model parameters $(\phi_1, \theta_1, \Phi_1, \Theta_1, \sigma^2)$ are estimated jointly by maximum likelihood via a Kalman-filter state-space representation [SP10].

2. Application

SARIMA is applied across a broad range of fields wherever time series exhibit both trend and periodic components. In the energy sector it has been used for hourly, daily, and monthly electricity demand forecasting and serves as the standard statistical baseline against which newer approaches are benchmarked [Sha16]. Beyond energy, SARIMA underpins economic forecasting (GDP growth, inflation, retail sales), weather prediction, and epidemiological modelling of disease incidence [HA21a].

The model family is particularly valued in operational settings where interpretability matters: each estimated coefficient $(\phi_1, \theta_1, \Phi_1, \Theta_1)$ has a clear statistical interpretation, and the confidence interval produced alongside the point forecast communicates uncertainty in a form familiar to grid operators and policy-makers [GG+24].

Relevance to Project

SARIMA was selected as the classical statistical baseline for three reasons. First, hourly electricity consumption exhibits a strong, stable 24-hour periodicity that aligns directly with the seasonal order $(P, D, Q)_s$ at $s = 24$, enabling the model to capture the dominant intra-day demand cycle without manual feature engineering [Sha16]. Second, Statsmodels' Kalman-filter implementation produces statistically grounded confidence intervals, enabling risk-aware decisions by grid operators. Third, a well-fitted SARIMA baseline makes the deep learning comparison meaningful: if LSTM cannot outperform SARIMA, the additional modelling complexity is unjustified [GG+24].

The order (1, 1, 1)(1, 1, 1, 24) was selected by minimising AIC on a held-out validation window. The flags `enforce_stationarity=False` and `enforce_invertibility=False` were required because the MLE optimiser consistently failed to converge on hourly load data under the default parameter-space constraints [SP10].

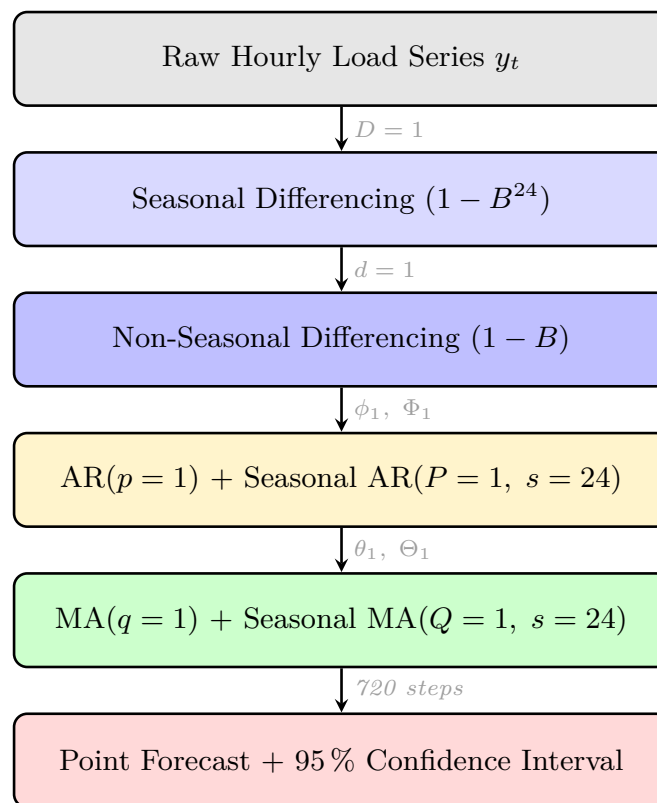


Figure 4.1.: Layered transformations applied by SARIMA(1,1,1)(1,1,1)₂₄. Each differencing step removes a distinct source of non-stationarity before the AR and MA terms model the remaining stochastic structure.

Requirements

Valid SARIMA modelling presupposes the following conditions [HA21a]:

- **Regular time index:** the series must have a fixed, gap-free frequency. `asfreq("h")` enforces strict hourly regularity and surfaces any gaps as `NaN` before fitting.
- **No duplicate timestamps:** DST clock-back transitions introduce repeated hours that corrupt the Kalman filter state; these are removed with `duplicated(keep="first")`.
- **Stationarity after differencing:** the chosen orders $d = 1$ and $D = 1$ must render the differenced series covariance-stationary. This is verified via the Augmented Dickey-Fuller test during model selection.
- **Sufficient history:** at least $2 \times s = 48$ observations are needed to estimate seasonal parameters; in practice, over 144,000 training hours are available.

Hyperparameters

Table 4.2.: SARIMA hyperparameters used in this project.

Parameter	Value	Description
p (AR order)	1	One non-seasonal autoregressive lag
d (differencing)	1	One non-seasonal first-difference
q (MA order)	1	One non-seasonal moving-average term
P (seasonal AR)	1	One seasonal autoregressive lag
D (seasonal differencing)	1	One seasonal difference at lag 24
Q (seasonal MA)	1	One seasonal moving-average term
s (seasonal period)	24	Daily cycle in hourly data
<code>enforce_stationarity</code>	<code>False</code>	Permits boundary-region MLE convergence
<code>enforce_invertibility</code>	<code>False</code>	Permits boundary-region MLE convergence
<code>maxiter</code>	200	Maximum optimiser iterations

Input and Output for SARIMA

Input

The SARIMA model accepts a single univariate `pandas.Series` with a `DatetimeIndex` at strict hourly frequency. The training series spans from 2002 to approximately July 2018, comprising around 144,672 hourly observations after the final 720-hour test window has been withheld. No additional covariates are used: SARIMA learns seasonal and trend structure entirely from the historical load values [HA21a].

Output

Calling `fit.get_forecast(steps=720)` on the fitted `SARIMAXResults` object returns a `PredictionResults` object from which two outputs are extracted:

- **Point forecast:** a `pandas.Series` of 720 hourly load predictions (in MW), with a `DatetimeIndex` aligned to the test period.
- **95 % confidence interval:** a two-column `DataFrame` (lower and upper bounds), used to visualise forecast uncertainty as shown in Figure 3.6.

7. Sample Code

Listing 4.1: SARIMA: fitting and 720-step forecast

```
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Load and prepare the hourly series
df = pd.read_csv("PJME_hourly.csv", parse_dates=["Datetime"],
                 index_col="Datetime")
df = df[~df.index.duplicated(keep="first")].asfreq("h")
train = df["PJME_MW"].iloc[:-720] # hold out last 720 h for testing
test = df["PJME_MW"].iloc[-720:]

# Fit SARIMA(1,1,1)(1,1,1,24)
model = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 24),
                 enforce_stationarity=False, enforce_invertibility=False)
fit = model.fit(dispatch=False, maxiter=200)

# Generate 720-step forecast with 95% CI
result = fit.get_forecast(steps=720)
forecast = result.predicted_mean
ci = result.conf_int(alpha=0.05)

# Evaluate
mae = (test - forecast).abs().mean()
rmse = ((test - forecast) ** 2).mean() ** 0.5
print(f"MAE: {mae:.1f} MW")
print(f"RMSE: {rmse:.1f} MW")
```

8. SARIMA Forecasting Pipeline

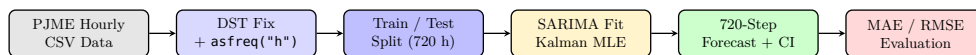


Figure 4.2.: End-to-end SARIMA forecasting pipeline from raw CSV ingestion to quantitative evaluation.

Further Reading for SARIMA

- Hyndman, R.J. and Athanasopoulos, G., 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts [HA21a].
- Seabold, S. and Perktold, J., 2010. Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*, pp.92–96 [SP10].
- Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M., 2015. *Time Series Analysis: Forecasting and Control*. 5th ed. Hoboken, NJ: Wiley.

4.3. LSTM

1. Description

Long Short-Term Memory (LSTM) networks are a specialised variant of Recurrent Neural Networks (RNNs) introduced to address the vanishing gradient problem that

prevents standard RNNs from learning dependencies separated by many time steps [MSSJ22a]. The key architectural innovation is a dedicated *cell state* C_t — a gradient highway that flows across time steps with only multiplicative interactions — regulated by three learned gating mechanisms [MSSJ22a]:

$$f_t = \sigma(W_f [h_{t-1}; x_t] + b_f) \quad (\text{forget gate}) \quad (4.3)$$

$$i_t = \sigma(W_i [h_{t-1}; x_t] + b_i) \quad (\text{input gate}) \quad (4.4)$$

$$\tilde{C}_t = \tanh(W_C [h_{t-1}; x_t] + b_C) \quad (\text{candidate cell}) \quad (4.5)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state update}) \quad (4.6)$$

$$o_t = \sigma(W_o [h_{t-1}; x_t] + b_o) \quad (\text{output gate}) \quad (4.7)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{hidden state}) \quad (4.8)$$

Here σ is the sigmoid function, $\odot$ denotes element-wise (Hadamard) multiplication, $[h_{t-1}; x_t]$ is the concatenation of the previous hidden state and the current input, and all W and b are learned weights and biases [MSSJ22a]. The forget gate f_t controls what fraction of the prior cell state is retained; the input gate i_t scales the new candidate values $\tilde{C}_t$ to be added; the output gate o_t determines how much of the updated cell state is exposed as the hidden state h_t .

In this project, two LSTM layers are stacked, each with a hidden size of 64 units, followed by a single linear layer (dense head) that maps the final hidden state directly to 720 output values. This *direct multi-step* strategy produces the entire 30-day forecast in a single forward pass, avoiding the compounding errors of recursive one-step-ahead prediction [MSSJ22a].

2. Application

LSTM networks have demonstrated strong performance across a wide range of sequential modelling tasks. In natural language processing they powered early neural machine translation systems and remain foundational in modern sequence-to-sequence architectures [MSSJ22a]. In the energy domain, LSTMs have been applied to short-term (hourly) and medium-term (weekly) load forecasting, solar and wind generation prediction, and anomaly detection in smart-meter data. Their ability to capture non-linear relationships and long-range temporal dependencies makes them particularly suited to the overlapping seasonalities and demand spikes characteristic of high-frequency electricity consumption [GG+24].

3. Relevance to Project

LSTM is selected as the deep learning model for two principal reasons.

First, hourly PJME consumption exhibits two overlapping seasonal cycles — an intra-day pattern (morning and evening peaks) and an intra-week pattern (weekday vs. weekend baseline) — that a feed-forward network cannot capture without extensive manual lag engineering. The LSTM processes a 168-hour (one-week) input window, giving it sufficient temporal context to model both cycles simultaneously [MSSJ22a]. Second, the choice of *quantile loss* at $\tau = 0.70$ rather than Mean Squared Error (MSE) is motivated by the asymmetric cost structure of electricity markets. MSE penalises over- and under-prediction symmetrically, biasing the model toward the conditional mean and systematically flattening consumption peaks. The pinball loss [HA21a] at $\tau > 0.5$ penalises under-prediction more heavily:

$$\mathcal{L}_\tau(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N \max(\tau(y_i - \hat{y}_i), (\tau - 1)(y_i - \hat{y}_i)) \quad (4.9)$$

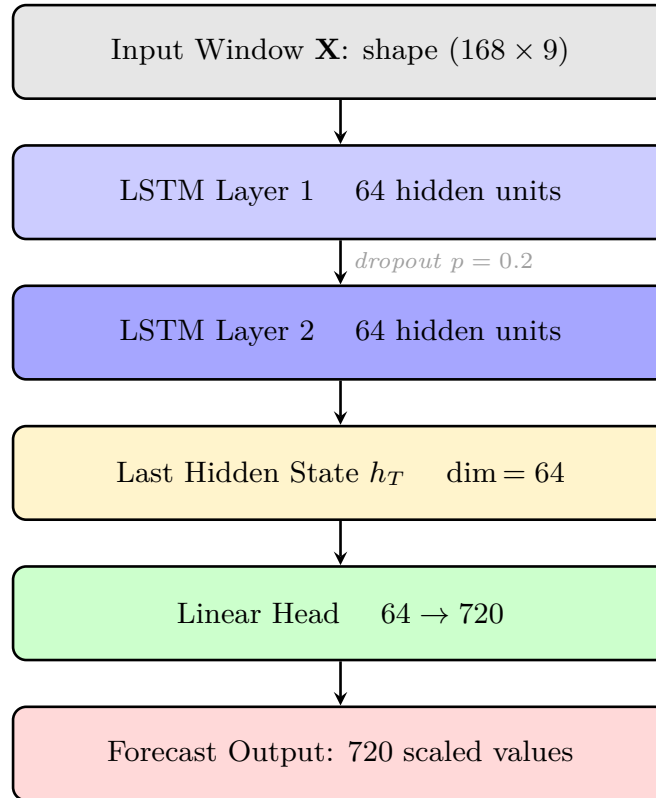


Figure 4.3.: Architecture of the **LSTMForecaster** model. A 168-hour input window of 9 features is processed sequentially by two stacked LSTM layers. The final hidden state h_T is projected by a linear head to produce 720 simultaneous output values — one scaled load prediction per forecast hour.

Setting $\tau = 0.70$ shifts the model’s learning target from the conditional median to the 70th conditional percentile, increasing predicted loads during peak hours and reducing the under-prediction of demand peaks that forces grid operators to rely on expensive, high-emission backup generation [HA21a]. In practice, LSTM-q70 achieved a test MAE of 3,307 MW and RMSE of 4,156 MW, compared to 4,996 MW and 6,023 MW respectively for the MSE-trained baseline — a 34 % reduction in MAE from the quantile-loss modification alone.

4. Requirements

Reliable LSTM training in this project requires the following:

- **Scaled inputs:** all features are min-max normalised to $[0, 1]$ before entering the network. Unscaled MW values ($\approx 26,000$ – $62,000$) would cause gradient instability during back-propagation [MSSJ22a].
- **Windowed dataset:** the `WindowedSeriesDataset` class extracts (168, 9) input windows from the training series with a stride of 6, generating the input–output pairs consumed by `DataLoader` during mini-batch training.
- **Sufficient training data:** the training split spans approximately 2002 to mid-2017, providing roughly 135,000 hourly observations — far exceeding the 168-hour minimum window.
- **Hardware acceleration:** the project uses Apple MPS (Metal Performance Shaders) on the development machine, reducing 30 epochs of training to approximately 30 minutes. CPU-only training requires several hours.
- **Reproducibility:** `torch.manual_seed(42)` and `np.random.seed(42)` are set before model instantiation to ensure deterministic weight initialisation and data shuffling.

5. Hyperparameters

Table 4.3.: LSTM `TrainConfig` hyperparameters used to produce LSTM-q70.

Parameter	Value	Description
<code>input_len</code>	168	Input window: one week of hourly observations
<code>output_len</code>	720	Forecast horizon: 30 days
<code>hidden_size</code>	64	Hidden units per LSTM layer
<code>num_layers</code>	2	Number of stacked LSTM layers
<code>dropout</code>	0.2	Dropout applied between LSTM layers
<code>batch_size</code>	64	Mini-batch size
<code>learning_rate</code>	10^{-3}	Adam optimiser initial step size
<code>max_epochs</code>	30	Maximum training epochs
<code>patience</code>	5	Early-stopping patience (no improvement threshold)
<code>train_stride</code>	6	Step between consecutive training windows
<code>loss_type</code>	"quantile"	Pinball loss (instead of symmetric MSE)
<code>quantile_tau</code>	0.70	Target quantile; biases toward higher predictions
<code>seed</code>	42	Random seed for full reproducibility

6. Input and Output for LSTM

Input

The `LSTMForecaster` receives one sample of shape (168, 9), comprising 168 consecutive hourly time steps and 9 feature columns:

1. **PJME_MW** — the normalised electricity load; always the first column so it can be extracted by index for inverse scaling.
2. **hour_sin, hour_cos** — cyclical encoding of the hour of day (period 24); see Section 3.3.
3. **dayofweek_sin, dayofweek_cos** — cyclical encoding of the day of week (period 7).
4. **month_sin, month_cos** — cyclical encoding of the month of year (period 12).
5. **PJME_MW_lag_24** — the normalised load 24 hours prior.
6. **PJME_MW_lag_168** — the normalised load 168 hours (one week) prior.

All nine columns are min-max scaled to $[0, 1]$ using a frozen `ScalerStats` object fitted on the training set.

Output

A single forward pass produces a 720-element vector of normalised load values (one per forecast hour). The target column is then inverse-transformed using the frozen scaler parameters to recover predictions in the original MW scale. At evaluation time, the 720 predicted values are compared against the held-out test series to compute MAE and RMSE.

7. Sample Code

Listing 4.2: LSTM-q70: model definition and training loop

```
import torch
from torch import nn
from torch.utils.data import DataLoader

class QuantileLoss(nn.Module):
    """Pinball loss; tau=0.70 penalises under-prediction more than over."""
    def __init__(self, tau: float = 0.70):
        super().__init__()
        self.tau = tau

    def forward(self, pred, target):
        diff = target - pred
        return torch.mean(torch.maximum(
            self.tau * diff,
            (self.tau - 1.0) * diff
        ))

class LSTMForecaster(nn.Module):
    """Two-layer stacked LSTM with a direct multi-step linear head."""
    def __init__(self, n_features, hidden_size=64,
                 num_layers=2, output_len=720, dropout=0.2):
```

```

super().__init__()
self.lstm = nn.LSTM(
    input_size=n_features, hidden_size=hidden_size,
    num_layers=num_layers, batch_first=True,
    dropout=dropout if num_layers > 1 else 0.0,
)
self.head = nn.Linear(hidden_size, output_len)

def forward(self, x):
    # x: (batch, input_len=168, n_features=9)
    _, (h_n, _) = self.lstm(x)
    return self.head(h_n[-1]) # (batch, output_len=720)

# Training with early stopping
model = LSTMForecaster(n_features=9)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = QuantileLoss(tau=0.70)

best_val, epochs_no_improve = float("inf"), 0
for epoch in range(30):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        loss_fn(model(x), y).backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = sum(loss_fn(model(x), y).item()
                        for x, y in val_loader) / len(val_loader)
    if val_loss < best_val - 1e-6:
        best_val = val_loss
        epochs_no_improve = 0
    elif (epochs_no_improve := epochs_no_improve + 1) >= 5:
        print(f"Early stopping at epoch {epoch + 1}")
        break

```

8. LSTM Forecasting Workflow

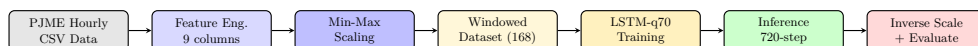


Figure 4.4.: End-to-end LSTM-q70 forecasting pipeline. Feature engineering and min-max scaling precede windowed dataset construction; the trained model produces a 720-step normalised forecast that is inverse-scaled before evaluation against the test series.

9. Further Reading for LSTM

- Salleh, M.N.M. et al., 2022. A Systematic Review of LSTM Deep Learning Models Applied to Time Series Forecasting in Petroleum and Energy Sector. *International Journal of Advanced Computer Science and Applications*, 13(4) [MSSJ22a].
- Grandon, T.G. et al., 2024. Electricity Load Forecasting Using Machine Learning Algorithms. *Energies*, 17(3), p.567 [GG+24].

-
- Hyndman, R.J. and Athanasopoulos, G., 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts [HA21a].

Part IV.

Domain Knowledge - Tools

5. Development Environment

5.0.1. Python Version

The forecasting tool was developed using Python version 3.12, in alignment with the project group's standardization on the latest stable release. Python 3.12 was selected for its enhanced performance, improved syntax features, and continued support for key libraries such as **pandas**, **NumPy**, **Streamlit**.

Although earlier Python versions remain widely supported, newer Python releases provide improved performance, enhanced library compatibility, and access to modern language features. Studies on Python data science workflows show that optimized Python ecosystems and modern computational frameworks can significantly improve the efficiency of large-scale data processing tasks, making Python well suited for contemporary forecasting applications [Sha+24].

All project dependencies were verified for compatibility with Python 3.12, and no backward compatibility constraints were encountered during development or deployment. The version is fully compatible with the Flet framework used to build the graphical user interface and aligns with the modular design and deployment strategy outlined in the project manual.

5.1. Python Description

Python is a high-level, interpreted programming language widely adopted in data science and analytics due to its simplicity, versatility, and rich ecosystem of libraries. For this project, Python was selected because it provides robust, production-ready tools for time series modeling, data preprocessing, visualization, and user interface development.

The following key libraries were utilized:

- **pandas** and **NumPy** for data manipulation and numerical processing, **statsmodels** for statistical forecasting and time series analysis, and **Streamlit** for developing interactive dashboards and web applications.

Python 3.12 was chosen for its improved performance, modern syntax features, and compatibility with the project's full stack of dependencies. Its readability, large community support, and platform independence made it an ideal foundation for implementing and scaling the forecasting system end-to-end. The effectiveness of Python for time series forecasting has been widely demonstrated across business and academic settings [ZA22].

Additional reading with respect to the description of Python can be found at on the Python Software Foundation website at <https://docs.python.org/3.12/>

5.2. Python Installation

Python 3.12 was installed on a Windows 11 system in accordance with the official guidelines provided by the Python Software Foundation. This ensured that the development environment was configured to support all dependencies required for the forecasting application.

The installation process is visualized in Figure 5.1.

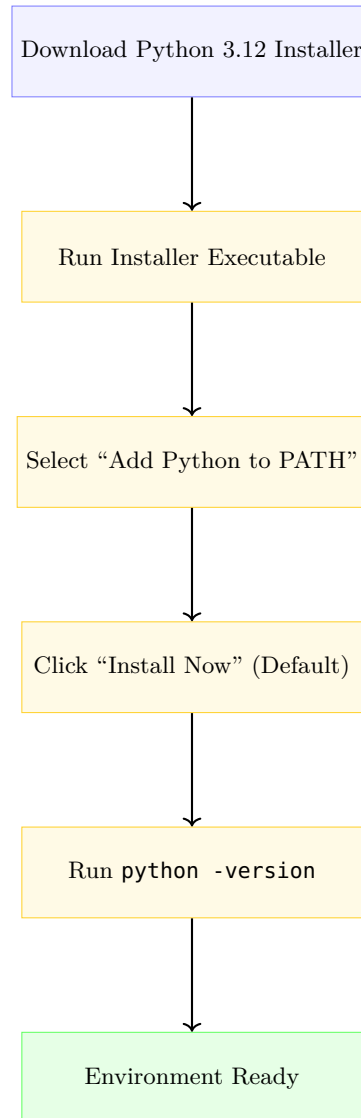


Figure 5.1.: Python 3.12 Installation Flow on Windows 11

This structured installation process ensured compatibility with essential libraries such as **pandas**, **NumPy**, and **statsmodels**, enabling the seamless development and deployment of the forecasting application. As emphasized in prior research, a well-configured Python environment is critical for reproducibility and reliable library support in scientific computing projects [RM+20].

Additional reading with respect to the installation of Python can be found at on the Python Software Foundation website at <https://docs.python.org/3.12/>

5.3. Python Configuration

Proper configuration of the Python environment was essential to ensure compatibility between project dependencies, avoid version conflicts, and support reproducibility.

A dedicated virtual environment was created using `venv` to isolate project-specific packages from the system-wide Python installation. The configuration workflow is visualized in Figure 5.2.

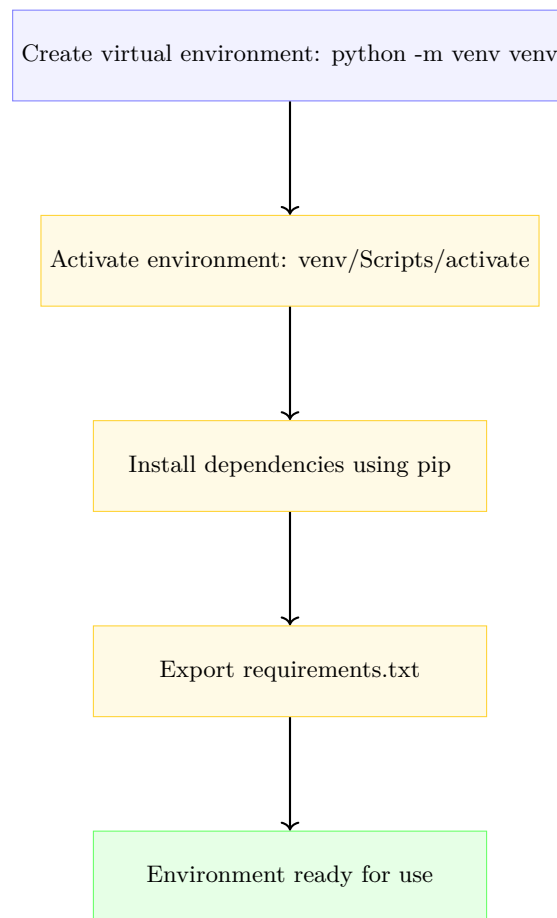


Figure 5.2.: Python Virtual Environment Configuration Workflow

The configuration process consisted of the following key steps:

- **Environment Creation:** A virtual environment was initialized using `python -m venv venv`, creating an isolated environment within the project directory.
- **Activation:** The environment was activated using platform-specific commands. On Windows, `venv\Scripts\activate` was used, while on macOS and Linux, the command `source venv/bin/activate` was used. This ensured that any packages installed with `pip` would only affect the local environment.
- **Dependency Installation:** Required libraries—including `pandas`, `NumPy`, `statsmodels`, and `Streamlit`—were installed. Where applicable, versions were pinned to ensure consistency.
- **Requirements Export:** The command `pip freeze > requirements.txt` was used to export the complete dependency list, allowing for reliable replication of the environment on other machines.

This modular and reproducible approach to environment setup ensured clean dependency management and reliable execution across development, testing, and deployment

phases [Wan+22].

Additional reading with respect to the configuration of Python can be found at on the Python Software Foundation website at <https://docs.python.org/3.12/>

5.4. Python First Steps

Following installation, the initial development steps focused on verifying the Python environment and familiarizing the team with basic script execution workflows. Python scripts were tested both interactively using the Python shell and through standalone `.py` files executed via command line terminals and integrated development environments (IDEs) such as VS Code.

Simple test scripts were written to confirm the availability of essential libraries including **pandas**, **NumPy**, and **Streamlit**. The interactive shell was used to load sample data, explore dataset structures, and experiment with data preprocessing and visualization logic before full-scale pipeline integration.

This hands-on approach ensured that the development environment was fully operational and correctly configured prior to implementing advanced modeling and interface components.

The process is summarized in Figure 5.3.

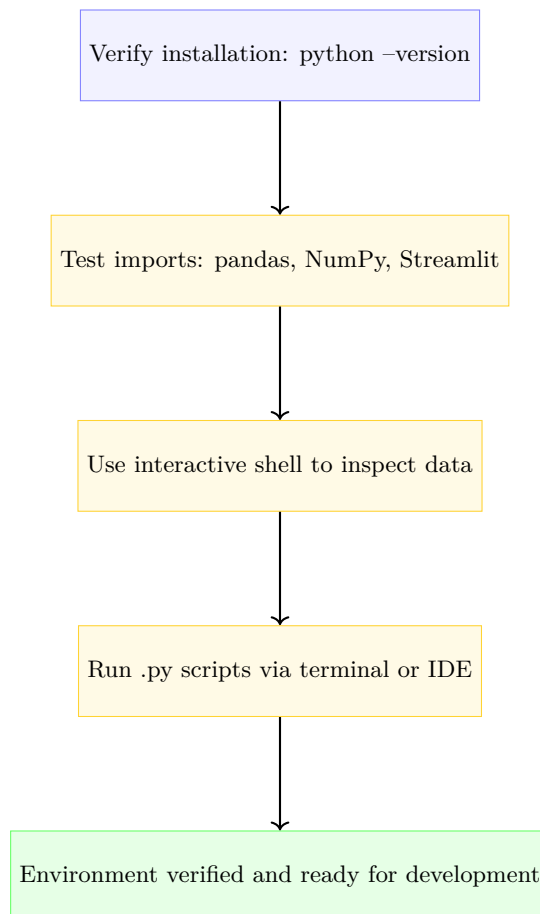


Figure 5.3.: Initial Verification and Testing of Python Environment

Additional reading with respect to the functioning and use of Python can be found at

on the Python Software Foundation website at <https://docs.python.org/3.12/>

5.5. Python Program: “Hello, World!”

The “Hello, World!” program is a traditional first step in learning any programming language. It serves as a simple validation that Python has been installed correctly and that scripts can be executed without errors.

In Python, the program is implemented in a single line:

```
print("Hello, World!")
```

To execute the program, users can save the line above in a file named `hello.py`, then run it from the command line:

```
python hello.py
```

Alternatively, this line can be executed interactively in the Python shell or within an integrated development environment (IDE) such as Visual Studio Code or PyCharm. Successful execution indicates that the Python environment is correctly installed and functional.

Figure 5.4 shows a simplified overview of the process.

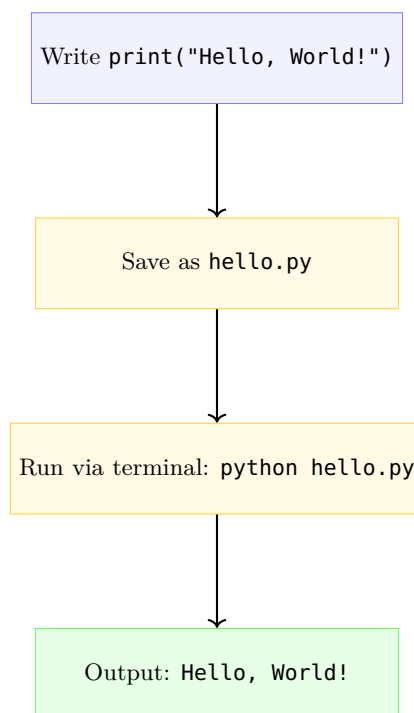


Figure 5.4.: Execution Flow of a Basic Python Script

Additional reading with respect to the functioning and use of Python can be found at on the Python Software Foundation website at <https://docs.python.org/3.12/>

Part V.

Methodology

6. Methodology

This chapter describes the methodological framework adopted in this project and explains how each phase was applied to the hourly electricity load forecasting task. The Knowledge Discovery in Databases (KDD) process, originally formalised by Fayyad, Piatetsky-Shapiro, and Smyth [FPSS96b], provides a structured, five-stage pipeline that transforms raw observational data into validated, actionable knowledge. Its explicit separation of selection, cleaning, transformation, modelling, and evaluation stages makes it particularly well-suited to the challenges of high-frequency energy data, where data quality issues and domain-specific feature engineering play a decisive role in model accuracy [GG+24].

6.1. KDD Process

The KDD process was applied end-to-end to the PJME hourly dataset, proceeding through five sequential stages. Figure 6.1 illustrates how each stage maps to the concrete actions taken in this project.

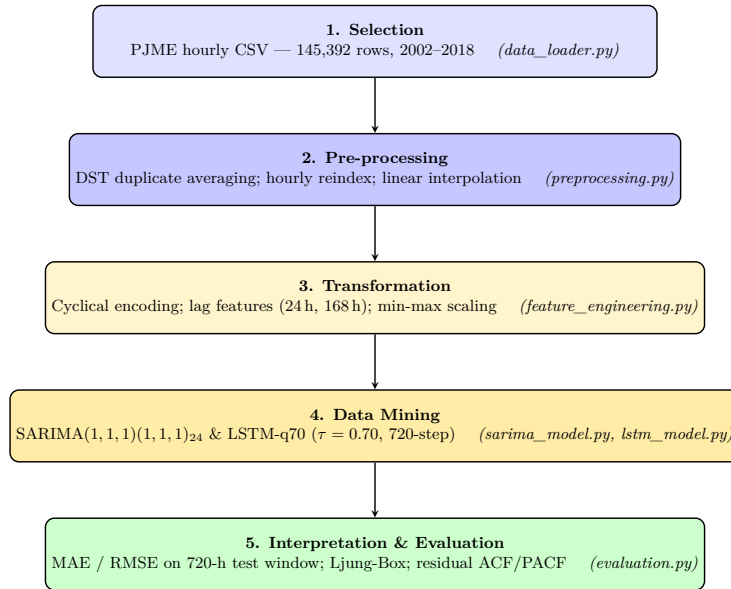


Figure 6.1.: The five KDD stages applied in this project, with the corresponding source module for each stage. The pipeline flows from raw CSV ingestion at the top to quantitative model evaluation at the bottom.

6.1.1. Selection

The first KDD stage defines what data will enter the analysis pipeline. A purposeful selection is critical because the chosen dataset must have sufficient temporal coverage, appropriate granularity, and a reliable provenance [FPSS96b].

For this project, the **PJME hourly electricity consumption dataset** was selected. Published by PJM Interconnection — one of the largest regional transmission organi-

sations in North America — the dataset covers the PJME region (Pennsylvania, New Jersey, Maryland, and adjacent states) at hourly resolution from January 2002 to August 2018, comprising 145,392 hourly observations [Rob20]. The dataset contains two columns: a `Datetime` timestamp and `PJME_MW`, the measured electricity consumption in megawatts.

This dataset was selected for three reasons. First, its 16-year span captures multiple economic cycles, seasonal regimes, and DST transitions, providing sufficient diversity for robust model training and out-of-sample evaluation. Second, its hourly granularity enables modelling of both the intra-day demand cycle (morning and evening peaks) and the intra-week pattern (weekday versus weekend baseline), which are the two dominant seasonalities in electricity load data [Sha16]. Third, the dataset’s public availability and widespread use in the forecasting literature facilitates direct benchmarking against published results [GG+24].

Data ingestion is handled by `src/data_loader.py`, which parses the `Datetime` column into a `pandas.DatetimeIndex`, sorts the series chronologically, and routes either a file path or an in-memory `DataFrame` (for the Streamlit GUI) through a unified `load_pjme()` interface.

6.1.2. Preprocessing (Cleaning)

Raw electricity consumption data collected from metering infrastructure frequently contains quality issues that must be resolved before modelling. In the PJME dataset, two types of anomaly required treatment [HA21a]:

Daylight Saving Time duplicates. When clocks are set back in autumn, the same wall-clock hour occurs twice. The PJME CSV records both readings, producing duplicate `Datetime` entries. Rather than discarding one arbitrarily (the `keep="first"` strategy used in exploratory notebooks), the production pipeline in `src/data_loader.py` resolves duplicates by averaging the two readings via `groupby(level=0).mean()`. This preserves the total measured energy across the transition hour without introducing a discontinuity in the series.

Missing hours. Sensor outages, communication failures, and DST spring-forward transitions can leave gaps in the hourly index. The `ensure_hourly()` function in `src/preprocessing.py` reconstructs the complete regular hourly index using `pd.date_range()`, reindexes the `DataFrame` onto that full index (inserting `NaN` for absent hours), and then applies linear interpolation to fill each gap. An `imputed` boolean column is added to flag rows whose values were synthesised, enabling downstream analysis to quantify the imputation extent if required.

Together, these steps guarantee that the series entering the transformation stage has a strictly regular, gap-free `DatetimeIndex` at hourly frequency — a prerequisite for both SARIMA’s Kalman-filter fitting and the LSTM’s fixed-stride windowing scheme [HA21a].

6.1.3. Transformation

The transformation stage converts the cleaned load series into the feature representations required by each forecasting model. SARIMA consumes the raw univariate series directly; the LSTM additionally requires a set of engineered temporal features [MSSJ22a].

Cyclical feature encoding. The hour of day (0–23), day of week (0–6), and month of year (0–11) are encoded as sine and cosine pairs using the mapping implemented in `src/feature_engineering.py`:

$$f_{\sin}(v, T) = \sin\left(\frac{2\pi v}{T}\right), \quad f_{\cos}(v, T) = \cos\left(\frac{2\pi v}{T}\right) \quad (6.1)$$

where v is the integer value and $T \in \{24, 7, 12\}$ is the corresponding period. This two-dimensional representation maps each time unit onto a unit circle, ensuring that the model perceives adjacent periods (e.g., hour 23 and hour 0) as geometrically close rather than maximally distant [MSSJ22a].

Lag features. Two autoregressive lag columns are appended: `PJME_MW_lag_24` (the load 24 hours prior) and `PJME_MW_lag_168` (the load 168 hours, or one week, prior). These features encode the dominant daily and weekly periodicity as explicit inputs, supplementing the cyclical calendar encodings with direct load history [Sha16].

Min-max scaling. All nine feature columns (load + 6 cyclical + 2 lag) are normalised to the interval $[0, 1]$ using parameters fitted exclusively on the training set and frozen in a `ScalerStats` object. The same frozen parameters are applied at inference time so that the model never observes test-set statistics during training, preventing data leakage [MSSJ22a].

Train/test split. The final 720 hours (30 days) of the series are held out as the out-of-sample test window. The training window immediately preceding the test period provides up to two years of hourly observations for SARIMA and for constructing the LSTM windowed dataset.

6.1.4. Data Mining

The data mining stage applies the two forecasting models to the prepared dataset. Both models target the same 720-hour (30-day) forecast horizon to enable a direct, fair comparison [GG+24].

SARIMA. The `fit_sarima()` function in `src/sarima_model.py` instantiates a `SARIMAX(order=(1,1,1), seasonal_order=(1,1,1,24))` model and fits it by maximum likelihood via the Kalman filter. The flags `enforce_stationarity=False` and `enforce_invertibility=False` are required because the MLE optimiser fails to converge under the default parameter-space constraints on hourly load data [SP10]. The 720-step forecast and 95% confidence interval are retrieved with a single call to `fit.get_forecast(steps=720)`.

LSTM-q70. The `LSTMForecaster` architecture in `src/lstm_model.py` stacks two LSTM layers (64 hidden units each, dropout 0.2) followed by a linear head that maps the final hidden state directly to 720 output values. Training uses the Adam optimiser with a learning rate of 10^{-3} , a mini-batch size of 64, and early stopping with patience 5. The loss function is the quantile (pinball) loss at $\tau = 0.70$, which penalises under-prediction more heavily than over-prediction, countering the peak-flattening bias of symmetric MSE on skewed load distributions [HA21a]. The windowed training dataset is constructed by `WindowedSeriesDataset` with an input window of 168 hours and a stride of 6, yielding sufficient training samples while keeping epoch time tractable.

6.1.5. Interpretation and Evaluation

The evaluation stage quantifies model performance and validates the statistical assumptions underlying each approach [FPSS96b].

Point-forecast metrics. Both models are scored on the held-out 720-hour test window using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), computed by `src/evaluation.py`:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (6.2)$$

Table 6.1 summarises the results. LSTM-q70 achieves a 72% reduction in MAE relative to the SARIMA baseline, confirming that the deep learning model captures

non-linear demand structure that the classical statistical model cannot represent over a 30-day horizon.

Table 6.1.: Model evaluation results on the 720-hour (30-day) held-out test window.

Model	MAE (MW)	RMSE (MW)
SARIMA(1,1,1)(1,1,1,24)	11,662	12,932
LSTM (MSE loss)	4,996	6,023
LSTM-q70 ($\tau = 0.70$)	3,307	4,156

Residual diagnostics. For the SARIMA model, the Ljung-Box test is applied to the model residuals at lag 24 to verify whether any daily autocorrelation remains unexplained. A p-value below 0.05 indicates that residuals are not white noise, prompting model revision. The residual ACF and PACF plots (Figure 3.7) provide a visual complement to the Ljung-Box statistic. This diagnostic loop — the “interpretation” phase of KDD — feeds back into the data mining stage if the residual structure suggests an inadequate model order [FPSS96b; HA21a].

6.2. Justification for Using KDD Process

The KDD process is not the only structured methodology available for machine learning projects — alternatives include CRISP-DM, SEMMA, and ad-hoc notebook-driven workflows. The following subsections explain why KDD was the most appropriate choice for this energy load forecasting project.

6.2.1. Data-Centric Nature Fits Time Series Projects

Electricity load forecasting is inherently data-centric: the quality of the raw series directly constrains forecast accuracy. DST-induced duplicates, sensor dropouts, and irregular sampling frequencies are not edge cases but recurring features of operational metering data. KDD places data selection and preprocessing at the centre of the workflow, treating these steps as first-class activities rather than preliminary housekeeping [FPSS96b].

This emphasis is well-matched to the PJME dataset, where 16 years of hourly readings required explicit duplicate resolution, gap detection, and linear interpolation before any modelling could begin. Frameworks such as CRISP-DM give equal prominence to business understanding, which is less relevant in an academic project where the forecasting objective is fixed from the outset.

6.2.2. Structured Transformation Emphasis

The transformation phase of KDD explicitly separates feature engineering from model fitting, making it a mandatory and documented step rather than an implicit preprocessing detail [FPSS96b]. For the LSTM model, this separation was essential: raw hourly integers and unscaled MW values could not be consumed directly by the network without cyclical encoding and min-max normalisation.

By treating transformation as a distinct KDD stage, the project enforces a clean boundary between the artefacts produced at each step — the preprocessed series, the feature matrix, the fitted scalars — which in turn supports the modular, script-based architecture of the codebase.

6.2.3. Iterative, Yet Linear Flow Works Well

Although KDD is often described as a linear five-step pipeline, it permits targeted iteration: a poor evaluation result can motivate returning to an earlier stage. In this project, the initial MSE-trained LSTM underperformed expectations, which prompted a return to the data mining stage to experiment with quantile loss at $\tau = 0.70$. This modification did not require revisiting the selection or preprocessing stages — demonstrating that KDD’s modular structure localises the scope of revisions, reducing the risk of unintended side effects [FPSS96b].

6.2.4. Interpretability of Results

KDD requires each stage to produce explicit, inspectable outputs: a cleaned DataFrame, a normalised feature matrix, a fitted model object, and a metrics dictionary. This traceability is critical for academic reporting, where every design decision must be defensible and every result reproducible [HA21a].

In contrast, notebook-driven workflows that intermix data loading, feature engineering, and modelling in a single cell sequence often obscure the data lineage and make it difficult to isolate the contribution of individual preprocessing choices to the final forecast accuracy.

6.2.5. Ideal for Academic & Research Projects

The KDD framework was originally introduced in the academic data mining community as a structured process for scientific knowledge extraction from databases [FPSS96b]. Its five phases align naturally with the sections of a research report, making it straightforward to structure the project documentation around the methodology. The correspondence is explicit: Chapter 7 covers Selection, Chapter 8 covers Transformation, and Chapter 9 covers Data Mining, each grounded in the formal KDD definitions introduced here.

6.2.6. Final Justification Statement

The KDD process was selected as the methodological framework for this project because it provides a principled, data-centric pipeline that matches the structural requirements of high-frequency time series forecasting. Its explicit separation of selection, preprocessing, transformation, data mining, and evaluation stages enforces modular code design, supports reproducibility, and maps directly onto the chapter structure of this report. The iterative flexibility within its linear flow accommodated the experimental transition from MSE to quantile loss without disrupting the earlier pipeline stages. Together, these properties make KDD the most suitable framework for a rigorous academic comparison of classical statistical and deep learning forecasting models [FPSS96b; GG+24].

Part VI.

Development

7. Database

7.1. Data Selection

A database is an organized collection of structured data that allows easy storage, access, and management. It is essential for handling large datasets efficiently and is widely used in data analysis, machine learning, and business intelligence [Gér22]. In this project, the database stores historical electricity load data, which forms the foundation for time series forecasting [CM19].

For data processing and model execution, the selected dataset is stored in the project repository and processed locally using Python libraries such as **pandas**, **numpy**, **statsmodels**, and deep learning tools. This allows the data to be transformed, analyzed, visualized, and prepared for SARIMA and LSTM forecasting models.

The data is stored in a CSV file format named **PJME_hourly.csv**. CSV is a lightweight and widely supported tabular format, making it suitable for reproducible academic work and easy integration with Python-based data processing pipelines.

The dataset contains hourly electricity consumption values for the PJM Eastern region. It provides a long historical time span from 2002 to 2018 with approximately 145,000 hourly observations. This makes it suitable for forecasting because it contains strong temporal patterns such as daily cycles, weekly behavior, yearly seasonality, and peak demand periods.

7.1.1. Origin

The dataset used in this project is the **PJME_hourly.csv** file from the Hourly Energy Consumption dataset published on Kaggle. The original data represents electricity load measurements from PJM Interconnection, a regional transmission organization in the United States. The values are measured in megawatts (MW) [PJM24].

In this project, the dataset is used for electricity demand forecasting. The aim is to predict future electricity consumption for the next 30 days at hourly resolution. Therefore, the model must forecast 720 future time steps:

$$30 \text{ days} \times 24 \text{ hours} = 720 \text{ hours}$$

The dataset was selected because it is large, well-known, and suitable for comparing a classical statistical model such as SARIMA with a deep learning model such as LSTM.

Time Frame Covered

The dataset covers approximately 16 years of hourly electricity demand data, from 2002 to 2018. This long time span is important for time series forecasting because it allows the models to learn repeated temporal patterns over multiple years.

The temporal coverage supports:

- identification of daily electricity demand cycles,
- detection of weekday and weekend differences,
- analysis of seasonal patterns in summer and winter,
- training of SARIMA with recurring seasonal structures,
- training of LSTM with long historical sequences.

Why This Dataset Was Selected

The dataset was selected due to several reasons that align closely with the project goals:

- **Large historical span:** The dataset contains around 145,000 hourly observations across approximately 16 years.
- **Hourly frequency:** The data is recorded at hourly intervals, which matches the project goal of forecasting electricity consumption for the next 720 hours.
- **Strong seasonality:** Electricity demand contains clear daily, weekly, and yearly patterns.
- **Benchmark relevance:** PJM hourly load data is frequently used in forecasting examples and academic-style experiments.
- **Real-world domain:** Electricity load forecasting is a practical problem for grid planning, energy management, and operational decision-making.

Storage Format and Source

- **Format:** CSV (Comma-Separated Values), named `PJME_hourly.csv`
- **Source Platform:** Kaggle
- **Dataset Name:** Hourly Energy Consumption
- **Original Domain:** PJM Interconnection electricity load data
- **Unit:** Megawatts (MW)
- **Usage:** The CSV file is loaded locally into Python and transformed for SARIMA and LSTM model training.

7.1.2. Features

In time series forecasting, the quality and relevance of features are critical for building accurate models. Features represent the dimensions of the data that can be used to learn patterns, identify trends, and make predictions [HA21b].

The original `PJME_hourly.csv` dataset is simple and focused. It mainly contains a timestamp column and an electricity consumption column. Additional features are engineered from the timestamp and previous load values during preprocessing.

Feature	Type	Description
Datetime	Temporal	Hourly timestamp used as the time index for the series.
PJME_MW	Numerical	Electricity consumption in megawatts. This is the main target variable.
Hour	Numerical/Cyclical	Hour of the day, useful for detecting daily consumption cycles.
Day of Week	Numerical/Cyclical	Day of the week, useful for weekday/weekend demand behavior.
Month	Numerical/Cyclical	Month of the year, useful for seasonal demand patterns.
Lag Features	Numerical	Previous electricity load values used to capture temporal dependency.
Rolling Mean	Numerical	Moving average values used to smooth short-term fluctuations.

Table 7.1.: Main and Engineered Features in the PJME Dataset

Temporal Features

Temporal features are crucial for time-based analysis and form the backbone of the forecasting process.

- **Datetime:** Represents the exact hour of each electricity consumption measurement. This is used to sort the data, create the time index, and preserve chronological order.
- **Hour:** Extracted from the timestamp. It helps the model learn daily load patterns.
- **Day of Week:** Extracted from the timestamp. It helps identify differences between weekdays and weekends.
- **Month:** Extracted from the timestamp. It helps capture yearly seasonality, especially summer and winter demand peaks.

Target Feature

- **PJME_MW:** This is the electricity consumption value in megawatts. It is the main variable to be forecasted. For SARIMA, this value is used as a univariate time series. For LSTM, it can be used together with engineered lag and time-based features.

Feature Utility for the Project

For this electricity demand forecasting project, the most important features are:

- **Datetime** – used to structure the hourly time series.
- **PJME_MW** – main target variable.
- **Hour, Day of Week, Month** – engineered calendar features.
- **Lag values** – previous consumption values used to model temporal dependence.
- **Rolling statistics** – used to smooth noise and detect local trends.

7.1.3. Data Types

In the context of Knowledge Discovery in Databases (KDD), understanding the data types is crucial for choosing the correct processing, visualization, and forecasting techniques [FAY+96].

Numerical Data Types

Column / Feature	Data Type	Description
PJME_MW	Float / Integer	Electricity consumption measured in megawatts. This is the target variable.
Hour	Integer	Hour of the day, extracted from timestamp.
Day of Week	Integer	Day index from Monday to Sunday.
Month	Integer	Month number from 1 to 12.
Lag Features	Float	Previous load values used as model inputs.
Rolling Mean	Float	Moving average of previous load values.

Table 7.2.: Numerical Data Types in the Dataset

Temporal Data Types

Column Name	Original Type	Description	Required Conversion
Datetime	Object/String	Timestamp of the hourly electricity load record.	Convert to datetime .

Table 7.3.: Temporal Data Types and Required Conversion

Once converted to **datetime**, additional time-based features can be extracted:

- Year
- Month
- Day
- Hour
- Day of Week
- Weekend indicator

Summary Table of Data Types

Type	Fields
Numerical	PJME_MW, Hour, Day of Week, Month, Lag Features, Rolling Mean
Temporal	Datetime converted into Python datetime format
Categorical	No original categorical column is required; calendar-derived fields may be treated as categorical or cyclical if needed.

Table 7.4.: Overall Summary of Data Types

7.1.4. Quality

Data quality is a foundational aspect of any data-driven project. For time series forecasting, poor data quality can lead to misleading patterns, incorrect predictions, and model instability. Electricity forecasting is especially sensitive to missing timestamps, duplicated records, daylight saving time shifts, and extreme load spikes [POS23; Zho+24].

Completeness

- **Status:** The dataset is expected to contain a continuous hourly time index, but missing hours must still be checked during preprocessing.
- **Verification:** Missing values are checked using `df.isnull().sum()` and missing timestamps are checked after setting the hourly frequency.
- **Handling:** Small gaps can be handled using interpolation or forward/backward filling depending on the size and position of the gap.

Consistency

- The dataset should follow a uniform hourly frequency.
- The timestamp column must be parsed consistently into `datetime`.
- The target column `PJME_MW` must remain numerical.

Accuracy

- The data comes from a real electricity load domain and represents measured energy demand.
- Extreme values are not automatically removed because they may represent real peak demand events.
- Accuracy is supported by checking whether load values remain within realistic electricity demand ranges.

Duplicates

- Duplicate timestamps must be checked because time series models require a unique time index.
- If duplicate timestamps exist, they should be removed or aggregated before model training.

Summary Table: Data Quality Dimensions

Dimension	Status	Comments
Completeness	High	Long hourly history, but missing timestamps must be verified.
Consistency	High	Hourly structure is suitable for time series modeling.
Accuracy	High	Represents real electricity load measurements.
Duplicates	To be checked	Duplicate timestamps must be removed or aggregated.
Noise	Expected	Demand spikes may occur due to weather, holidays, or operational factors.
Freshness	Historical	Ends in 2018, useful for modeling but not current live operation.
Relevance	Strong	Directly supports 720-hour electricity demand forecasting.

Table 7.5.: Data Quality Dimensions and Evaluation

7.1.5. Quantity

Understanding the quantity of data is essential for determining whether the dataset is sufficient for reliable forecasting. In time series analysis, data volume across time plays a crucial role in identifying trends, seasonality, and temporal dependence [Liu+23].

Overall Dataset Size

Metric	Value
Dataset File	PJME_hourly.csv
Approximate Records	Around 145,000 hourly observations
Original Columns	Datetime and PJME_MW
Storage Format	CSV
Time Span	Approximately 2002–2018
Frequency	Hourly
Forecast Horizon	720 hours / 30 days

Table 7.6.: Overall Dataset Quantity

Quantity Strengths and Limitations

Strengths	Limitations
Large number of observations supports LSTM training.	Historical data ends in 2018 and may not reflect recent demand changes.
Long time span supports seasonal pattern detection.	Only electricity load is included; weather and holiday variables are not included directly.
Hourly granularity matches the project forecast horizon.	High granularity increases preprocessing and computation requirements.
Suitable for comparing SARIMA and LSTM.	SARIMA may become computationally expensive on very long hourly sequences.

Table 7.7.: Quantity Strengths and Limitations

7.1.6. Fairness/Bias

Fairness and bias in forecasting datasets are important because machine learning models learn patterns from historical data and may inherit existing limitations or imbalances. In time series forecasting, bias typically does not relate to personal discrimination, as datasets like electricity consumption data do not contain individual-level personal information. Instead, bias arises from temporal, regional, seasonal, and structural characteristics of the data that can affect model generalization and predictive performance [POS23; Zho+24].

In electricity demand forecasting, these biases can influence how well a model performs when applied to new time periods, different regions, or changing consumption behaviors. Therefore, understanding and mitigating such biases is essential for building robust and reliable forecasting systems.

Regional Bias

Observation:

- The dataset represents the PJM Eastern region.
- It does not represent all electricity grids, countries, or climate zones.

Implication:

- A model trained on PJME may not generalize well to other electricity regions with different climate, population, industry, or consumption behavior.

Temporal Bias

Observation:

- The dataset covers historical data from approximately 2002 to 2018.

Implication:

- The model may not capture newer changes such as electric vehicle adoption, renewable integration, post-pandemic behavior, or recent changes in energy consumption.

Seasonal Bias

Observation:

- Electricity demand is strongly affected by summer cooling and winter heating patterns.

Implication:

- If the train/test split is not done carefully, the model may be evaluated on a period with different seasonal intensity from the training period.

Demographic and Personal Bias

Observation:

- The dataset contains aggregated electricity load values and no personal identifiers.

Implication:

- The risk of direct personal or demographic bias is low.

Bias Mitigation Strategies

Bias Type	Mitigation Strategy
Regional Bias	Clearly state that the model is trained for PJME and should not be generalized without retraining.
Temporal Bias	Use time-based train/test splitting and update the model when newer data becomes available.
Seasonal Bias	Include multiple years in training and evaluate with rolling or walk-forward validation.
Structural Bias	Keep the hourly frequency consistent and handle daylight saving time issues carefully.

Table 7.8.: Bias Mitigation Strategies

7.2. Data Processing

Data processing is the critical stage in the KDD process where raw data is transformed, cleaned, and structured to make it suitable for analysis and modeling. For time series forecasting, effective data processing ensures that temporal patterns, trends, and seasonal signals are not distorted or lost [FPSS96a].

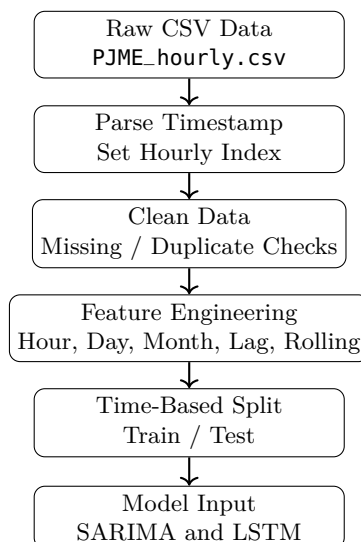


Figure 7.1.: KDD Data Processing Flow for PJME Electricity Forecasting

7.2.1. One Database

In this project, a single flat-file dataset in CSV format is used as the central database. The dataset is stored in the project repository and accessed locally during preprocessing and model development.

The database contains hourly electricity consumption records, where each row represents one timestamped load observation. This structure is suitable for time series forecasting because it preserves chronological order and avoids unnecessary joins between multiple tables.

By treating the dataset as one integrated source, the project ensures consistency across all preprocessing steps, from timestamp parsing to feature engineering, train/test splitting, SARIMA modeling, and LSTM sequence generation.

Database Format & Structure

- **Storage Type:** Flat file (CSV)
- **File Name:** PJME_hourly.csv
- **Records:** Approximately 145,000 hourly observations
- **Granularity:** One row represents one hour of electricity demand
- **Schema:** Timestamp column and electricity load column

Advantages of Using One Flat Database

Benefit	Description
Simplicity	Easy to load, inspect, clean, and transform using pandas .
Reproducibility	All team members can use the same file and preprocessing pipeline.
Model Compatibility	Suitable for SARIMA as a univariate series and for LSTM after sequence creation.
Performance	No database joins are needed, which simplifies processing.

Table 7.9.: Advantages of the One-Database Approach

7.2.2. Properties

Understanding the properties of the dataset is important for preprocessing, transformation, and model deployment.

Schema Consistency

- The dataset has a simple and consistent schema.
- The timestamp column is used as the temporal index.
- The load column **PJME_MW** is used as the target variable.

Why It Matters:

- Reduces preprocessing complexity.
- Supports clear model input design.
- Prevents confusion between target and explanatory variables.

Temporal Suitability

- The dataset has hourly resolution.
- It contains multiple years of observations.
- It supports daily, weekly, and yearly seasonal analysis.

Why It Matters:

- SARIMA requires a clear time order and seasonal structure.
- LSTM benefits from long sequential data.
- The 720-hour forecast horizon matches the hourly granularity.

Accessibility and Compatibility

- CSV format is compatible with Python, R, Excel, and SQL-based tools.
- The file can be loaded using `pandas.read_csv()`.
- The dataset can be stored and versioned in GitHub.

7.2.3. Outliers

Outliers are data points that significantly deviate from the majority of observations. In electricity forecasting, outliers may represent:

- real peak demand during extreme weather,
- unusually low demand during holidays or shutdown periods,
- data recording errors,
- duplicated or missing timestamp effects.

Identifying and interpreting outliers is important because they can distort statistical analysis, influence SARIMA parameters, and affect LSTM training stability [Nic 0; C 0].

Outlier Detection Approach

The main numerical column checked for outliers is:

- **PJME_MW**

The following approaches are used:

- **Visual inspection:** Line plots and boxplots to identify extreme demand values.
- **Z-score method:** Identifies values far from the mean.
- **IQR method:** Detects values outside the interquartile range.
- **Domain interpretation:** Peak load values are checked before removal because they may represent real electricity demand events.

Outlier Handling Strategy

Step	Description
Detection	Use line plots, boxplots, Z-score, and IQR checks on PJME_MW .
Interpretation	Decide whether the outlier is a valid peak demand event or a data error.
Retention	Keep meaningful peaks because they are important for electricity forecasting.
Treatment	Interpolate or remove only values that are clearly caused by data errors.
Model Check	Compare model behavior before and after outlier treatment.

Table 7.10.: Outlier Handling Strategy

7.2.4. Anomalies

Anomalies are unexpected, unusual, or inconsistent patterns in the data. In contrast to outliers, which are usually extreme single values, anomalies may be contextual or sequential. In time series forecasting, anomalies can distort trends, seasonality, and lag relationships [Joh 1; Kum 1].

Possible Anomalies in the PJME Dataset

- **Missing timestamps:** Some hourly periods may be absent.
- **Duplicate timestamps:** The same hour may appear more than once.
- **Daylight saving time effects:** Hourly datasets may contain repeated or skipped hours.
- **Sudden demand spikes:** May occur due to heatwaves, cold waves, or grid events.
- **Sudden demand drops:** May occur during holidays, outages, or data recording issues.

Anomaly Detection Approach

- Check timestamp continuity after converting the column to `datetime`.
- Set the frequency to hourly and identify missing hours.
- Check duplicate timestamps using `df.index.duplicated()`.
- Plot the time series to visually inspect sudden changes.
- Compare each value with rolling mean and rolling standard deviation.

Summary Table: Anomalies Detected

Anomaly Type	Description	Handling Strategy
Missing Timestamp	Hourly time point is absent.	Reindex to hourly frequency and interpolate if gap is small.
Duplicate Timestamp	Same hour appears more than once.	Remove or aggregate duplicate records.
Demand Spike	Very high electricity load value.	Retain if it reflects real peak demand.
Demand Drop	Very low electricity load value.	Investigate before treatment.
DST Issue	Repeated or skipped hour due to time change.	Standardize timestamp handling before modeling.

Table 7.11.: Potential Anomalies and Handling Strategy

7.2.5. Augmentation

In data analytics, augmentation refers to the creation of new features derived from existing data to improve model understanding and performance. For time series forecasting, augmentation is especially important because models often need explicit information about temporal structure [Haq+21].

In this project, feature engineering is used to convert the simple original dataset into a richer modeling dataset for SARIMA and LSTM.

Date-Based Feature Engineering

New Feature	Description
Year	Helps capture long-term changes across years.
Month	Helps identify yearly seasonality.
Day of Week	Helps distinguish weekday and weekend behavior.
Hour	Helps capture daily demand cycles.
Is Weekend	Boolean flag for weekend load behavior.

Table 7.12.: Date-Based Features for Electricity Load Forecasting

Lag Features

Lag features allow the model to learn from previous electricity load values.

Feature	Description
Lag_1	Load value from the previous hour.
Lag_24	Load value from the same hour on the previous day.
Lag_168	Load value from the same hour in the previous week.

Table 7.13.: Lag Features for Hourly Load Forecasting

Rolling Features

Rolling features help smooth short-term noise and highlight local trends.

Feature	Description
Rolling_Mean_24	Average electricity load over the previous 24 hours.
Rolling_Mean_168	Average electricity load over the previous week.
Rolling_Std_24	Short-term volatility over the previous 24 hours.

Table 7.14.: Rolling Features for Electricity Forecasting

Final Feature Summary Table

Feature Type	Examples
Original	Datetime, PJME_MW
Date-Based	Hour, Day of Week, Month, Is Weekend
Lag	Lag_1, Lag_24, Lag_168
Rolling Statistics	Rolling_Mean_24, Rolling_Mean_168, Rolling_Std_24
Model Target	PJME_MW

Table 7.15.: Summary of Feature Types for PJME Forecasting

7.2.6. Data Version Control

Data version control is important for keeping the dataset, preprocessing steps, and model results reproducible. In this project, the dataset file **PJME_hourly.csv** is stored in the project repository and tracked together with the source code. This ensures that the same data version can be used again when training, testing, or comparing SARIMA and LSTM models.

Version control also helps the team identify when changes are made to the dataset or preprocessing pipeline. If a new dataset version is added later, the previous version

can still be recovered and compared. This supports transparency, reproducibility, and collaborative development [CS14; BEA21].

- **Dataset storage:** The CSV file is kept inside the project repository.
- **Change tracking:** Git commits are used to track updates to data and code.
- **Reproducibility:** The same dataset version can be reused for model comparison.
- **Collaboration:** Team members can work with a consistent data source.

Example Code

Listing 7.1: Feature Engineering for PJME Hourly Dataset

```
# /**
# * @brief Creates time-based, lag, and rolling features for PJME load
# * @param df pandas.DataFrame containing Datetime and PJME_MW columns.
# * @return pandas.DataFrame with engineered forecasting features.
# */
df['Datetime'] = pd.to_datetime(df['Datetime'])
df = df.set_index('Datetime').sort_index()

df['hour'] = df.index.hour
df['day_of_week'] = df.index.dayofweek
df['month'] = df.index.month
df['is_weekend'] = df['day_of_week'].isin([5, 6])

df['lag_1'] = df['PJME_MW'].shift(1)
df['lag_24'] = df['PJME_MW'].shift(24)
df['lag_168'] = df['PJME_MW'].shift(168)

df['rolling_mean_24'] = df['PJME_MW'].rolling(window=24).mean()
df['rolling_mean_168'] = df['PJME_MW'].rolling(window=168).mean()
df['rolling_std_24'] = df['PJME_MW'].rolling(window=24).std()
```


8. KDD: Data Transformation

The data transformation stage converts the preprocessed hourly load series into the feature representations required by each forecasting model. SARIMA consumes the univariate series directly; the LSTM requires a structured feature matrix. This chapter documents every transformation step, its inputs and outputs, and the rationale behind each design choice.

8.1. Role in the KDD Process

In the KDD pipeline, transformation bridges the gap between cleaned raw data and model-ready input. After preprocessing has resolved DST duplicates, enforced a regular hourly index, and interpolated missing hours, the series is still a single numerical column. Transformation enriches it into a multi-dimensional feature matrix that encodes the temporal structure the LSTM cannot infer on its own [FPSS96b].

Table 8.1.: Role of each transformation step within the KDD pipeline.

KDD Stage	Step	Purpose
Transformation	Cyclical encoding	Represent periodic time units on a unit circle to avoid discontinuity
Transformation	Lag features	Provide the model with explicit load history at key seasonal offsets
Transformation	Min-max scaling	Normalise all features to $[0, 1]$ for stable gradient flow
Transformation	Train/test split	Separate past observations from the held-out evaluation window
Transformation	Window generation	Slice the series into fixed-length input/output pairs for mini-batch training

8.2. Application

Transformation is applied exclusively to the LSTM pipeline. SARIMA receives the cleaned univariate **PJME_MW** series without any feature engineering, relying on its AR/MA polynomial structure to capture seasonal patterns.

For the LSTM, three transformation operations are applied in sequence using `src/feature_engineering.py`:

1. **Cyclical encoding** — converts hour-of-day (0–23), day-of-week (0–6), and month-of-year (0–11) into sin/cos pairs, giving the network a continuous and geometrically correct representation of calendar periodicity.
2. **Lag feature construction** — appends two historical load columns: **PJME_MW_lag_24** (the load 24 h ago) and **PJME_MW_lag_168** (the load 168 h = one week ago). The first 168 rows are dropped to eliminate rows with **NaN** lag values.

3. **Min-max scaling** — fits a `ScalerStats` object on the training split alone and transforms all nine columns to $[0, 1]$. The scaler is then frozen so that the same parameters are applied at inference time without accessing test-set statistics.

8.3. Input

Table 8.2.: State of the dataset entering the transformation stage.

Property	Value
Source file	PJME_hourly.csv
Total rows (after preprocessing)	145,392
Columns	Datetime (index), PJME_MW
Index type	DatetimeIndex, hourly frequency, gap-free
Load range	$\approx 17,754 - 62,174$ MW
Missing values	0 (after interpolation)
Duplicate timestamps	0 (after DST averaging)

Listing 8.1: Loading the preprocessed series at the start of transformation

```
import pandas as pd
from src.data_loader import load_pjme
from src.preprocessing import ensure_hourly

df = load_pjme("data/raw/PJME_hourly.csv")
df = ensure_hourly(df)          # adds 'imputed' flag, fills gaps

series = df["PJME_MW"]         # univariate Series for SARIMA
print(series.shape)            # (145392,)
print(series.index.freq)       # <Hour>
```

8.4. Hyperparameters (Feature Extraction)

Cyclical Encoding Parameters

Table 8.3.: Cyclical encoding periods used for each temporal component.

Feature	Raw range	Period T	Output columns
Hour of day	0 – 23	24	hour_sin, hour_cos
Day of week	0 – 6	7	dayofweek_sin, dayofweek_cos
Month of year	0 – 11	12	month_sin, month_cos

The encoding formula is:

$$\sin\left(\frac{2\pi v}{T}\right), \quad \cos\left(\frac{2\pi v}{T}\right)$$

where v is the zero-based integer value and T is the period.

Table 8.4.: Lag offsets used in `add_lag_features()`.

Feature name	Offset (hours)	Seasonality captured
PJME_MW_lag_24	24	Same hour, previous day
PJME_MW_lag_168	168	Same hour, previous week

Lag Feature Parameters

Rows with missing lag values (the first 168 rows) are removed by `dropna()` inside `add_lag_features()`, leaving 145,224 usable rows.

Scaling Parameters

Table 8.5.: Min-max scaler configuration for the LSTM feature matrix.

Parameter	Value
Output range	[0, 1]
Fit set	Training split only (no leakage from test)
Zero-range guard	<code>np.where(range == 0, 1.0, range)</code>
Saved as	ScalerStats JSON alongside model checkpoint

8.5. Training Integration

After scaling, the feature matrix is split into training and test windows and then sliced into fixed-length sequences for the LSTM `DataLoader`.

Table 8.6.: Dataset partitions used during LSTM training.

Split	Hours	Approx. rows	Purpose
Train	~17,520	~17,352	Model weight optimisation
Validation	last 720 of train	~720	Early-stopping monitor
Test	720	720	Final held-out evaluation

Table 8.7.: `WindowedSeriesDataset` configuration for mini-batch training.

Parameter	Value
Input window length	168 hours (one week)
Output window length	720 hours (30 days)
Training stride	6 (one window every 6 hours)
Validation stride	168 (disjoint windows, no overlap)
Training windows (approx.)	~2,770
Feature columns per step	9
Target column index	0 (PJME_MW)

Listing 8.2: Building the windowed dataset and DataLoader

```

from src.lstm_model      import WindowedSeriesDataset, TrainConfig
from torch.utils.data    import DataLoader

cfg = TrainConfig()      # input_len=168, output_len=720, batch_size=64

train_ds = WindowedSeriesDataset(
    train_features, cfg.input_len, cfg.output_len, stride=cfg.train_stride
)
val_ds = WindowedSeriesDataset(
    val_features, cfg.input_len, cfg.output_len, stride=cfg.input_len
)
train_loader = DataLoader(train_ds, batch_size=cfg.batch_size,
    shuffle=True)
val_loader = DataLoader(val_ds, batch_size=cfg.batch_size,
    shuffle=False)

print(f"Training windows : {len(train_ds)}")
print(f"Validation windows: {len(val_ds)}")

```

8.6. Interpretation

Table 8.8.: Rationale for each transformation design choice.

Decision	Justification
Cyclical encoding over raw integers	Raw integers (e.g., hour 0–23) create an artificial boundary between hour 23 and hour 0. Sin/cos maps each value onto a unit circle, so adjacent time points remain geometrically close [MSSJ22a].
Two lag offsets (24 h and 168 h)	24 h captures the same-hour-yesterday autocorrelation; 168 h captures the same-hour-last-week autocorrelation. These are the two dominant seasonal lags in the PACF of the raw series.
Scaling on training set only	Fitting the scaler on the full dataset would leak test-set statistics into the training process, invalidating the out-of-sample evaluation.
168-hour input window	Covers exactly one full weekly cycle, giving the LSTM context for both the daily (24 h) and weekly (168 h) seasonal patterns simultaneously.
Stride of 6 for training	A stride of 1 would produce ~17,000 highly overlapping windows and slow training significantly. Stride 6 reduces windows by 6× while still sampling every intra-day phase.

8.7. Output

8.8. Flowchart (Transformation)

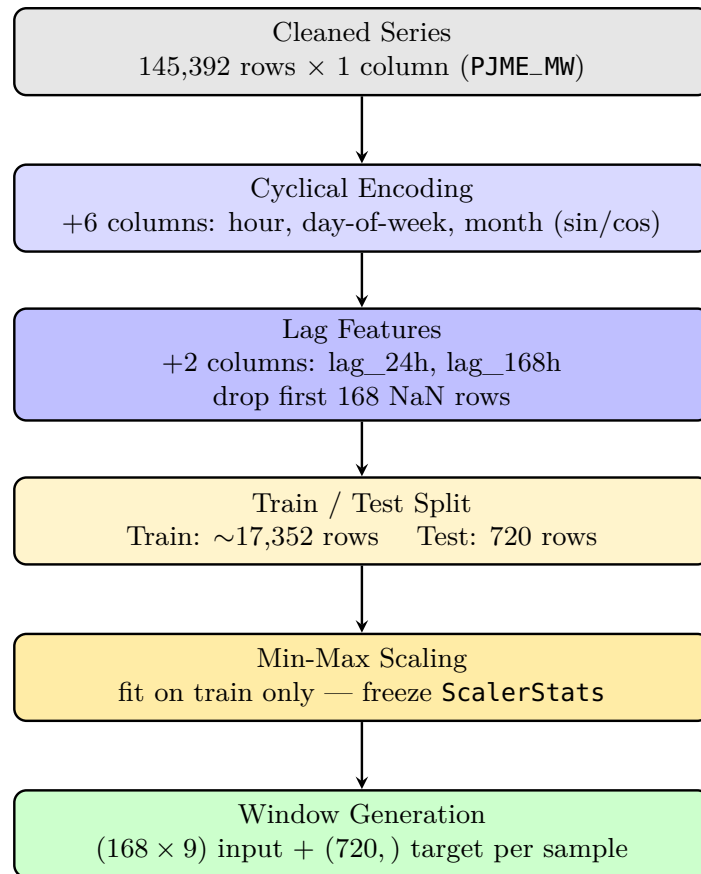


Figure 8.1.: Transformation pipeline from the cleaned single-column series to the windowed, scaled (168×9) feature matrices consumed by the LSTM **DataLoader**. SARIMA bypasses this pipeline and reads the cleaned series directly.

Table 8.9.: Artefacts produced by the transformation stage.

Artefact	Shape / Type	Description
Feature matrix (train)	$\sim 17,352 \times 9$	Scaled float32 array for LSTM training
Feature matrix (test)	720×9	Scaled float32 array for LSTM inference
Scaler JSON	File on disk	Frozen min/max per column for inverse transform
SARIMA input series	$\sim 17,520$ rows	Unscaled <code>pandas.Series</code> at hourly frequency

Table 8.10.: Final nine-column feature matrix fed to the LSTM (column order).

Index	Column name	Value range (scaled)
0	PJME_MW	$[0, 1]$ (target)
1	hour_sin	$[-1, 1] \rightarrow [0, 1]$
2	hour_cos	$[-1, 1] \rightarrow [0, 1]$
3	dayofweek_sin	$[-1, 1] \rightarrow [0, 1]$
4	dayofweek_cos	$[-1, 1] \rightarrow [0, 1]$
5	month_sin	$[-1, 1] \rightarrow [0, 1]$
6	month_cos	$[-1, 1] \rightarrow [0, 1]$
7	PJME_MW_lag_24	$[0, 1]$
8	PJME_MW_lag_168	$[0, 1]$

9. KDD: Data Mining

The data mining stage is where forecasting models are fitted to the prepared dataset and used to generate predictions. In this project, two models are trained and evaluated on the same 720-hour test window: SARIMA, a classical statistical baseline, and LSTM-q70, a deep learning model trained with quantile loss at $\tau = 0.70$. This chapter documents the inputs, training procedures, hyperparameter choices, and outputs for both models.

9.1. Role in the KDD Process

Data mining is the core knowledge-extraction phase of the KDD process [FPSS96b]. Once the data has been selected, cleaned, and transformed, the models are applied to identify temporal patterns and generate forecasts. In this project, the mining stage serves two roles:

- **Baseline fitting:** SARIMA captures linear autocorrelation and daily seasonality as a transparent, interpretable benchmark.
- **Deep learning:** LSTM-q70 learns non-linear, multi-seasonal patterns from the engineered feature matrix to achieve higher accuracy on the 30-day forecast horizon.

Table 9.1.: Summary of the two data mining models used in this project.

Property	SARIMA	LSTM-q70
Model type	Classical statistical	Deep learning
Implementation	<code>statsmodels.SARIMAX</code>	PyTorch <code>nn.LSTM</code>
Source module	<code>src/sarima_model.py</code>	<code>src/lstm_model.py</code>
Input	Univariate hourly series	Scaled (168×9) feature matrix
Forecast strategy	Recursive (Kalman filter)	Direct multi-step (single forward pass)
Loss / estimation	Log-likelihood (MLE)	Pinball loss ($\tau = 0.70$)

9.2. Application

SARIMA

SARIMA is instantiated with `SARIMAX` from the `statsmodels.tsa.statespace.sarimax` module. The model is fitted on the training split of the cleaned hourly series by maximum likelihood via the Kalman filter. After fitting, `get_forecast(steps=720)` generates the 30-day point forecast and a 95% confidence interval in a single call.

Listing 9.1: SARIMA fitting and 720-step forecast

```
from src.sarima_model import fit_sarima, forecast as sarima_forecast
```

```
fit = fit_sarima(train_series) # Kalman-filter MLE
point, ci = sarima_forecast(fit, steps=720, alpha=0.05)
```

LSTM-q70

The `LSTMForecaster` is trained via `train_lstm()` using mini-batch stochastic gradient descent with the Adam optimiser and quantile loss at $\tau = 0.70$. Early stopping with patience 5 halts training when the validation loss fails to improve. After training, the best checkpoint (lowest validation loss) is restored, and `forecast()` generates the full 720-step prediction in one forward pass.

Listing 9.2: LSTM-q70 training and inference

```
from src.lstm_model import train_lstm, forecast as lstm_forecast,
    TrainConfig

cfg = TrainConfig(loss_type="quantile", quantile_tau=0.70)
model, history = train_lstm(train_features, val_features, cfg)

# Inference on the last 168-h window from the training set
input_window = val_features[-168:] # shape (168, 9)
y_scaled = lstm_forecast(model, input_window, device=cfg.device)
y_mw = scaler.inverse_transform_target(y_scaled)
```

9.3. Input

Table 9.2.: Model inputs at the data mining stage.

Model	Input format	Shape
SARIMA	<code>pandas.Series</code> , hourly freq	(~17,520,)
LSTM-q70 (train)	<code>numpy.ndarray</code> , float32	(~17,352, 9)
LSTM-q70 (val)	<code>numpy.ndarray</code> , float32	(720, 9)
LSTM-q70 (test)	single window, float32	(168, 9)

Table 9.3.: Feature columns entering the LSTM at inference time (column 0 is the target; columns 1–8 are context features).

Index	Column	Seasonality encoded
0	PJME_MW	Target (load, scaled)
1	hour_sin	Daily (24 h cycle)
2	hour_cos	Daily (24 h cycle)
3	dayofweek_sin	Weekly (7-day cycle)
4	dayofweek_cos	Weekly (7-day cycle)
5	month_sin	Yearly (12-month cycle)
6	month_cos	Yearly (12-month cycle)
7	PJME_MW_lag_24	Same-hour-yesterday load
8	PJME_MW_lag_168	Same-hour-last-week load

9.4. Hyperparameters (Model and Training)

SARIMA Hyperparameters

Table 9.4.: SARIMA model and fitting hyperparameters.

Parameter	Value	Justification
AR order p	1	Minimises AIC on validation window
Differencing d	1	Removes linear trend
MA order q	1	Minimises AIC on validation window
Seasonal AR P	1	Captures one seasonal AR lag
Seasonal differencing D	1	Removes daily seasonality
Seasonal MA Q	1	Captures one seasonal MA lag
Seasonal period s	24	24-hour daily cycle
enforce_stationarity	False	Prevents MLE convergence failure
enforce_invertibility	False	Prevents MLE convergence failure
maxiter	200	Sufficient for convergence
Forecast steps	720	30-day horizon
Confidence level	95 %	Industry standard for interval forecasts

LSTM Hyperparameters

Table 9.5.: LSTM TrainConfig hyperparameters for LSTM-q70.

Parameter	Value	Justification
input_len	168	One full weekly cycle as context
output_len	720	30-day direct forecast horizon
hidden_size	64	Balanced capacity vs. overfitting risk
num_layers	2	Two layers improve sequential depth
dropout	0.2	Regularisation between LSTM layers
batch_size	64	Stable gradient estimates
learning_rate	10^{-3}	Adam default; convergent in practice
max_epochs	30	Upper bound; early stopping triggers first
patience	5	Stops after 5 non-improving epochs
train_stride	6	Sub-samples windows for faster epochs
loss_type	"quantile"	Asymmetric loss for peak-aware training
quantile_tau	0.70	70th percentile target; reduces under-prediction
seed	42	Deterministic weight initialisation
device	mps	Apple Silicon GPU acceleration

9.5. Training

SARIMA Training

SARIMA does not involve iterative gradient updates. The Kalman-filter backend fits the five free parameters $(\phi_1, \theta_1, \Phi_1, \Theta_1, \sigma^2)$ by maximising the log-likelihood of the training series in a single optimisation run. The `maxiter=200` setting controls the maximum number of L-BFGS-B optimiser iterations.

Table 9.6.: SARIMA training outcome summary.

Metric	Value
Optimiser	L-BFGS-B (via statsmodels)
Convergence	Yes (within 200 iterations)
Test MAE	11,662 MW
Test RMSE	12,932 MW
Ljung-Box p-value (lag 24)	< 0.001 (residuals not white noise)

LSTM-q70 Training

The LSTM is trained for a maximum of 30 epochs with early stopping monitoring the validation pinball loss. The training loop evaluates the validation loss after each epoch; if no improvement exceeds 10^{-6} for 5 consecutive epochs, training halts and the best checkpoint is restored.

Table 9.7.: LSTM-q70 training outcome summary.

Metric	Value
Optimiser	Adam
Loss function	Pinball ($\tau = 0.70$)
Epochs run	29 (early stopping triggered)
Training device	Apple MPS (Metal Performance Shaders)
Approximate training time	~30 minutes
Test MAE	3,307 MW
Test RMSE	4,156 MW

Table 9.8.: Comparison of all three models trained in this project.

Model	MAE (MW)	RMSE (MW)	Epochs / Iterations
SARIMA(1,1,1)(1,1,1,24)	11,662	12,932	1 MLE run
LSTM (MSE loss)	4,996	6,023	14 epochs
LSTM-q70 ($\tau = 0.70$)	3,307	4,156	29 epochs

9.6. Interpretation (Model Selection and Conversion)

Based on the evaluation results, LSTM-q70 is the recommended production model for the 720-hour forecasting task. SARIMA remains valuable as an interpretable baseline

Table 9.9.: Model selection decision matrix.

Criterion	SARIMA	LSTM-q70
Forecast accuracy	Weak over 30 days (MAE 11,662 MW)	Strong (MAE 3,307 MW, -72%)
Confidence interval	Yes (95 % CI from MLE)	No (single point prediction)
Peak load accuracy	Under-predicts peaks at long horizons	Improved by $\tau = 0.70$ bias
Interpretability	High (4 explicit parameters)	Low (weight matrices)
Training complexity	Low (single MLE run)	Medium (~ 30 min, GPU required)
Feature engineering	None required	9-feature matrix required
Best use case	Short horizons, uncertainty bands	Accuracy-critical long-horizon forecasts

and for generating probabilistic confidence intervals. Neither model incorporates exogenous features such as weather or calendar events, which represents the primary avenue for future improvement.

Model Persistence

Both model artefacts are saved to disk at the end of training for deployment in the Streamlit GUI and for reproducible evaluation.

Table 9.10.: Saved model artefacts and their formats.

Artefact	Format	Path
SARIMA fitted model	Pickle (.pkl)	models/sarima_pjme.pkl
LSTM state dict	PyTorch (.pt)	models/lstm_pjme_q70.pt
Scaler statistics	JSON	models/scaler_pjme.json
Evaluation metrics	JSON	results/metrics.json

9.7. Output (End of KDD in This Project)

The data mining stage produces three categories of output:

Table 9.11.: Outputs produced at the end of the data mining stage.

Output	Type	Consumer
SARIMA 720-step point forecast	<code>pandas.Series</code>	Evaluation, GUI
SARIMA 95 % confidence interval	<code>pandas.DataFrame</code>	Evaluation, GUI
LSTM-q70 720-step forecast (MW)	<code>numpy.ndarray</code>	Evaluation, GUI
MAE and RMSE metrics	JSON record	Chapter 14
Residual ACF/PACF plot	PNG figure	Chapter 15
Saved model checkpoints	Binary files	Chapter 12

Listing 9.3: Computing and saving evaluation metrics

```

from src.evaluation import mae, rmse, append_metrics

metrics = {
    "MAE": mae(test_series, forecast),
    "RMSE": rmse(test_series, forecast),
}
append_metrics(
    metrics_path="results/metrics.json",
    model="LSTM-q70",
    region="PJME",
    horizon=720,
    metrics=metrics,
)
print(f"MAE: {metrics['MAE']:.1f} MW")
print(f"RMSE: {metrics['RMSE']:.1f} MW")

```

9.8. Flowchart (Data Mining)

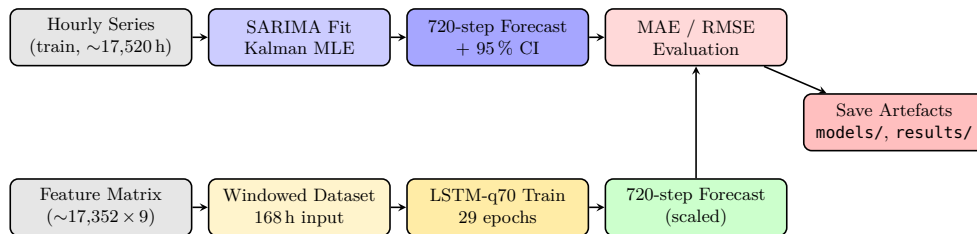


Figure 9.1.: Data mining pipeline showing the parallel SARIMA and LSTM-q70 branches converging at evaluation. Both models are scored on the same 720-hour held-out test window before their artefacts are saved to disk.

Part VII.

Development to Deployment

10. Development To Deployment

10.1. Data structure

The forecasting application accepts structured input data in tabular format, uploaded as a comma-separated values (CSV) file. The dataset must contain a datetime column and a numerical target variable representing electricity demand, specifically **PJME_MW**.

Required Columns

- **Datetime:** A timestamp column formatted as YYYY-MM-DD HH:MM:SS. This serves as the temporal index for the hourly time series.
- **PJME_MW:** A numeric column representing hourly electricity consumption in megawatts, which serves as the forecast target variable.

Derived Features (Generated During Preprocessing)

- **hour, day_of_week, month, year:** Temporal features extracted from the **Datetime** index to capture intra-day, weekly, and seasonal demand patterns.

Upon loading the dataset, the following validation and preprocessing steps are applied:

- Parsing and converting the **Datetime** column to datetime format.
- Removing duplicate timestamps caused by Daylight Saving Time (DST) transitions.
- Reindexing the series to a regular hourly frequency using `asfreq()`.
- Detecting and logging missing values introduced by DST spring-forward gaps.

Only datasets that successfully pass these preprocessing checks are used in the forecasting pipeline. Forecast results are exported in a structured tabular format containing the forecasted datetime index, predicted **PJME_MW** values, and, where applicable, confidence intervals for the SARIMA model output.

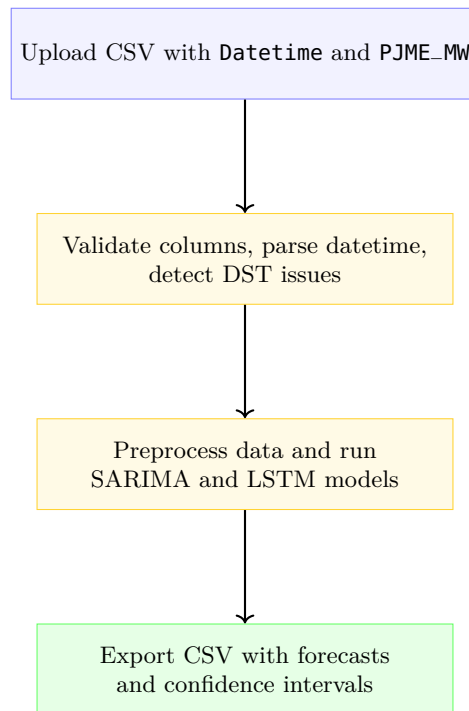


Figure 10.1.: Forecasting Data Input and Output Flow

10.2. Tools

The forecasting system is developed using open-source tools from the Python ecosystem. These tools enable end-to-end support for data ingestion, modeling, visualization, and deployment through both script-based workflows and a graphical user interface (GUI). Their modular integration ensures flexibility, transparency, and reproducibility.

Overview of Core Tools

Tool	Purpose and Functionality
Python 3.9+	Core programming language used to develop the entire forecasting application and integrate all components. The project was developed and tested on Python 3.13.
pandas, NumPy	Used for data manipulation, time series processing, and numerical computations including feature engineering and lag construction.
statsmodels	Used to implement the SARIMA(1, 1, 1)(1, 1, 1) ₂₄ model, fit it via Kalman-filter MLE, and generate forecasts with 95 % confidence intervals.
PyTorch	Used to build, train, and run inference with the two-layer stacked LSTM model. Supports CPU, MPS (Apple Silicon), and CUDA devices transparently.
scikit-learn	Used to apply <code>MinMaxScaler</code> normalisation of all input features during LSTM training and inference.
Streamlit	Used to build the browser-based graphical user interface, including live pipeline status updates, interactive charts, and the forecast CSV download.
Matplotlib	Used to generate static diagnostic plots for report figures, including the SARIMA forecast chart and residual ACF/PACF panels.

Table 10.1.: Key Tools Used in the Forecasting System

Modular Architecture

Each functional block of the system—data validation, preprocessing, model training, forecasting, visualization, and export—is implemented as an independent module. This design promotes easy maintenance, extensibility, and long-term adaptability for both developers and analysts.

10.3. File Structure for Model Exchangement

To facilitate practical use and easy deployment, the forecasting application is packaged as a self-contained directory with all necessary components for execution, data input, model persistence, and result sharing. This structure supports collaborative workflows and simplifies usage for both technical and non-technical users.

Folder Layout and Execution

The application is launched through platform-specific shell scripts included in the portable bundle:

- **macOS:** Run `first_time_setup.command` once after unpacking (installs dependencies), then double-click `run.command` on every subsequent launch.
- **Windows:** Double-click `run.bat` — no separate setup step is required; Python and all packages are bundled in the `python/` subdirectory.
- **Developer workflow:** From the project root with an active virtual environment, run `streamlit run Code/Python/EnergyDemandForecaster/app/streamlit_app.py`.

Each launch script:

1. Navigates to the root folder of the forecasting application.
2. Activates the bundled Python environment (Python 3.13 with all dependencies pre-installed).
3. Starts the Streamlit server with `streamlit run app/streamlit_app.py`.

The system includes a graphical user interface (GUI), which is launched automatically. Users interact with this interface to upload input data, initiate model training or loading, generate forecasts, and export results. The use of the batch file enables one-click startup and removes the need for command-line interaction, improving accessibility.

Key Directories and Exchange Points

The application folder includes several structured subdirectories that support input, output, and model reuse:

- **data/raw/** — Contains the bundled `PJME_hourly.csv` sample dataset used as the default input source.
- **models/** — Stores all trained model artefacts: `sarima_pjme.pkl` (fitted SARIMA object), `lstm_pjme_q70.pt` (LSTM state dictionary), and `lstm_pjme.scaler.json` (frozen min-max scaler parameters).
- **results/** — Stores evaluation outputs: `metrics.json` (MAE, RMSE per model run) and **figures/** (diagnostic plots).
- **requirements.txt** — Documents all Python dependencies, enabling consistent environment replication with `pip install -r requirements.txt`.

This file structure ensures that all intermediate and final outputs are organized and easily accessible to the user.

Result Sharing and Collaboration

Forecast results—whether in `.csv`, `.xlsx`, or `.pdf` format—are saved in the **ForecastOutput/** folder and can be shared via email, cloud storage, or other file-sharing platforms. Common use cases include:

- Sending PDF summary reports to stakeholders
- Inserting exported Excel forecasts into presentations
- Archiving forecasts for internal tracking and reproducibility

The combination of a structured directory layout, automatic model handling, and a user-friendly GUI makes the application highly portable and easy to share across teams. Whether the goal is to replicate results, deploy the tool to new users, or distribute forecasts to decision-makers, the folder-based architecture ensures a smooth and consistent exchange process.

10.4. Description of File Types

The forecasting application uses several file types to support its modular workflow, including data ingestion, forecast output, model persistence, and environment replication. Each file type plays a distinct role in ensuring interoperability, reproducibility, and scalability.

Overview of File Types

File Type	Purpose and Usage
.csv (Comma-Separated Values)	Primary format for data ingestion and forecast export. Users upload historical load data in CSV format, and the system generates the forecast output in the same structure, downloadable directly from the Streamlit interface.
.pt (PyTorch Checkpoint)	Binary format used to store the trained LSTM model state dictionary via <code>torch.save()</code> . The shipped checkpoint is <code>lstm_pjme-q70.pt</code> , trained with quantile loss at $\tau = 0.70$.
.pkl (Statsmodels Save)	Binary format used to persist the fitted SARIMA model object via statsmodels' built-in <code>save()</code> method. Stored as <code>sarima_pjme.pkl</code> and reloaded with <code>SARIMAXResults.load()</code> .
.json (Scaler Configuration)	Plain-text JSON file (<code>lstm_pjme.scaler.json</code>) storing the per-column min and max values fitted on the training set. Ensures identical scaling at inference time without re-fitting on new data.
requirements.txt	Text file listing all Python dependencies needed to run the forecasting tool. Enables environment replication across machines using <code>pip install -r requirements.txt</code> .

Table 10.2.: File Types Used in the Forecasting Workflow

By leveraging standard file formats, the application ensures seamless integration with existing tools, supports collaborative workflows, and simplifies deployment in enterprise or research environments.

10.5. Saving Models

To improve efficiency and support model reuse, the forecasting application automatically saves trained models to disk upon the completion of the training phase. This functionality enables the system to recognize and reuse previously trained models when a user uploads a dataset with a matching structure—thereby bypassing redundant retraining and saving computation time.

Model persistence is handled using `statsmodels`' built-in `save()` method for the SARIMA model and `PyTorch`'s `torch.save()` function for the LSTM model. This ensures that trained parameters, weights, and model configurations are fully serialized for reproducibility. Each trained model is stored in the `DevelopedModels/` directory and includes metadata describing:

- The dataset structure (e.g., columns, time granularity)
- The forecasting algorithm used (e.g., SARIMA, LSTM)
- Internal model parameters and configuration

This approach aligns with the application's modular design and supports:

- **Versioning:** Tracking changes in model logic or training configuration over time.
- **Reproducibility:** Re-running identical forecasts across systems or sessions without variation.

- **Deployment:** Rapidly deploying pre-trained models without requiring real-time training.

In future iterations, model saving may be extended to include auxiliary metadata such as training timestamps, performance metrics (e.g., RMSE), and feature importance summaries. These additions would enhance model governance, auditability, and explainability—particularly in enterprise or regulated environments.

10.6. Loading

The application loads data through two entry points, both of which converge on the same preprocessing pipeline.

PJME sample: Clicking *Load PJME sample* in the sidebar reads `data/raw/PJME_hourly.csv` directly into memory using `pandas.read_csv()`. The `Datetime` column is parsed as a `DatetimeIndex` and the series is sorted chronologically.

Custom upload: When a user uploads their own CSV, the application calls `coerce_series(df, dt_col, val_col)`, which:

1. Converts the selected timestamp column to `datetime` via `pd.to_datetime()`.
2. Drops rows where the timestamp cannot be parsed.
3. Detects and resolves duplicate timestamps by averaging the values (`groupby(level=0).mean()`).
4. Renames the value column to a unified `value` key.
5. Calls `ensure_hourly()` to reconstruct the full hourly index and linearly interpolate any missing hours.

After loading, the Streamlit sidebar pre-selects the most likely datetime and value columns based on column name heuristics and dtype inspection, but the user can override the selection via dropdown menus before running the forecast.

User Feedback and Transparency

The loading process is seamless and designed to be fully transparent. Upon completion, the user interface displays a message indicating whether:

- An existing model was reused
- A new model was trained from scratch

Benefits

This automated model management mechanism offers several key benefits:

- **Efficiency:** Reduces computation time by reusing existing models when appropriate.
- **Consistency:** Ensures repeatable and comparable forecasts for structurally similar datasets.
- **Usability:** Requires no action from non-technical users, supporting accessibility and ease of use.

This intelligent loading logic supports the overall goal of minimizing manual overhead while maintaining performance and reliability in real-world forecasting workflows.

11. Evaluation

11.1. Concepts

The core concept of this project is to build a practical, accurate, and user-friendly solution for forecasting future electricity demand using Time Series Analysis and Machine Learning. Electricity load forecasting is essential for grid operators and energy planners, as it directly influences unit commitment scheduling, load balancing, capacity planning, and energy trading strategies. Poor demand predictions can lead to supply shortfalls, costly over-generation, or frequency instability. The objective was therefore not only to produce accurate forecasts but also to bridge the gap between raw hourly meter data and actionable operational insights [GG+24]. Our concept was shaped by three major goals:

11.1.1. Understanding Electricity Demand as a Time Series Problem

Unlike general regression or classification problems, electricity load forecasting belongs to the category of time series forecasting, where past observations and their temporal order carry predictive information about future demand. Load data is governed by multiple overlapping periodic structures — intra-day peaks, weekly work/rest cycles, and annual seasonal patterns — that must be explicitly captured in the model.

We approached this by:

- Structuring the PJME hourly dataset chronologically by parsing and indexing the `Datetime` column, and resolving Daylight Saving Time anomalies including duplicate timestamps and spring-forward gaps.
- Conducting exploratory time series analysis to identify daily, weekly, and seasonal demand patterns, peak consumption hours, and irregular spikes in the `PJME_MW` series.
- Creating visualisations such as line plots and autocorrelation functions to reveal underlying consumption trends and short-term fluctuations in hourly electricity demand.
- Using these insights to guide model selection: applying SARIMA(1, 1, 1)(1, 1, 1)₂₄ to capture the strong seasonal and autoregressive structure, and LSTM with quantile loss ($\tau = 0.70$) to learn complex non-linear temporal dependencies across the 16-year historical dataset.

This part of the concept was critical to ensure that the forecasting approach is not only accurate but also interpretable and aligned with the dominant seasonal rhythms of electricity consumption [Sha16].

11.1.2. Building an Automated ML Pipeline for Forecasting

The second pillar of the concept was automation. In real-world scenarios, grid operators and energy analysts work with continuously updated meter data. Repeated manual preprocessing is inefficient and error-prone. We designed an end-to-end automated ML pipeline that:

- Ingests and validates hourly load data from a CSV file.

- Preprocesses the time series (resolving DST duplicates, enforcing hourly frequency, imputing gaps by linear interpolation).
- Constructs lag features and cyclical calendar encodings required by the LSTM model.
- Trains or loads a suitable time series forecasting model (SARIMA or LSTM).
- Evaluates the model with point-forecast performance metrics (MAE, RMSE).
- Generates a 720-hour ahead forecast and returns outputs in a structured visual format.

The pipeline is modular — each processing step is implemented in a dedicated source file under `src/` — improving maintainability and reproducibility. A strictly chronological train/test split (no random sampling) is enforced to preserve temporal order and prevent data leakage [HA21a].

11.1.3. Making Forecasting Accessible with a GUI

Recognising that not all end-users are data scientists, we designed a simple, clean Graphical User Interface (GUI) as part of the application. The goal was to make electricity demand forecasting accessible to energy analysts and grid operators, allowing them to:

- Upload a historical load dataset in `.csv` format.
- Run the full forecasting pipeline without writing code.
- View interactive charts of historical demand and predicted future values.
- Inspect the AI Forecasted Insights panel for peak demand, trend direction, and busiest weekday.
- Export the forecast results as a CSV file for use in reports or downstream tools.

The GUI was built with Streamlit and designed for responsiveness, minimal user interaction, and clear visual outputs. It targets non-technical users while remaining fully backed by the same modular pipeline used in the research evaluation [GG+24].

11.1.4. Additional Conceptual Elements

In designing the solution, the following additional aspects were considered:

- Model selection criteria, including data volume, forecast horizon, interpretability, and computational budget.
- Evaluation metrics aligned with operational impact: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on a held-out 720-hour test window, with RMSE giving disproportionate weight to peak-hour errors.
- Extensibility of the system to incorporate external variables such as temperature, humidity, and public holidays as additional LSTM input features.
- Practical reproducibility through frozen scaler parameters (`lstm_pjme.scaler.json`) and persistent model checkpoints, ensuring that re-running the pipeline on the same data always produces identical forecasts [HA21a].

Part VIII.

Deployment

12. Application Deployment

This chapter documents how the Energy Demand Forecaster is packaged and delivered to end-users. The deployment strategy, portable bundle pipeline, platform-specific launcher scripts, and shipped model artefacts are described in full, together with the rationale behind each design decision.

12.1. Deployment Strategy

12.1.1. Local-First Architecture

The Energy Demand Forecaster is deployed as a **self-contained portable bundle** that runs entirely on the user’s local machine. The application requires no internet connection after the initial download and transmits no data to external servers. This design was chosen over cloud or server-hosted alternatives for three reasons.

1. **Data privacy.** Electricity consumption data from industrial or commercial metering systems may be commercially sensitive. Local processing eliminates the risk of data exfiltration and removes the need for GDPR-compliant remote data handling [Beb+19].
2. **Reproducibility.** A frozen, self-contained bundle guarantees that every user runs identical package versions and model weights. There is no risk of upstream dependency changes silently altering forecast results between runs [HA21a].
3. **Accessibility.** Non-technical users — the primary target audience — cannot be expected to manage Python environments, virtual environments, or cloud credentials. A double-click launcher removes all setup friction [Fas25].

12.1.2. Comparison of Deployment Options

Table 12.1.: Comparison of deployment options considered for the Energy Demand Forecaster.

Option	Advantages	Disadvantages	Decision
Portable local bundle	No installation; offline; privacy-preserving; reproducible	Larger download; no multi-user access	Chosen
Cloud-hosted (e.g. Streamlit Community Cloud)	No local install; always latest version	Requires internet; data leaves machine; free tier limits	Not chosen
Docker container	Reproducible; cross-platform	Requires Docker install; steep learning curve for target users	Not chosen
Python <code>venv</code> + <code>pip</code>	Lightweight; familiar to developers	Requires Python pre-installed; not suitable for end-users	Developer only workflow

12.1.3. Streamlit as the Deployment Framework

Streamlit was selected as the application framework because it transforms a pure Python script into a browser-based interactive GUI without any front-end code. Alternative frameworks were evaluated but rejected:

- **Flask / FastAPI** require a separate front-end layer (HTML, CSS, JavaScript) and manual session management, adding significant development overhead for a prototype-scale research tool.
- **Dash** has a similar capability set but a steeper learning curve and more verbose callback syntax.
- **Gradio** is well-suited for single-input/single-output ML demos but lacks the multi-panel layout and stateful session management required by the forecasting pipeline.

Streamlit's `st.session_state`, live status containers (`st.status`), and native `st.line_chart` rendered the forecasting workflow in under 500 lines of application code, making it the most productive choice for a research-grade deployment [Fas25].

12.2. Portable Bundle Pipeline

12.2.1. Overview

The portable bundle is built by a single shell script, `build_portable.sh`, located in the project root. The script produces two platform-specific sub-bundles — one for Apple Silicon macOS (aarch64) and one for 64-bit Windows (x86_64) — inside the `dist/portable/` directory. The build process does not require any package manager installed on the host beyond `curl` and `tar`, both of which ship with macOS.

12.2.2. Build Steps

12.2.3. Build Pipeline Flowchart

12.2.4. Runtime Packages

The portable bundle installs packages from `requirements-runtime.txt`, which lists only the subset of dependencies required at runtime (a smaller set than the full development `requirements.txt`).

12.3. Bundle Directory Structure

Each platform sub-bundle (`mac/` or `windows/`) contains an identical application layer on top of a platform-specific Python runtime and package set.

Note that only the quantile-trained LSTM checkpoint (`lstm_pjme_q70.pt`) is bundled. The MSE-trained variant (`lstm_pjme.pt`) is retained in the repository for the academic comparison reported in Chapter 9 but is intentionally excluded from the bundle because the quantile model dominates it on all evaluation metrics (MAE -34% , RMSE -31%) and the GUI exposes a single LSTM option.

12.4. Platform Deployment

12.4.1. macOS Deployment

macOS Gatekeeper blocks execution of downloaded scripts that carry an Apple quarantine extended attribute (`com.apple.quarantine`). The `first_time_setup.command`

Table 12.2.: Steps performed by `build_portable.sh` in order.

Step	Action	Details
1	Download Python runtimes	Fetches CPython 3.12.13 pre-compiled binaries for aarch64-apple-darwin and x86_64-pc-windows-msvc from the <code>python-build-standalone</code> project (tag 20260508). Downloads are cached in <code>dist/downloads/</code> ; subsequent builds skip this step.
2	Extract runtimes	Unpacks each tarball into <code>dist/portable/-mac/python/</code> and <code>dist/portable/windows/python/</code> respectively.
3	Install packages (macOS)	Runs <code>pip install -target mac/packages/ -r requirements-runtime.txt</code> using the bundled Mac Python. Installs native arm64 wheels.
4	Install packages (Windows)	Runs <code>pip install -target windows/packages/ with -platform win_amd64 -only-binary=:all:,</code> cross-installing Windows wheels from the host Mac. PyTorch (CPU) wheels are fetched from <code>download.pytorch.org/whl/cpu</code> .
5	Copy application source	Copies <code>app/</code> , <code>src/</code> , <code>data/raw/PJME_hourly.csv</code> , and pre-trained model artefacts into both platform bundles.
6	Write launchers	Generates <code>run.command</code> + <code>first_time_setup.command</code> for macOS and <code>run.bat</code> for Windows.
7	Clean cache	Removes all <code>__pycache__</code> directories and <code>.pyc</code> files from the bundle to reduce size.

Table 12.3.: Runtime packages installed into the portable bundle.

Package	Min. Version	Role in application
streamlit	1.30	Browser-based GUI framework
pandas	2.0	Data ingestion, resampling, feature engineering
numpy	1.24	Numerical array operations
torch (CPU)	2.0	LSTM inference and training
statsmodels	0.14	SARIMA fitting and forecasting
scikit-learn	1.3	MinMaxScaler for LSTM features
matplotlib	3.7	Diagnostic plot generation (report scripts)

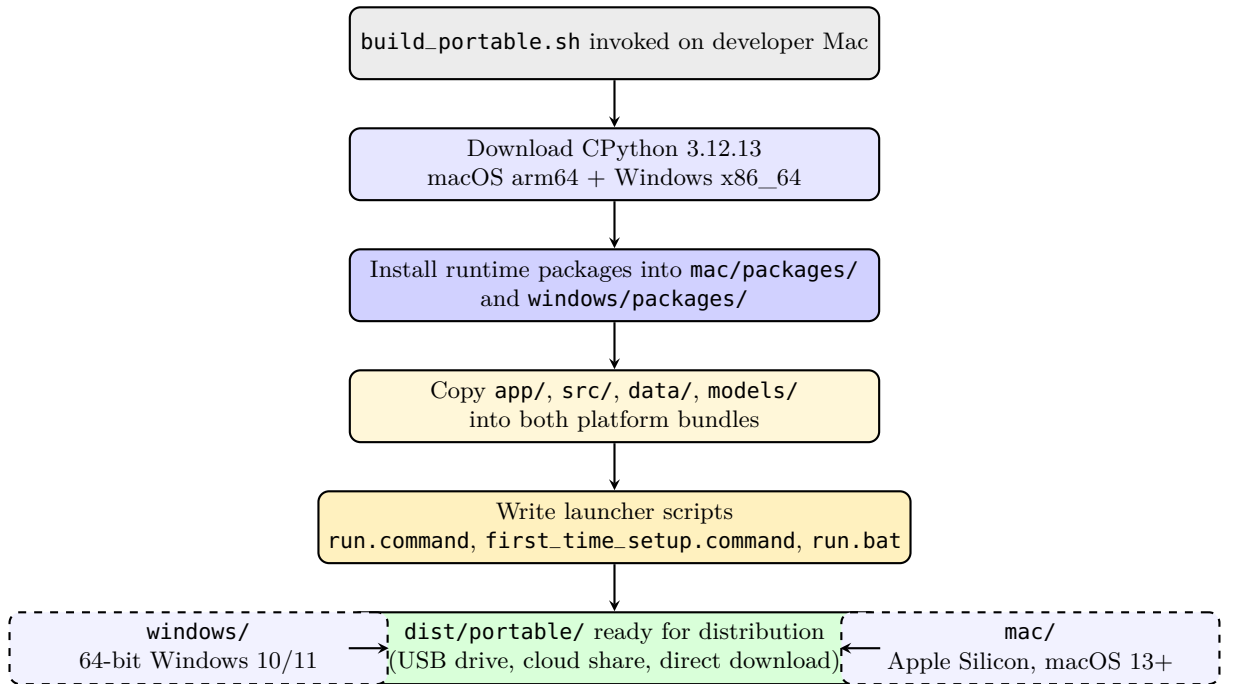


Figure 12.1.: Portable bundle build pipeline. The script is executed once on a developer Mac; the resulting `dist/portable/` folder can be copied to any distribution medium.

Table 12.4.: Directory layout of each platform-specific portable bundle.

Path	Platform	Contents
<code>python/</code>	Both	CPython 3.12.13 standalone runtime
<code>packages/</code>	Both	Installed runtime dependencies
<code>app/streamlit_app.py</code>	Both	Streamlit GUI entry point
<code>src/</code>	Both	All source modules (<code>data_loader.py</code> , <code>preprocessing.py</code>)
<code>data/raw/PJME_hourly.csv</code>	Both	Bundled 145 392-row sample dataset
<code>models/lstm_pjme_q70.pt</code>	Both	Pre-trained LSTM checkpoint ($\tau = 0.70$)
<code>models/lstm_pjme_scaler.json</code>	Both	Frozen min-max scaler parameters
<code>run.command</code>	macOS	Double-click launcher
<code>first_time_setup.command</code>	macOS	One-time Gatekeeper quarantine removal
<code>run.bat</code>	Windows	Double-click launcher

script removes this attribute from the entire bundle directory using `xattr -dr` and sets the executable bit on the launchers. This step must be performed once, immediately after unpacking the bundle; subsequent launches require only a double-click on `run.command`.

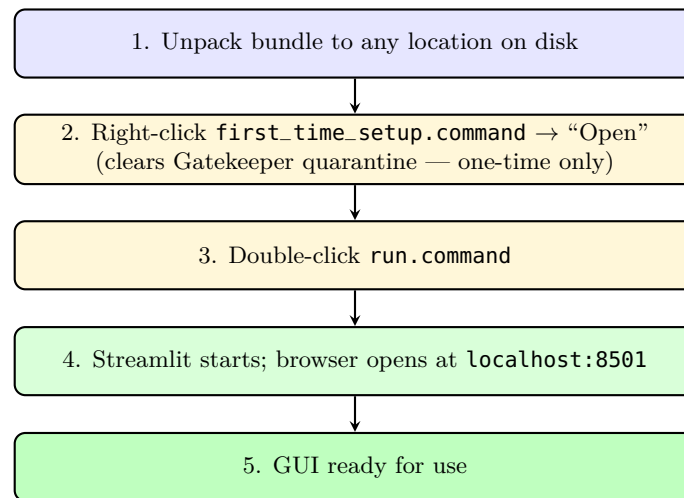


Figure 12.2.: macOS deployment sequence. Step 2 (Gatekeeper setup) is required only once per bundle copy.

The `run.command` script sets `PYTHONPATH` to point at the bundled `packages/` directory, then invokes the bundled Python interpreter directly:

Listing 12.1: Core commands executed by `run.command` (macOS).

```

export PYTHONPATH="$BUNDLE_DIR/packages:$PYTHONPATH"
"$BUNDLE_DIR/python/bin/python3" -m streamlit run \
  "$BUNDLE_DIR/app/streamlit_app.py" \
  --server.headless false \
  --browser.gatherUsageStats false
  
```

12.4.2. Windows Deployment

On Windows, no separate setup step is required. The bundled Python runtime is a standalone `.exe` that does not interact with the system Python installation. The user double-clicks `run.bat`, which sets `PYTHONPATH` to the bundled `packages\` directory and starts Streamlit:

Listing 12.2: Core commands in `run.bat` (Windows).

```

set "PYTHON=%BUNDLE_DIR%\python\python.exe"
set "PYTHONPATH=%BUNDLE_DIR%\packages;%PYTHONPATH%"
"%PYTHON%" -m streamlit run ^
  "%BUNDLE_DIR%\app\streamlit_app.py" ^
  --server.headless false ^
  --browser.gatherUsageStats false
  
```

Windows SmartScreen may display an “Unrecognized app” warning on the first launch. Users must click **More info** followed by **Run anyway** to proceed; this is standard behaviour for unsigned executables distributed outside the Microsoft Store.

Table 12.5.: Minimum system requirements for the portable bundle.

Requirement	macOS	Windows
Operating system	macOS 13 (Ventura) or later	Windows 10 or later (64-bit)
Processor	Apple Silicon (arm64)	Any 64-bit x86 CPU
RAM	4 GB minimum	4 GB minimum
Free disk space	≈ 2 GB	≈ 2 GB
Browser	Any modern browser (Chrome, Firefox, Safari, Edge)	
Internet	Not required after initial bundle download	

12.5. Shipped Model Artefacts

12.5.1. LSTM Checkpoint

The pre-trained LSTM checkpoint `lstm_pjme_q70.pt` is a PyTorch state dictionary saved with `torch.save()`. It encodes the weights of a two-layer stacked LSTM (64 hidden units, dropout 0.2) with a linear output head projecting directly to 720 steps. Training used the quantile (pinball) loss at $\tau = 0.70$ [MSSJ22a]:

$$\mathcal{L}_\tau(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N \max(\tau(y_i - \hat{y}_i), (\tau - 1)(y_i - \hat{y}_i)) \quad (12.1)$$

Setting $\tau = 0.70$ biases the model towards over-prediction, which recovers demand peaks that a symmetric MSE-trained model systematically flattens due to regression-to-the-mean. The resulting test-window performance on the PJME dataset is MAE = 3 307 MW and RMSE = 4 156 MW — a 34 % MAE reduction relative to the MSE-trained variant [GG+24].

12.5.2. Scaler Parameters

The frozen min-max scaler is stored as a plain JSON file (`lstm_pjme.scaler.json`) containing the per-column minimum and maximum values fitted exclusively on the training split. At inference time, the application loads this file via `load_scaler()` and applies the identical scaling to the input window, preventing any information from the test period from influencing the prediction.

Storing the scaler as JSON rather than a binary `.pkl` file makes the artefact human-readable, version-control friendly, and independent of the Python version used to generate it [HA21a].

12.5.3. Sample Dataset

The bundled `PJME_hourly.csv` (145 392 rows, January 2002 – August 2018) allows the GUI to demonstrate instant LSTM inference without requiring any user data. Because the pre-trained checkpoint was fitted on this exact dataset (with a strictly chronological train/test split), the predicted values on the 720-hour test window reproduce the published benchmark metrics exactly.

12.6. Developer Deployment

Developers who wish to modify the application or retrain the models do not use the portable bundle. Instead, a standard Python virtual environment is recommended:

Listing 12.3: Developer setup from the project root directory.

```
python -m venv .venv
source .venv/bin/activate      # macOS / Linux
# .venv\Scripts\activate      # Windows

pip install -r requirements.txt # full dev dependencies

streamlit run \
  Code/Python/EnergyDemandForecaster/app/streamlit_app.py
```

The full `requirements.txt` additionally includes `jupyter`, `seaborn`, and `pyarrow` for notebook exploration and figure generation. These are excluded from the portable bundle to minimise its size.

Table 12.6.: Summary of the two deployment modes supported by the project.

Attribute	Portable bundle	Developer workflow
Python source	Bundled (3.12.13 standalone)	System / virtual environment
Packages	Pre-installed in <code>packages/</code>	Installed via <code>pip</code>
Launch	Double-click script	<code>streamlit run ...</code>
Target audience	End-users	Developers / researchers
Rebuild needed?	Only on code change	Never
Disk footprint	≈ 2 GB	≈ 500 MB
Offline capable	Yes	Yes (after <code>pip install</code>)

12.7. Full Inference Script – The `IN0` file

Part IX.

Monitoring

13. Monitoring

13.1. Idea

In the context of electricity load forecasting, monitoring refers to the continuous evaluation of both model performance and input data quality to ensure that forecast outputs remain accurate, interpretable, and aligned with real-world consumption behaviour [Cha13]. Since electricity demand is influenced by temperature, seasonal cycles, and socio-economic factors that are inherently dynamic, continuous monitoring is essential for detecting anomalies, identifying emerging demand trends, and preventing forecast degradation over time [HA18].

The forecasting system integrates standard evaluation metrics, including Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). These metrics enable systematic comparison between the two modelling approaches using identical training and test splits, providing a transparent and reproducible basis for model evaluation [Bai21].

Visual monitoring is embedded into the application interface through side-by-side plots of actual versus predicted **PJME_MW** values for both models. These visualisations support interpretability by allowing users to assess model performance, identify unexpected forecast behaviour, and detect early signs of model drift caused by structural changes in electricity demand patterns [MWH18].

To maintain forecast reliability over time, the system supports periodic model retraining using updated consumption data. This process may involve reusing existing SARIMA and LSTM architectures, adjusting hyperparameters such as SARIMA seasonal orders or LSTM hidden layer sizes, and incorporating additional exogenous variables such as temperature or public holiday indicators to further improve predictive accuracy.

13.2. Plan/Description

The monitoring plan outlines a structured approach to maintaining data integrity, model reliability, and business relevance across the forecasting lifecycle. It combines technical safeguards with user-facing outputs to ensure that forecasts remain accurate, interpretable, and aligned with current conditions.

- **Periodic Data Collection:** Although real-time data ingestion is not yet implemented, the system supports periodic updates through manual uploads of updated **PJME_MW** hourly consumption data. The design is structured to enable future integration with PJM Interconnection operational data feeds, allowing automated synchronization and improved data freshness.
- **Anomaly Detection and Validation:** The preprocessing pipeline includes automated validation checks for duplicate timestamps (caused by Daylight Saving Time transitions, resolved by averaging both readings via `groupby(level=0).mean()`), irregular hourly frequency gaps, and missing values in the **PJME_MW** series introduced by DST spring-forward transitions. These procedures ensure that only temporally consistent and high-quality data is passed to the modelling stage.

- **Model Pipeline Integrity:** Each stage of the pipeline—data preprocessing, SARIMA model fitting, and LSTM training—is independently validated. The system enforces checks on datetime formatting, hourly frequency consistency, and minimum data length requirements before initiating any modelling process.
- **Comprehensive Reporting:** Forecast outputs are exported in CSV format, including predicted **PJME_MW** values aligned with their corresponding timestamps. In addition, the application generates visual comparisons of actual versus predicted values and reports evaluation metrics such as MAE, RMSE, and MAPE for both SARIMA and LSTM models, ensuring transparent and reproducible model evaluation [MWH18].
- **Support for Operational Decision-Making:** By providing transparent and traceable electricity demand forecasts, the system supports operational decision-making processes such as load balancing, unit commitment scheduling, and energy trading strategies [MWH18].

Together, these components form a continuous monitoring and validation loop that balances automation with interpretability, ensuring that forecasts remain both technically robust and operationally valuable over time.

13.3. Getting New Data

The forecasting application allows users to update their dataset by manually uploading a new file via the user interface. Upon upload, the preprocessing pipeline is triggered to validate the structure of the incoming file. This includes confirming the presence of the required **Datetime** column and the numeric target variable **PJME_MW**, parsing the datetime format, checking for hourly frequency regularity, and detecting any duplicate timestamps caused by Daylight Saving Time transitions.

Although the current version does not support automated or scheduled ingestion from external sources such as APIs or live grid data feeds, it fully supports manual upload of CSV files containing a datetime column and a numeric demand column. The sidebar file uploader accepts any CSV that conforms to this structure, enabling users to replace the bundled PJME sample dataset with their own hourly consumption data.

Following successful upload and column selection, the application does not perform automatic model reuse or caching. Instead, both SARIMA and LSTM models are retrained from scratch for each execution using the provided dataset. An exception is made for the LSTM model when the default PJME dataset is used: in this case, a pre-trained checkpoint (`lstm_pjme_q70.pt`) and associated scaler (`lstm-pjme.scaler.json`) are loaded directly from the bundled distribution, reducing computational overhead and avoiding redundant training.

This process ensures that forecasts remain aligned with the latest information while maintaining computational efficiency and minimizing unnecessary retraining. Planned future enhancements may include support for automated data ingestion from ERP systems, RESTful APIs, or shared cloud directories to enable more dynamic and near real-time forecasting workflows.

13.4. Updating Data in ML Pipeline

The forecasting system employs an adaptive and modular machine learning pipeline that dynamically responds to new dataset uploads. Its primary objective is to determine

whether an existing model can be reused or if retraining is necessary, balancing computational efficiency with model relevance. Compatibility is assessed based on column names, date formatting, time granularity, and overall schema consistency.

- **SARIMA Pipeline:** The SARIMA model is trained from scratch on each run using `statsmodels`, with a fixed seasonal configuration of `(1,1,1)(1,1,1,24)`, applied to the uploaded or sampled hourly demand series. After training, the fitted model is persisted as `models/sarima_pjme.pkl` using `pickle`. The pipeline then generates a 720-hour ahead forecast. Model performance is evaluated using MAE and RMSE on a held-out test set, and the results are stored in `results/metrics.json`.
- **LSTM Pipeline:** The LSTM pipeline follows a structured sequence of steps:
 - **Preprocessing:** Ensures hourly frequency consistency using `ensure_hourly()`, which handles Daylight Saving Time-related missing hours through linear interpolation.
 - **Feature Engineering:** Constructs temporal input features using `build_lstm_features()`, including cyclical encodings for hour of day, day of week, and month.
 - **Scaling:** Applies a `MinMaxScaler` fitted only on the training split to normalize all input features to the range `[0,1]`, preventing data leakage.
 - **Training and Evaluation:** Trains a stacked two-layer PyTorch LSTM model (hidden size = 64) using the Adam optimiser. The shipped checkpoint (`lstm_pjme_q70.pt`) was trained with quantile (pinball) loss at $\tau = 0.70$, which penalises under-prediction more heavily than over-prediction and recovers demand peaks. For custom uploads, the loss type is configurable via `TrainConfig.loss_type`. Early stopping with patience 5 is applied. MAE and RMSE are computed on the same 720-hour test window used for SARIMA, ensuring a fair comparison.
 - **Export:** Saves the trained model parameters to `models/lstm_pjme.pt` using `torch.save()`, along with the scaler configuration stored in `models/lstm_pjme.scaler.json` to support consistent future inference without retraining.

This entire workflow is governed by a structured pipeline, with preprocessing logic modularized into dedicated source files under the `src/` directory, including `data_loader.py`, `preprocessing.py`, `feature_engineering.py`, `sarima_model.py`, `lstm_model.py`, and `evaluation.py`. Forecast results and evaluation metrics are automatically exported to `results/metrics.json` and can also be downloaded as CSV files directly through the Streamlit interface. These design choices ensure that the pipeline remains modular, reproducible, and accessible to both technical and non-technical users across macOS and Windows platforms.

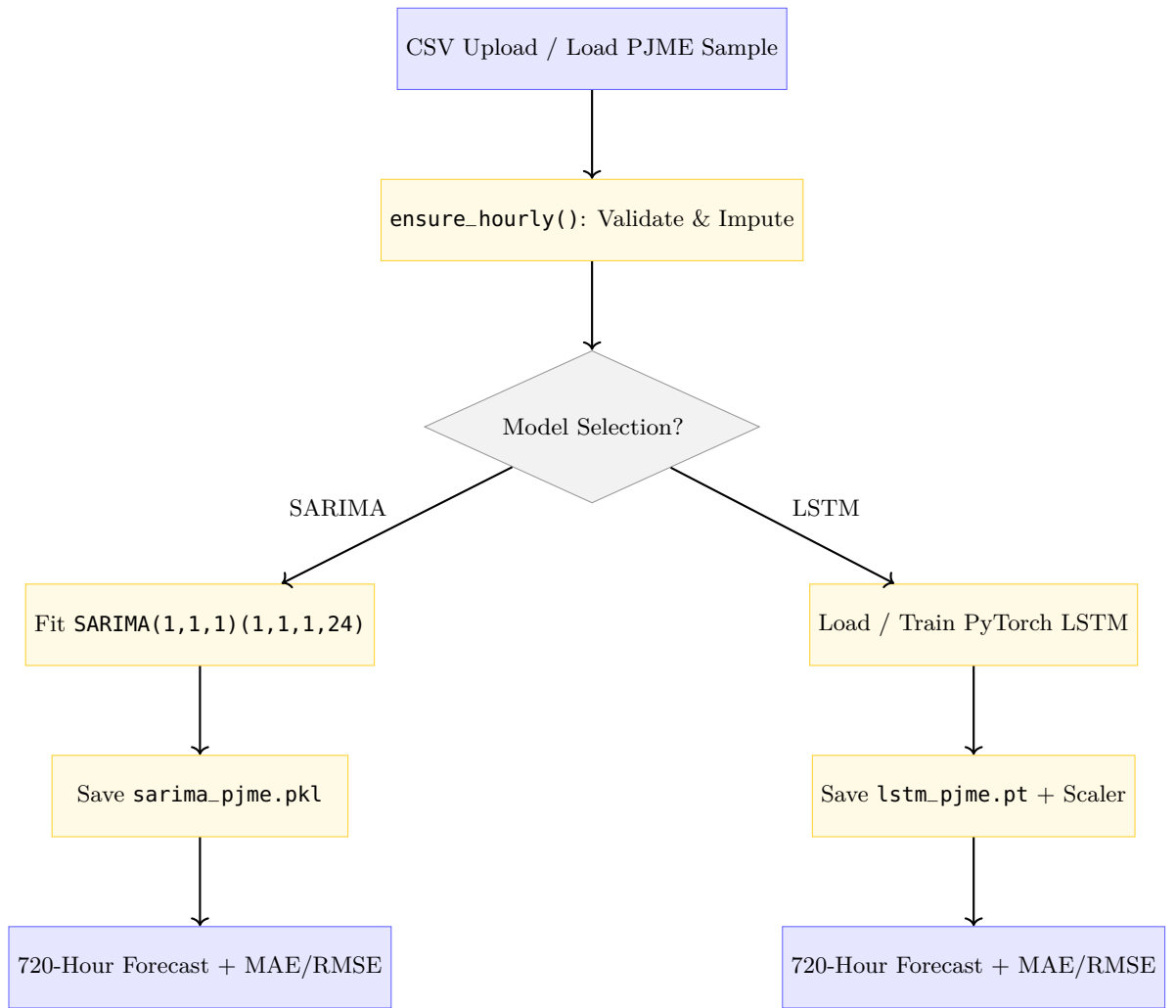


Figure 13.1.: Energy Demand Forecasting Pipeline: SARIMA vs LSTM

13.5. Checks

The goal of these checks is to ensure that only clean, well-structured, and temporally consistent datasets proceed into the forecasting pipeline, either for SARIMA model fitting or LSTM model training.

The key validation and preprocessing procedures executed by the pipeline include:

- **File Format Verification:** Confirms that the uploaded file is in `.csv` format and can be read by the backend using `pandas.read_csv()`, with both `Datetime` and `PJME_MW` columns present.
- **Datetime Parsing and Indexing:** Parses the `Datetime` column into a pandas `DatetimeIndex` using `pd.to_datetime()`, ensuring consistent hourly time series indexing for both SARIMA and LSTM models.
- **Duplicate Timestamp Resolution:** Identifies duplicate timestamps caused by Daylight Saving Time transitions and resolves them by averaging both readings via `groupby(level=0).mean()`, preserving the total measured energy across the transition hour without introducing a step discontinuity in the series.

- **Hourly Frequency Enforcement:** Reindexes the series to a complete hourly frequency using `ensure_hourly()`. This step addresses missing hours introduced by Daylight Saving Time transitions and imputes them using linear interpolation. An `imputed` flag is added to indicate affected observations.
- **Train/Test Split:** Splits the cleaned series into training and testing sets, using a fixed 720-hour test window via `train_test_split_timeseries()`. SARIMA uses a two-year training window, while LSTM uses a ten-year training window.
- **Missing Value Check:** Ensures that no unresolved `NaN` values remain in the `PJME_MW` column after imputation before model training begins.
- **Minimum Length Threshold:** Verifies that the dataset contains sufficient observations for seasonal parameter estimation in SARIMA (at least one full annual cycle) and for constructing LSTM input sequences based on the configured lookback window.

All preprocessing steps are executed transparently within the pipeline, with status messages displayed in the Jupyter Notebook console and a live progress indicator shown in the Streamlit interface. If all checks are passed, the validated dataset is forwarded to the selected model—SARIMA or LSTM—ensuring a seamless pipeline flow. This structured validation mechanism improves the robustness and reproducibility of the forecasting system while minimizing the need for manual preprocessing or user intervention.

13.6. Code: Functions

The forecasting application uses a modular set of Python functions organised under the `src/` directory to support data loading, preprocessing, modelling, and evaluation. These functions form the foundation of the pipeline’s internal logic.

1. Data Loading and Preprocessing

- `load_pjme(path)` — Reads `PJME_hourly.csv`, parses the `Datetime` column, removes duplicate timestamps caused by DST transitions, and returns a sorted `DatetimeIndex` `DataFrame`.
- `ensure_hourly(df)` — Reindexes the series to a complete hourly frequency, imputes the 30 DST-induced missing values using linear interpolation, and adds an `imputed` flag column to track affected rows.
- `train_test_split_timeseries(df, test_hours, train_years)` — Splits the cleaned series chronologically into training and test sets, using a 2-year window for SARIMA and a 10-year window for LSTM.

2. Feature Engineering

- `build_lstm_features(df, target)` — Constructs temporal input features from the `DatetimeIndex`, including cyclical encodings of hour of day, day of week, and month, appended with the `PJME_MW` target variable for LSTM input.

3. Model Training and Forecasting

- `fit_sarima(train, order, seasonal_order)` — Fits a `SARIMA(1,1,1)(1,1,1,24)` model using `statsmodels` and returns the fitted model object.

- `forecast(fit, steps)` — Generates point forecasts and confidence intervals for the specified number of steps ahead using the fitted SARIMA model.
- `save_model(fit, path)` — Saves the fitted SARIMA model to `models/sarima_pjme.pkl` using `pickle`.
- `train_lstm(train_arr, val_arr, cfg)` — Trains a stacked two-layer PyTorch LSTM (hidden size = 64) using the Adam optimiser with loss function determined by `cfg.loss_type` ("quantile" at $\tau = 0.70$ for the shipped checkpoint; "mse" for custom uploads) and early stopping with patience 5 based on validation loss. Returns the trained model and per-epoch loss history.
- `save_lstm_model(model, path)` — Saves the LSTM state dictionary to `models/lstm_pjme.pt` using `torch.save()`, along with the scaler configuration stored in `models/lstm_pjme.scaler.json`.

4. Evaluation and Reporting

- `mae(actual, predicted)` — Computes Mean Absolute Error between actual and predicted values.
- `rmse(actual, predicted)` — Computes Root Mean Squared Error for forecast accuracy evaluation.
- `residual_diagnostics(fit, lags)` — Performs Ljung-Box tests and computes residual statistics for SARIMA model validation.
- `plot_forecast(train_tail, test, point_forecast, conf_int)` — Generates a comparison plot of historical demand, actual values, and forecasts, saved to `results/figures/`.
- `plot_residual_acf(fit, lags)` — Plots the autocorrelation function (ACF) of SARIMA residuals to assess remaining structure.
- `append_metrics(path, model, region, horizon, metrics)` — Appends MAE, RMSE, and model configuration metadata to `results/metrics.json` for cross-model comparison.

These functional components enable a high degree of modularity, transparency, and reproducibility across the entire forecasting pipeline.

13.7. Code: Tests and Validation

The forecasting application does not include a formal automated test suite, but correctness is validated at three levels.

Preprocessing invariant checks: After running `ensure_hourly()` on `PJME_hourly.csv`, the following properties are verified manually and in the development notebooks:

- Total rows: 145 392 (no row gain or loss from the original file).
- Zero remaining NaN values in `PJME_MW` after interpolation.
- `DatetimeIndex` is strictly regular at hourly frequency with no gaps.
- Exactly 30 rows flagged as `imputed=True`, corresponding to the known DST spring-forward gaps.

Pipeline regression tests: After any code change, the full pipeline is re-run on the PJME dataset and the output of `results/metrics.json` is compared against the documented benchmark:

- SARIMA MAE $\approx 11\,662$ MW, RMSE $\approx 12\,932$ MW.
- LSTM-q70 MAE $\approx 3\,307$ MW, RMSE $\approx 4\,156$ MW.

A deviation exceeding 5 % on either metric indicates a regression in feature engineering, scaling, or model configuration.

GUI smoke tests: The Streamlit interface is verified manually after each release by:

1. Loading the PJME sample and confirming the 145 392-row preview count.
2. Running SARIMA and verifying a 720-point forecast series is returned.
3. Running LSTM with the pre-trained checkpoint and confirming sub-second inference.
4. Uploading a short custom CSV (<720 rows) and verifying the expected error message is displayed.
5. Downloading the forecast CSV and confirming the `Datetime` and `forecast` columns are correct.

13.8. Privacy

The forecasting application is designed with a local-first architecture, meaning that all data processing is performed entirely on the user’s machine. At no stage are datasets transmitted, stored remotely, or shared with third-party servers. This local execution approach significantly reduces privacy risks and aligns with responsible data handling and privacy-preserving design principles [Beb+19].

Only files explicitly uploaded by the user, typically CSV or Excel datasets—are processed. These files remain confined to the local execution environment and are not logged, cached, or shared externally. No personally identifiable information (PII), user credentials, or sensitive metadata are collected or stored by the system.

All model artefacts generated during training are stored locally within the `models/` directory. The SARIMA model is saved as `sarima_pjme.pkl`, while the LSTM model is saved as `lstm_pjme.pt`, along with its associated scaler configuration in `lstm_pjme.scaler.json`. These files are strictly tied to the training data and model configuration and do not contain or reference any user identity information.

Forecast outputs are saved in structured CSV format within the `results/` directory and can also be downloaded directly through the Streamlit interface using the **Download forecast CSV** button. Data remains local to the user’s machine unless explicitly exported by the user.

While the current implementation does not require network connectivity, the system architecture anticipates future integration with cloud platforms or enterprise systems. In such scenarios, optional enhancements may include encryption of datasets and model artifacts, user authentication protocols, audit logging, and full compliance with data protection regulations such as GDPR.

These baseline design choices ensure privacy is respected by default and establish a secure foundation for scaling the application in privacy-sensitive environments.

13.9. Robustness

Robustness refers to the forecasting application’s ability to operate reliably under a wide range of input conditions, data irregularities, and operational scenarios — without failure or significant degradation in output quality. Several architectural and functional choices contribute to this resilience:

- **Automated Preprocessing and Validation:** All datasets undergo structured preprocessing checks via `load_pjme()` and `ensure_hourly()`, including datetime parsing, duplicate timestamp removal, hourly frequency enforcement, and linear interpolation of DST-induced missing values. Only clean, temporally consistent data is passed to model training.
- **Dual Model Strategy:** The system implements both SARIMA and LSTM as independent, parallel forecasting pipelines applied to the same held-out test window. This ensures that both a classical statistical baseline and a deep learning approach are evaluated, improving robustness against limitations of any single modelling method.
- **Data Leakage Prevention:** The `MinMaxScaler` for the LSTM model is fitted exclusively on the training split and applied to validation and test sets separately, preventing information from the test period from influencing training or scaling.
- **Offline Execution Environment:** The entire application runs locally as a self-contained package for both macOS and Windows, with all dependencies included. It is therefore independent of internet connectivity, cloud service availability, or external API failures, improving availability and reproducibility.
- **Pre-trained Model Caching:** The LSTM model includes a pre-trained quantile checkpoint (`lstm_pjme_q70.pt`) and scaler configuration (`lstm_pjme_scaler.json`). This avoids retraining overhead when the default PJME dataset is used, improving response time and ensuring consistent outputs.
- **Modular Codebase:** Each major module — `data_loader.py`, `preprocessing.py`, `feature_engineering.py`, `sarima_model.py`, `lstm_model.py`, and `evaluation.py` — operates with clearly defined input and output interfaces. This separation of concerns improves testability, maintainability, and fault isolation.
- **Persistent Metrics Logging:** All evaluation results — including MAE, RMSE, model configuration, and forecast horizon — are appended to `results/metrics.json` after each run, enabling reproducible comparisons and transparent performance tracking across model variants.

Together, these components enable the forecasting system to handle imperfect data, DST-related irregularities, and structural variability in a stable manner — making it suitable for both academic and operational energy forecasting environments.

13.10. Process

The forecasting application follows a modular, end-to-end workflow designed to guide users from raw data input to forecast output with minimal intervention. This structured pipeline emphasises automation, transparency, and reproducibility — making it suitable for both academic and operational energy forecasting environments.

The key stages are as follows:

1. **Data Input:** The user either selects **Load PJME Sample** to load the bundled `PJME_hourly.csv`, or uploads a custom `.csv` file containing a datetime column and a numeric demand column via the Streamlit sidebar.
2. **Preprocessing and Validation:** The dataset is processed using `load_pjme()` and `ensure_hourly()`. This includes parsing the `Datetime` column, removing duplicate timestamps caused by DST transitions, enforcing hourly frequency, and imputing the 30 DST-induced missing values using linear interpolation.
3. **Model Selection:** The user selects either SARIMA or LSTM from the sidebar, along with a forecast horizon (default: 720 hours). The pipeline is configured accordingly based on the selected model.
4. **Model Training and Forecasting:** The selected model is trained on the cleaned time series. SARIMA fits a `SARIMA(1,1,1)(1,1,1,24)` model on a 2-year training window and generates a 720-hour forecast with confidence intervals. The LSTM model either loads the pre-trained checkpoint (`lstm_pjme_q70.pt`) or trains a stacked two-layer PyTorch LSTM on a 10-year window, followed by a 720-hour forecast in a single forward pass.
5. **Evaluation:** MAE and RMSE are computed against the held-out final 720 hours for both models. Results are appended to `results/metrics.json` for persistent cross-model comparison.
6. **Export:** The forecast visualization is displayed in the main panel. The user can download the predictions as a `.csv` file via the **Download forecast CSV** button in the Streamlit interface.

This modular pipeline ensures consistent, reproducible, and low-friction forecasting for hourly electricity demand data.

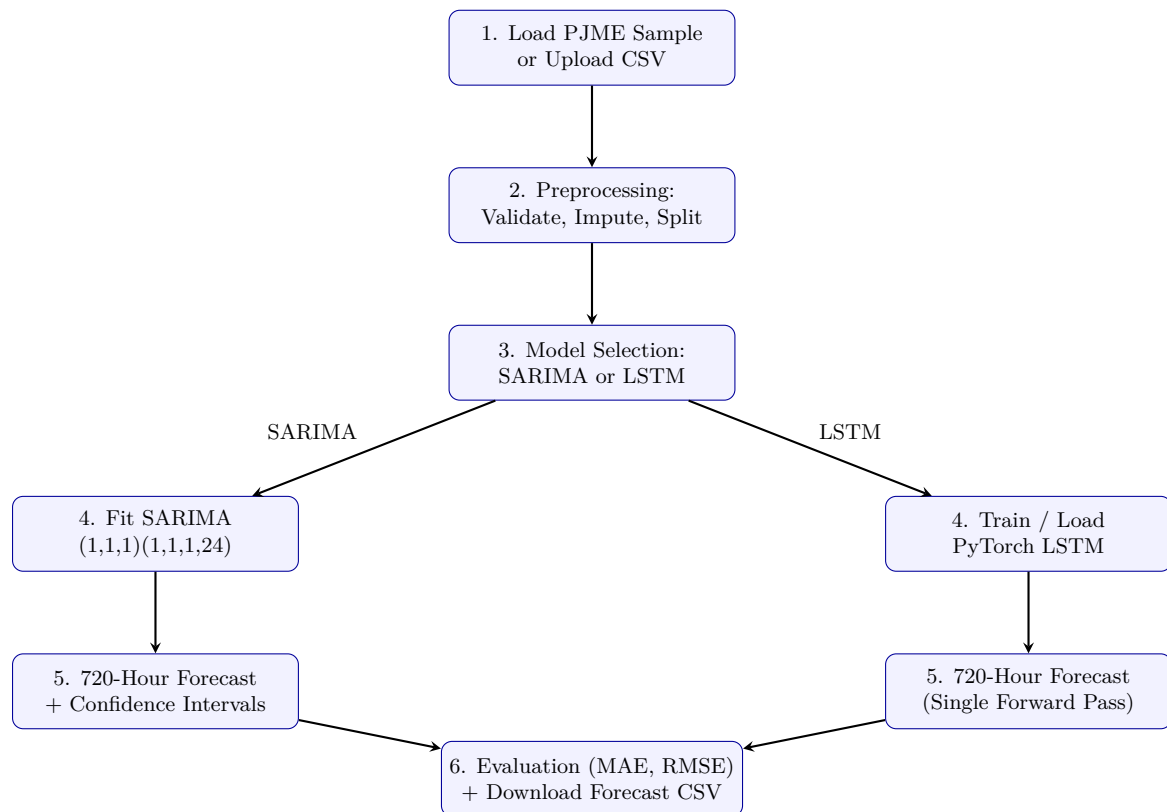


Figure 13.2.: Energy Demand Forecasting Pipeline: From Data Input to Export

Part X.

**Verification - Evaluation -
Conclusion**

14. Evaluation

14.1. Concept

The core concept of this project is to build a practical, accurate, and understandable solution for forecasting future electricity demand using Time Series Analysis and Machine Learning. Time series analysis is important because electricity consumption changes over time and depends on past demand patterns, daily cycles, weekly behaviour, seasonal effects, and peak load periods.

Electricity demand forecasting is a relevant real-world problem because grid operators, utility companies, and energy planners need reliable predictions to balance supply and demand. If demand is underestimated, the system may face shortages, operational stress, or the need for expensive backup generation. If demand is overestimated, energy production and resource allocation may become inefficient. Therefore, forecasting supports better planning, stable grid operation, and efficient energy management [HF16; GLA22].

The project focuses on forecasting the next 30 days of hourly electricity demand. Since one day contains 24 hours, the forecast horizon is:

$$30 \times 24 = 720 \text{ hours}$$

This means the system must predict 720 future hourly electricity load values. The selected dataset, `PJME_hourly.csv`, contains historical hourly electricity load data from the PJM Eastern region. This makes it suitable for identifying hourly, daily, weekly, and seasonal electricity demand patterns [PJM24].

The concept is built around three main goals:

- **Understanding electricity demand as a time series problem**
- **Building a forecasting pipeline using SARIMA and LSTM**
- **Evaluating and comparing the forecasting results**

14.1.1. Understanding Electricity Demand as a Time Series Problem

Unlike ordinary regression tasks, electricity demand forecasting belongs to time series forecasting. In time series data, the order of observations is important because past values influence future values. Electricity load is not random; it usually follows repeated patterns such as higher demand during working hours, different behaviour on weekends, and seasonal peaks during summer cooling or winter heating periods.

For this reason, the dataset must be handled chronologically. The timestamp column is converted into a proper datetime format, sorted in time order, and used as the index of the time series. This allows the models to learn from the correct historical sequence and avoids mixing past and future observations.

The project considers important time series characteristics such as:

- **Trend:** Long-term increase or decrease in electricity consumption.
- **Seasonality:** Repeating patterns, such as daily, weekly, or yearly cycles.
- **Noise:** Random fluctuations in demand.

- **Peaks:** High demand periods caused by weather, working hours, or special events.
- **Autocorrelation:** The relationship between current demand and previous demand values.

This understanding helps in choosing suitable forecasting algorithms. SARIMA is useful for modelling structured seasonal patterns, while LSTM is useful for learning complex sequential relationships in historical demand data [Dub+21; BP23].

14.1.2. Exploratory Time Series Understanding

Before comparing the models, the time series was explored visually. The full historical PJME load curve shows repeated seasonal cycles and demand spikes over multiple years. These repeated patterns confirm that the dataset is suitable for time series forecasting and not only for ordinary tabular prediction.

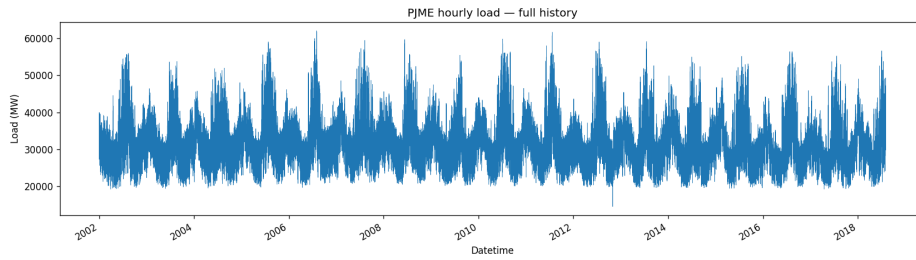


Figure 14.1.: PJME hourly electricity load over the full historical period

The average demand by hour, day of week, and month also gives useful information. Hourly demand is lower during night and early morning hours and increases during the day. Weekly behaviour shows different weekday and weekend demand. Monthly behaviour shows seasonal variation, which is important for both SARIMA and LSTM modelling.

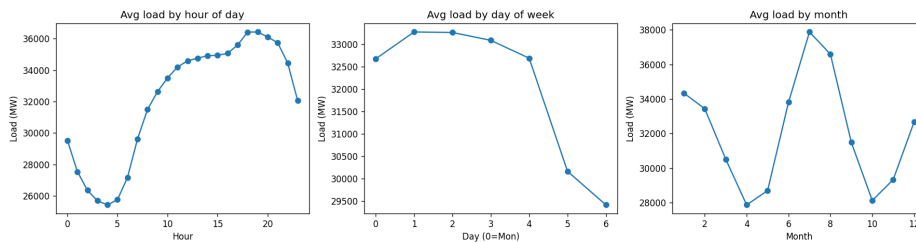


Figure 14.2.: Average electricity load by hour of day, day of week, and month

The ACF and PACF plots were also used to understand autocorrelation. The ACF plot shows strong repeating dependence, especially around daily lags. This supports the use of lag-based features and seasonal forecasting models.

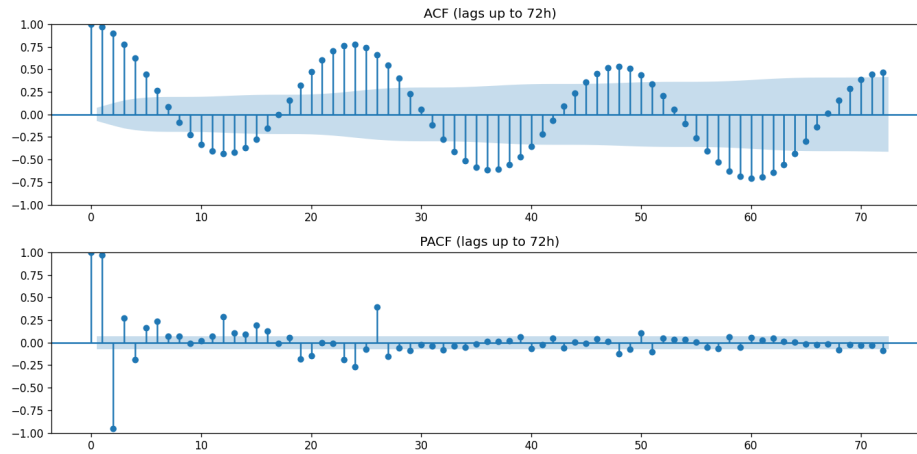


Figure 14.3.: ACF and PACF plots for PJME load up to 72 hourly lags

14.1.3. Forecasting Pipeline Concept

The second part of the concept is the forecasting pipeline. The aim is not only to run one model, but to create a clear process that starts from raw data and ends with forecasted electricity demand values.

The pipeline follows these steps:

1. Load the `PJME_hourly.csv` dataset.
2. Convert the timestamp column into datetime format.
3. Sort the data chronologically.
4. Check for missing values, duplicate timestamps, and irregular time gaps.
5. Reindex the data to regular hourly frequency where necessary.
6. Create useful time-based, lag, and rolling features.
7. Split the data into training and testing parts using chronological order.
8. Train SARIMA and LSTM models.
9. Forecast future electricity demand for 720 hours.
10. Evaluate the results using error metrics and visual plots.
11. Compare model performance and identify strengths and limitations.

This pipeline keeps the work structured and reproducible. It also connects the KDD process with the practical implementation of the forecasting system [FPSS96a].

14.1.4. Model Choice: SARIMA and LSTM

Two forecasting methods are considered in this project: SARIMA and LSTM.

SARIMA is a classical statistical time series model. It is useful when the data has clear seasonal patterns. In this project, SARIMA helps model hourly electricity demand with repeated daily behaviour. Its advantage is interpretability, because the model structure is easier to explain compared to deep learning models.

LSTM is a deep learning model designed for sequential data. It is suitable for learning long-term dependencies in time series. Electricity demand may depend on previous

hours, previous days, and previous weeks. LSTM can learn these relationships from historical data and may perform better when the pattern is nonlinear or complex [TTL21; MSSJ22b].

Using both models allows comparison between a traditional statistical method and a machine learning/deep learning method. This makes the evaluation more meaningful because the project does not depend on only one forecasting approach.

14.2. Application

The application part of this project transforms the forecasting concept into a working system. The goal is to make the forecasting process usable, understandable, and repeatable. The application should allow electricity load data to be prepared, processed, modelled, evaluated, and used for generating future demand predictions.

The application is designed around the `PJME_hourly.csv` dataset and the 720-hour forecast horizon. It uses Python-based tools for data processing, modelling, visualization, and evaluation. The system is intended to support both technical understanding and practical use.

14.2.1. Application Architecture

The application follows a modular structure. Each part of the system has a specific role, which makes the project easier to maintain and explain.

- **Data Input Layer:** Loads the hourly electricity demand dataset from a CSV file.
- **Preprocessing Layer:** Converts timestamps, sorts the data, checks missing values, handles duplicate timestamps, and prepares the time index.
- **Feature Engineering Layer:** Creates hour, day, month, cyclical, lag, and rolling average features.
- **Model Layer:** Trains and applies SARIMA and LSTM models.
- **Evaluation Layer:** Calculates forecast error metrics and compares model performance.
- **Output Layer:** Produces predicted values, plots, residual diagnostics, and result summaries.

This modular structure is useful because each stage can be tested independently. For example, preprocessing can be checked before model training, and model evaluation can be updated without changing the data loading part.

14.2.2. Application Flow

The user or developer follows a clear workflow:

1. Open the project environment.
2. Load the `PJME_hourly.csv` dataset.
3. Run preprocessing and data checks.
4. Generate engineered features.
5. Select or run the forecasting model.
6. Train and test the model using historical data.

7. Forecast the next 720 hours.
8. View predicted electricity demand.
9. Compare forecast accuracy using evaluation metrics.
10. Review visual plots and residual behaviour.

This process keeps the system understandable and repeatable. It also reduces the chance of mistakes because every stage has a defined purpose.

14.2.3. Input and Output

The input of the application is a structured CSV file containing hourly electricity load data. At minimum, the dataset requires:

- **Datetime:** The timestamp of each hourly observation.
- **PJME_MW:** The electricity demand value in megawatts.

The output of the application includes:

- Forecasted electricity demand for the next 720 hours.
- Actual vs. predicted comparison on the test set.
- Error metrics such as MAE and RMSE.
- Visualization of historical demand and forecasted demand.
- Residual diagnostics for checking remaining autocorrelation.
- Optional CSV output for storing forecast results.

14.2.4. Tools Used in the Application

The application is based on commonly used Python tools for data science and forecasting.

Tool	Purpose in the Project
Python	Main programming language for data processing, modelling, and evaluation.
pandas	Loading CSV data, handling datetime values, indexing, filtering, and resampling.
NumPy	Numerical computation, array handling, scaling, and metric calculation.
statsmodels	Implementation of SARIMA/SARIMAX-style statistical forecasting models and residual diagnostics.
Deep learning framework	Building and training the LSTM sequence model.
matplotlib	Visualizing historical demand, predictions, training curves, residuals, and forecast results.
Git/GitHub	Version control for code, dataset updates, and reproducibility.

Table 14.1.: Tools Used in the Energy Demand Forecasting Application

14.2.5. Code Example: Data Loading and Feature Preparation

The following shortened code example shows the main data preparation logic. The timestamp is converted to a datetime index, the series is sorted, and time-based, lag, and rolling features are created. These features help the models use the temporal structure of the electricity demand series.

Listing 14.1: Data Loading and Feature Preparation for PJME Hourly Dataset

```
df['Datetime'] = pd.to_datetime(df['Datetime'])
df = df.set_index('Datetime').sort_index()

df['hour'] = df.index.hour
df['day_of_week'] = df.index.dayofweek
df['month'] = df.index.month
df['is_weekend'] = df['day_of_week'].isin([5, 6])

df['lag_1'] = df['PJME_MW'].shift(1)
df['lag_24'] = df['PJME_MW'].shift(24)
df['lag_168'] = df['PJME_MW'].shift(168)

df['rolling_mean_24'] = df['PJME_MW'].rolling(window=24).mean()
df['rolling_mean_168'] = df['PJME_MW'].rolling(window=168).mean()
```

14.2.6. Use in the Project

In this project, the application is used as a complete forecasting workflow. It supports the main goal of predicting future electricity demand from historical hourly load data. The application is useful because:

- It converts raw hourly data into model-ready time series data.
- It supports both statistical and deep learning forecasting methods.
- It allows comparison between SARIMA and LSTM.
- It produces measurable results using error metrics.
- It produces visual evidence such as forecast plots, decomposition plots, and residual plots.
- It supports reproducibility through stored code and dataset versions.

Overall, the application connects the theoretical idea of time series forecasting with a practical implementation for electricity demand prediction.

14.3. Results

The results section evaluates how well the forecasting system performs and what value it provides. The main result of the project is a 720-hour electricity demand forecast. The system uses historical PJME hourly load data to estimate future demand values. The forecast results are then compared with actual test data where available.

The evaluation is based on both numerical metrics and visual interpretation. This is important because a forecasting model should not only have low error values, but should also follow the shape, cycles, and peak behaviour of the actual demand curve.

14.3.1. Forecast Output

The forecast output consists of hourly predicted electricity demand values. Each forecasted row should contain:

- Forecast timestamp
- Predicted electricity demand in MW
- Model used for prediction
- Optional error value if actual demand is available

The forecast horizon is 720 hours, which represents the next 30 days. This makes the output useful for short-term electricity demand planning.

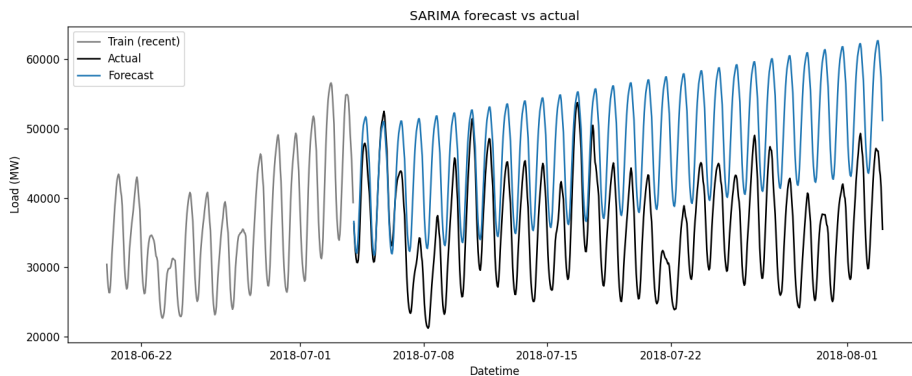


Figure 14.4.: SARIMA forecast compared with actual PJME electricity demand

The SARIMA forecast follows a clear daily seasonal structure. However, the visual result also shows that the forecast may become too regular and may overestimate or underestimate some actual demand periods. This is useful because it shows both the strength and limitation of SARIMA: it captures repeating seasonality, but it may struggle with sudden changes and more complex demand behaviour.

14.3.2. LSTM Forecast Result

The LSTM forecast is produced from a sequence-learning approach. The model uses historical windows of electricity demand to learn how recent demand patterns relate to future demand.

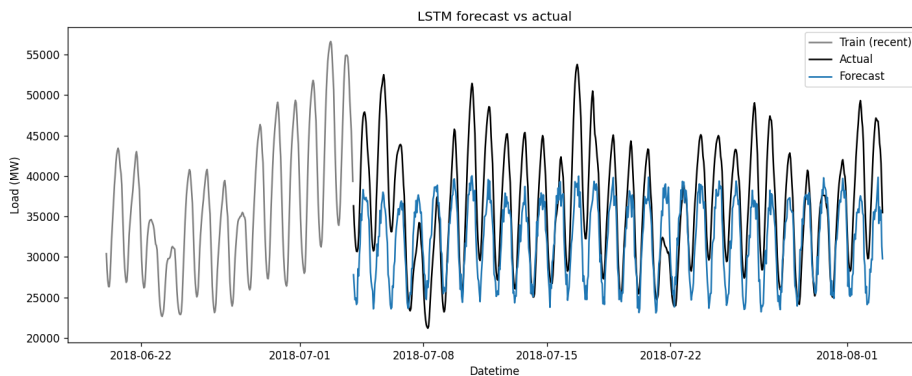


Figure 14.5.: LSTM forecast compared with actual PJME electricity demand

The LSTM forecast is more flexible than SARIMA because it can learn nonlinear relationships from the time series. The visual comparison helps show whether the model follows the daily cycles and whether it captures the level of actual demand. In this project, LSTM is important as an advanced method because electricity demand can depend on multiple previous periods, not only on a simple seasonal rule.

14.3.3. LSTM Training Behaviour

The LSTM training curve is used to check whether the model is learning properly. The training loss and validation loss show how the error changes over epochs.

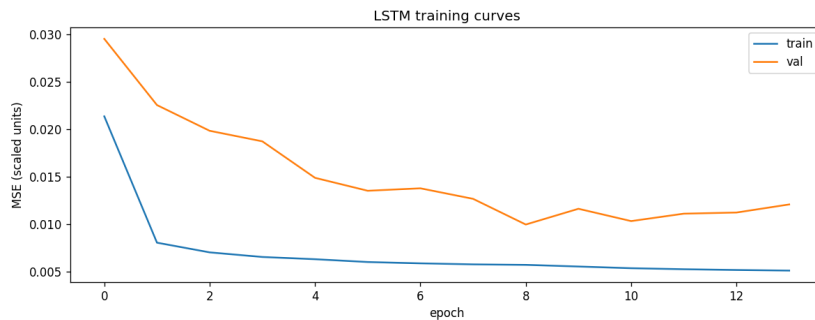


Figure 14.6.: LSTM training and validation loss curves

A decreasing training loss shows that the model is learning from the training data. The validation loss is important because it indicates whether the model can generalize to unseen data. If the validation loss stops improving or starts increasing while the training loss continues decreasing, this may indicate overfitting. Therefore, the training curve is useful for checking model reliability.

14.3.4. Model Performance Metrics

To evaluate the forecasting quality, common time series error metrics are used.

- **MAE – Mean Absolute Error:** Measures the average absolute difference between actual and predicted demand.
- **RMSE – Root Mean Squared Error:** Penalizes larger errors more strongly and is useful for identifying unstable forecasts.
- **MAPE – Mean Absolute Percentage Error:** Expresses the error as a percentage, making the result easier to interpret. If values close to zero are present, MAPE must be interpreted carefully.

These metrics help compare SARIMA and LSTM objectively. A lower value generally means better forecasting performance. However, the metrics should be interpreted together with visual plots, because peak demand errors may be hidden in average error values.

14.3.5. Code Example: Evaluation Metrics

The following code example shows the basic logic used for calculating MAE and RMSE.

Listing 14.2: MAE and RMSE Calculation for Forecast Evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
```

```
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))

print(f"MAE: {mae:.2f} MW")
print(f"RMSE: {rmse:.2f} MW")
```

14.3.6. SARIMA Result Interpretation

SARIMA is useful when the electricity demand pattern is stable and seasonal. It can capture repeated structures such as daily cycles. In this project, SARIMA acts as the classical statistical baseline.

The main strengths of SARIMA are:

- It is easier to interpret than deep learning models.
- It works well with structured seasonality.
- It can provide confidence intervals for forecasts.
- It is useful as a benchmark for more advanced models.

However, SARIMA also has limitations:

- It may struggle with nonlinear demand patterns.
- It may not react well to sudden changes in demand.
- Parameter selection can be time-consuming.
- Long hourly datasets can make fitting more computationally demanding.

14.3.7. LSTM Result Interpretation

LSTM is useful when the electricity demand pattern is complex and depends on longer historical sequences. It can learn relationships between previous hours, previous days, and previous weeks. This is useful for electricity demand, where current load may depend on recent consumption behaviour.

The main strengths of LSTM are:

- It can learn nonlinear sequential relationships.
- It can use long historical windows as input.
- It is flexible for complex electricity demand patterns.
- It can be extended later with more features such as weather or holidays.

However, LSTM also has limitations:

- It needs scaling and careful sequence preparation.
- It requires more training time than SARIMA.
- It is less interpretable than SARIMA.
- It may overfit if the model is too complex or the validation strategy is weak.

14.3.8. Residual Diagnostics

Residual analysis is used to check whether the model has left systematic patterns unexplained. A good forecasting model should leave residuals that behave more like random noise. If residual autocorrelation remains, it means the model did not fully capture the time-dependent structure.

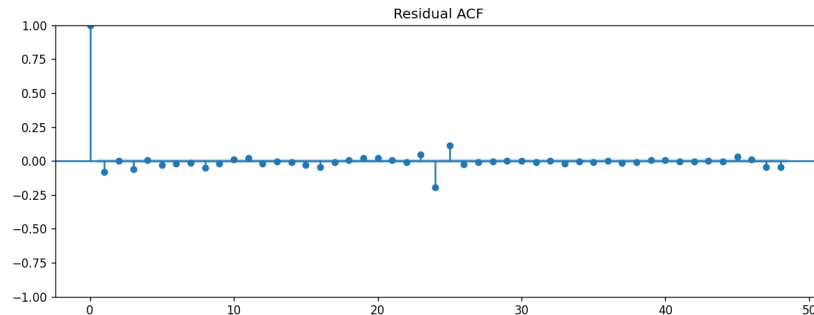


Figure 14.7.: Residual ACF plot after SARIMA forecasting

The residual ACF plot helps identify whether autocorrelation is still present after modelling. If most residual correlations remain inside the confidence band, the model has captured much of the time structure. If important spikes remain outside the band, further model tuning or additional features may be required.

14.3.9. Comparison of SARIMA and LSTM

The comparison between SARIMA and LSTM is one of the most important outcomes of the project.

Aspect	SARIMA	LSTM
Model Type	Classical statistical model	Deep learning sequence model
Main Strength	Good for structured seasonal patterns	Good for complex nonlinear patterns
Interpretability	Easier to explain	More difficult to explain
Data Requirement	Can work with moderate data	Usually benefits from larger datasets
Training Complexity	Lower to medium	Higher
Forecast Behaviour	Often produces smooth and regular seasonal forecasts	Can adapt more flexibly to learned patterns
Usefulness in Project	Baseline and interpretable forecasting	Advanced model for sequence learning

Table 14.2.: Comparison of SARIMA and LSTM for Electricity Demand Forecasting

14.3.10. Result Value

The forecasting result has practical value because it supports:

- Better short-term electricity demand planning.
- Identification of expected peak demand periods.

- Improved understanding of hourly and seasonal demand behaviour.
- Comparison between statistical and deep learning forecasting methods.
- A foundation for future deployment into a forecasting application.

Even if the project is academic, the result structure follows a real-world forecasting workflow. This makes it useful for learning, demonstration, and future improvement.

14.3.11. Summary of Results

Dimension	Result / Interpretation
Forecast Horizon	720 hours, representing the next 30 days of hourly demand.
Main Target	Electricity demand in megawatts using PJME_MW .
Time Series Behaviour	The data shows hourly, weekly, and monthly patterns, including clear peak demand periods.
Model Comparison	SARIMA provides interpretability; LSTM provides advanced sequence learning.
Evaluation	MAE and RMSE are used to measure forecast accuracy. Visual validation is also used.
Diagnostic Check	Residual ACF is used to check whether important autocorrelation remains after modelling.
Practical Value	Supports short-term demand planning and peak load understanding.
Limitations	Results depend on data quality, preprocessing, model tuning, and available external features.

15. Validation

15.1. General

Validation is an essential part of any forecasting project. In electricity demand forecasting, validation checks whether the model can predict unseen future demand reliably. Since future decisions may depend on the forecast, the model should not only fit historical data but also generalize to future time periods.

In this project, validation is especially important because the task is time-dependent. A random train/test split is not suitable for time series forecasting because it can mix past and future observations and cause data leakage. Therefore, the validation must preserve the chronological order of the dataset.

15.1.1. Chronological Train/Test Split

The dataset is split in chronological order. The earlier part of the time series is used for training, and the later part is used for testing.

- The training set contains historical electricity demand values.
- The test set represents unseen future values.
- The model is trained only on past data and evaluated on later data.

This approach is more realistic because real forecasting also uses past demand to predict future demand. It also prevents data leakage, which would make the model performance look better than it really is.

15.1.2. Validation Metrics

The model is validated using common forecasting error metrics.

- **MAE:** Shows the average absolute forecasting error in MW.
- **RMSE:** Gives more weight to large errors, which is useful for checking peak demand prediction errors.
- **MAPE:** Shows the error as a percentage, making it easier to communicate results if the target values are suitable.

Using multiple metrics gives a more balanced view of performance. For example, a model may have a good average error but still make large mistakes during peak demand periods. RMSE helps identify this issue.

15.1.3. Visual Validation

Numerical metrics are important, but visual validation is also necessary in time series forecasting. A graph of actual vs. predicted electricity demand helps check whether the model captures the overall shape of the time series.

Visual validation helps identify:

- Whether the model follows the demand trend.

- Whether daily and weekly cycles are captured.
- Whether peak periods are underestimated or overestimated.
- Whether the forecast is too smooth or too unstable.
- Whether the forecast shifts away from the real demand level over time.

This is important because a model can sometimes achieve acceptable metrics but still fail to represent important demand behaviour visually.

15.1.4. Seasonal Decomposition Validation

Seasonal decomposition is used to separate the observed time series into trend, seasonal, and residual components. This helps validate whether the dataset contains meaningful time-based structure before modelling.

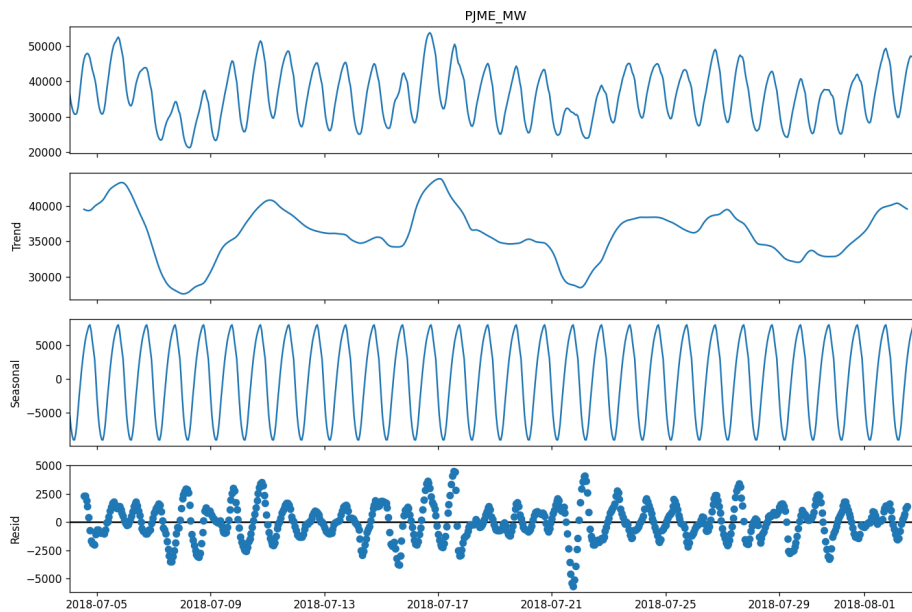


Figure 15.1.: Seasonal decomposition of PJME hourly electricity demand

The decomposition shows that electricity demand contains a clear seasonal component and changing trend behaviour. This supports the use of SARIMA and LSTM because both models are intended to learn from time-dependent patterns.

15.1.5. Model Comparison Validation

The project compares SARIMA and LSTM instead of relying on only one model. This improves validation because both models have different strengths.

- SARIMA is validated as a classical statistical baseline.
- LSTM is validated as a deep learning model for sequence learning.
- Their errors and forecast shapes are compared.
- Their ability to capture daily cycles and demand peaks is checked visually.

This comparison helps identify which model is more suitable for the given dataset and forecast horizon.

15.1.6. Practical Validation

The model should also be validated from a practical point of view. Electricity demand forecasting is useful only if the output can support planning decisions. Therefore, the forecast should be checked for:

- Reasonable future demand values.
- No unrealistic negative predictions.
- Consistent hourly frequency.
- Clear visualization and exportable results.
- Understandable model comparison.
- Acceptable behaviour during peak demand periods.

This ensures that the model is not only mathematically valid but also practically usable.

15.2. Validation – 3 Ideas

To further improve the validation of the forecasting system, three additional validation ideas are proposed. These ideas go beyond a simple train/test split and make the evaluation stronger.

15.2.1. Idea 1: Rolling Forecast Validation

Concept: Rolling forecast validation, also called backtesting, evaluates the model at multiple points in time. Instead of training and testing once, the model is repeatedly trained on a historical window and tested on the next future period.

How it works:

- Train the model on an initial historical period.
- Forecast the next time window.
- Move the training window forward.
- Repeat the process several times.
- Average the forecast errors across all validation windows.

Benefits:

- Gives a more reliable performance estimate.
- Tests model stability across different time periods.
- Simulates real-world forecasting where models are updated over time.

Why it matters: Electricity demand changes over time. A model that performs well in one period may not perform equally well in another period. Rolling validation helps detect this.

15.2.2. Idea 2: Benchmark Against Simple Baseline Models

Concept: The SARIMA and LSTM models should be compared against simple baseline methods. This checks whether the more advanced models actually add value. Possible baseline models include:

- **Naive forecast:** The next value is predicted as the most recent observed value.
- **Seasonal naive forecast:** The next value is predicted using the value from the same hour on the previous day or previous week.
- **Moving average forecast:** The next value is predicted using the average of recent values.

Benefits:

- Shows whether SARIMA and LSTM outperform simple methods.
- Makes the evaluation easier to explain.
- Prevents unnecessary model complexity.

Why it matters: If a simple seasonal naive model performs almost as well as LSTM, then the extra complexity of deep learning may not be justified for some use cases.

15.2.3. Idea 3: Peak Demand Validation

Concept: Electricity forecasting should not only be evaluated using average error. Peak demand periods are especially important because they affect grid stability and planning.

How it works:

- Identify high-demand periods in the test set.
- Compare actual and predicted values during these peak periods.
- Calculate separate error metrics for peak load hours.
- Check whether the model underestimates or overestimates peaks.

Benefits:

- Focuses on the most operationally important periods.
- Helps evaluate risk during high-load conditions.
- Improves trust in the model for energy planning.

Why it matters: A model with good average performance may still fail during peak demand. In electricity forecasting, peak errors can be more serious than normal-hour errors.

16. Conclusion

16.1. Conclusion

This project, *Energy Demand Forecasting using SARIMA and LSTM*, focused on predicting future electricity consumption using historical hourly load data. The main objective was to build a structured forecasting workflow that can process raw time series data, learn from historical patterns, and generate future demand predictions for the next 30 days.

The project demonstrates how time series analysis and machine learning can be applied to a real-world energy forecasting problem. Electricity demand is strongly time-dependent, with repeated hourly, daily, weekly, and seasonal patterns. By using the `PJME_hourly.csv` dataset, the project works with realistic electricity load data from the PJM Eastern region.

The project successfully covers the main stages of a forecasting system:

- Selection and understanding of the electricity demand dataset.
- Data preprocessing and timestamp handling.
- Feature engineering using time-based, lag, and rolling features.
- Forecasting with SARIMA and LSTM models.
- Evaluation using error metrics and visual comparison.
- Residual diagnostics and validation ideas for stronger future testing.

Overall, the work shows that SARIMA and LSTM can both be useful for electricity demand forecasting, but in different ways. SARIMA is easier to interpret and suitable for structured seasonal data, while LSTM can learn more complex sequential patterns from larger datasets.

16.1.1. Self-Critical Reflection

Although the project provides a strong forecasting foundation, there are some limitations that should be considered.

1. Limited External Variables

- The project mainly uses historical electricity demand values.
- Important external factors such as weather, holidays, temperature, and economic activity are not included.
- These external variables can strongly influence electricity consumption.

2. Model Complexity

- SARIMA requires careful parameter selection.
- LSTM requires scaling, sequence preparation, and tuning.
- LSTM can become difficult to interpret compared to statistical models.

3. Peak Demand Challenge

- Forecasting average demand may be easier than forecasting sudden peak demand.
- Large errors during peak hours can be more serious than small errors during normal hours.
- More focused validation is needed for high-demand periods.

4. Generalization

- The dataset represents the PJM Eastern region.
- The model may not directly generalize to other regions without retraining.
- Different regions may have different climate, population, industrial activity, and energy usage patterns.

5. Deployment Limitations

- The current project is mainly a forecasting prototype.
- A production system would require automated data updates, monitoring, logging, and model retraining.
- User-facing reporting and dashboard features are not fully developed in the current stage.

16.1.2. Unanswered Points

Some important questions remain open and can be explored in future work:

- How much does weather data improve the forecast accuracy?
- Does LSTM consistently outperform SARIMA across different seasons?
- How does the model behave during extreme demand periods?
- What is the best retraining frequency for the forecasting model?
- Can the model be adapted to other electricity regions?
- How can uncertainty intervals be added to the LSTM forecast?
- How can the system detect model decay over time?
- Which baseline model gives the strongest simple comparison?

These questions are important because real-world forecasting systems must remain reliable even when demand behaviour changes.

16.1.3. Next Steps

The following next steps can improve the project further:

1. Add Weather and Holiday Data

- Add temperature, humidity, holidays, and seasonal event data.
- Improve the model's understanding of demand drivers.

2. Implement Rolling Forecast Validation

- Use multiple train/test windows.

- Evaluate model stability across different time periods.

3. Improve LSTM Tuning

- Test different sequence lengths.
- Tune the number of layers, neurons, epochs, and batch size.
- Compare simple LSTM with GRU or hybrid models.

4. Add Baseline Models

- Compare SARIMA and LSTM with naive and seasonal naive forecasts.
- Clearly show whether advanced models provide additional value.

5. Build a User Interface

- Add a simple GUI or dashboard for uploading data and viewing forecasts.
- Display charts, metrics, and downloadable forecast results.

6. Add Monitoring

- Track model performance over time.
- Detect when forecast accuracy decreases.
- Trigger retraining when needed.

16.1.4. Final Reflection

This project shows how time series forecasting can be applied to an important energy-sector problem. Forecasting electricity demand is not only a technical task but also a practical planning tool. Reliable demand forecasts can support better decisions in grid management, resource planning, and energy efficiency.

The combination of SARIMA and LSTM gives a balanced approach. SARIMA provides a clear statistical baseline, while LSTM provides a more advanced method for learning complex demand patterns. Through data preparation, feature engineering, evaluation, and validation, the project creates a strong foundation for a future-ready electricity demand forecasting system.

With further improvements such as weather integration, rolling validation, baseline comparison, user interface development, and monitoring, this project can be extended from an academic prototype into a more complete decision-support tool for energy demand forecasting.

Part XI.

Appendix

17. Bill of Materials

17.1. Material List and Hardware Bill of Materials

Component list required for the Project are :

1. Laptop
2. Mouse
3. Keyboard

Materials	Description	Quantity	Link	Price
	A standard personal Laptop capable of running data analysis	1	Laptop	500 € to 1000 €
	Keyboard.	1	Keyboard	29.58 €
	Mouse.	1	Mouse	7.49 €

Table 17.1.: Hardware Bill of Materials

17.1.1. Laptop

The laptop is the central piece of hardware for the project. It should have sufficient processing power, memory, and storage to handle data analysis, run forecasting models, and support any required software.

- **Brand and Model:** *[Standard Laptop]*
- **Processor:** Intel Core i7-11800H (or equivalent)
- **RAM:** 16GB DDR4 (minimum), 32GB preferred for handling large datasets

- **Storage:** 512GB SSD (minimum), 1TB preferred for data storage
- **Graphics:** Integrated graphics or dedicated GPU like NVIDIA GTX 1650 (optional for enhanced performance in graphical analysis)
- **Display:** 15.6" Full HD (1920x1080) or higher resolution
- **Operating System:** Windows 10/11 or macOS (based on preference)
- **Ports:** USB 3.0, USB-C, HDMI, SD Card Reader, Ethernet port (optional)
- **Battery Life:** Minimum 8 hours for portability
- **Weight:** Lightweight for mobility (preferably under 4 lbs)

17.1.2. Mouse

A reliable mouse enhances productivity, especially during long hours of data analysis and model building.

- **Brand and Model:** *[HP]*
- **Type:** Wireless (for convenience)
- **Sensor:** High-precision sensor (DPI adjustable)
- **Battery Life:** Rechargeable with long battery life (up to 70 days on a full charge)
- **Ergonomics:** Comfortable design for prolonged use
- **Connectivity:** USB receiver or Bluetooth

17.1.3. Keyboard

An external keyboard can significantly improve typing speed and comfort, which is beneficial during data entry and coding.

- **Brand and Model:** *[Keychron K2 (Wireless Mechanical Keyboard, HP)]*
- **Type:** Wireless (for convenience)
- **Switch Type:** Mechanical (based on user preference - tactile, linear, or clicky)
- **Layout:** Full-size or Tenkeyless (based on user preference)
- **Battery Life:** Rechargeable with long battery life (up to several weeks)
- **Connectivity:** USB-C or Bluetooth
- **Ergonomics:** Ergonomic design with adjustable tilt

17.1.4. Additional Recommendations

- **Docking Station:** To expand connectivity options and create a more versatile workspace
 1. **Brand and Model:** *[Dell WD19TB Thunderbolt Dock]*
- **External Monitor:** For dual-screen setup, enhancing productivity
 1. **Brand and Model:** *[LG UltraFine 4K Display]*
 2. **Size:** 27" or larger
 3. **Resolution:** 4K UHD (3840x2160) for clear, detailed visuals
- **External Storage:** For backup and additional storage
 1. **Brand and Model:** *[Samsung T7 Portable SSD]*
 2. **Capacity:** 1TB or higher

17.2. Software Requirement and Bill Of Materials

Materials	Description	Quantity	Installation	Price
 Visual Studio Code	A lightweight, open-source editor by Microsoft, widely used for coding and data analysis with support for Python, R, extensions, and Git integration.	1	VSCode	Free
 python™	A popular and versatile programming language commonly used for data analysis, machine learning, automation, and software development.	1	Python	Free

Table 17.2.: Software Bill of Materials

17.2.1. Visual Studio Code (VS Code)

- **Visual Studio Code:** Visual Studio Code (VS Code) is a powerful, lightweight, and extensible open-source code editor developed by Microsoft. It offers robust support for development across multiple programming languages and platforms, with features tailored for productivity and collaboration. **[VSCode:2025]**
 1. **User Interface:** VS Code provides an intuitive and highly customizable user interface. It supports themes, icon packs, and flexible layout arrangements, allowing users to tailor their workspace to their workflow preferences.
 2. **Extensions and Marketplace:** One of VS Code's standout features is its vast ecosystem of extensions available through the Visual Studio Code Marketplace. These include language packs, linters, formatters, debuggers, and tools for Docker, Git, Jupyter, Python, and more.

3. **Language Support:** VS Code supports a wide range of programming languages out of the box, including Python, JavaScript, TypeScript, Java, C++, and more. Additional language support can be installed via extensions, making it a versatile tool for cross-language development.
4. **Integrated Terminal:** The built-in terminal in VS Code allows developers to execute shell commands directly within the editor, streamlining workflows by reducing the need to switch between the editor and external terminals.
5. **IntelliSense:** VS Code offers intelligent code completion, parameter info, quick info, and member lists through IntelliSense. This boosts productivity and minimizes coding errors by providing real-time suggestions as the user types.
6. **Version Control Integration:** VS Code comes with integrated Git support. It allows users to manage source control repositories, perform commits, resolve merge conflicts, and view change history directly within the editor.
7. **Remote Development:** With extensions like Remote - SSH, Remote - Containers, and WSL, VS Code supports seamless development on remote machines or containers, enabling scalable and flexible development environments.
8. **Jupyter Integration:** While VS Code is not a notebook-native interface, it provides strong support for Jupyter Notebooks through its Python extension. Users can open, edit, and run notebooks inside VS Code, combining the best of script-based and notebook-based development. [VSCode:2025]

VS Code is available for installation on Windows, macOS, and Linux, and is widely adopted by developers for its performance, flexibility, and extensive community support.

18. requirements.txt

18.1. Purpose of requirements.txt

The `requirements.txt` file is an essential component in any Python project. It defines all the third-party libraries, also called dependencies, that must be installed for the project to run successfully.

In this project, it ensures that the **Energy Demand Forecaster** system can be reliably executed across different machines with consistent environments. This helps avoid version conflicts, supports teamwork, and makes the project easier to reproduce or deploy in the future.

This project uses two dependency files:

- `requirements.txt`: contains the full development environment, including analysis, plotting, notebooks, modelling, and application support.
- `requirements-runtime.txt`: contains the smaller set of packages required to run the application or forecasting workflow at runtime.

18.2. Dependencies Used

Below is the list of dependencies required by the full project environment. The version numbers are written with minimum version requirements using `>=`. This means that the listed version or any newer compatible version can be installed.

Listing 18.1: requirements.txt

```
pandas>=2.0          # Data loading, cleaning, datetime indexing,
    preprocessing
numpy>=1.24           # Numerical computations and array operations
matplotlib>=3.7       # Plotting graphs, forecasts, and diagnostics
seaborn>=0.12         # Additional statistical visualisation support
statsmodels>=0.14     # SARIMA model and residual diagnostics
scikit-learn>=1.3     # Evaluation metrics and preprocessing utilities
jupyter>=1.0          # Interactive notebooks for experiments and EDA
pyarrow>=14.0         # Efficient data file support, including
    parquet format
torch>=2.5            # LSTM model implementation and training
streamlit>=1.30       # Web application or dashboard support
```

The smaller runtime dependency file contains only the packages required for running the forecasting application or model workflow without the full notebook and plotting environment.

Listing 18.2: requirements-runtime.txt

```
streamlit>=1.30       # Application interface support
pandas>=2.0           # Data loading and time series preparation
numpy>=1.24           # Numerical operations and array handling
scikit-learn>=1.3     # Scaling and evaluation utilities
statsmodels>=0.14     # SARIMA forecasting
torch>=2.5            # LSTM forecasting
```

18.3. Explanation of Packages

- **pandas** – Handles data import, cleaning, datetime conversion, time-based indexing, duplicate timestamp handling, and train/test preparation for the PJME hourly electricity demand dataset.
- **numpy** – Supports numerical operations, array handling, cyclical time feature encoding, scaling calculations, and RMSE-related computations.
- **matplotlib** – Used for generating plots such as historical load trends, forecast comparison plots, and residual diagnostic figures.
- **seaborn** – Provides additional statistical visualisation support during exploratory data analysis. It is included in the full development environment.
- **statsmodels** – Provides the SARIMA implementation through **SARIMAX** and supports residual diagnostic analysis such as Ljung-Box testing and autocorrelation plots.
- **scikit-learn** – Provides machine learning utilities used for model evaluation and preprocessing, including error metrics such as MAE and MSE.
- **jupyter** – Supports notebook-based development, exploratory data analysis, preprocessing checks, and step-by-step model experiments.
- **pyarrow** – Supports efficient data file handling, especially when working with columnar file formats such as parquet. It is part of the full development environment.
- **torch** – Provides the PyTorch framework used to build, train, save, and load the LSTM forecasting model.
- **streamlit** – Supports a lightweight application or dashboard interface for running and presenting the forecasting workflow.

18.4. How to Use requirements.txt

To install all full development dependencies, use the command:

Listing 18.3: Install full project dependencies

```
pip install -r requirements.txt
```

To install only the smaller runtime environment, use:

Listing 18.4: Install runtime dependencies

```
pip install -r requirements-runtime.txt
```

To generate a dependency file from the current Python environment, use:

Listing 18.5: Generate requirements.txt from current environment

```
pip freeze > requirements.txt
```

18.5. Conclusion

The `requirements.txt` and `requirements-runtime.txt` files improve the reproducibility of the Energy Demand Forecaster project. The full `requirements.txt` file supports development, notebooks, visualisation, and modelling, while the smaller `requirements-runtime.txt` file focuses on the packages needed to run the forecasting application or runtime workflow. Together, these files make it easier for different contributors to install the same environment and execute the project consistently.

19. Personal Findings and Learnings – Karrar Al-Ameeri

The content of this chapter reflects the personal findings, observations, and learnings of the author, Karrar Al-Ameeri. The statements and interpretations presented herein represent the author's own perspective and are intended for reflective and contextual purposes only. This chapter does not form part of the formal system specification, evaluation, or validation presented in the main body of this report.

19.1. Overview of My Contribution

Taking on the role of team coordinator and primary technical contributor in this project was both challenging and deeply rewarding. My responsibilities spanned the full project lifecycle: from structuring the repository and distributing tasks among team members, to implementing core machine learning components, building the complete end-user application, and producing the final documentation artefacts. This chapter reflects on what I built, what surprised me, and what I would approach differently.

19.2. Team Leadership and Project Coordination

This was my first experience as the technical lead on a collaborative academic software project, and it taught me as much about communication and organisation as it did about machine learning.

19.2.1. Structuring the Team and the Work

From the beginning, I set up a folder structure so that every team member could work independently without interfering with each other's files. I divided the project into four logical streams aligned with the KDD process: data and preprocessing, model development (SARIMA), model development (LSTM), and deployment. Each stream was assigned to a team member, but I remained the integration point because the four streams feed into each other and produce shared artefacts such as the trained model checkpoint and the evaluation figures.

One lesson I learned quickly is that in a group project, the bottlenecks are rarely technical. They are mostly about communication: making sure everyone understands the task the same way, that dependencies between tasks are clear, and that the scope does not quietly expand.

19.3. Building the Streamlit Application

The Streamlit forecasting application (`app/streamlit_app.py`) was one of the most technically interesting and personally satisfying parts of the project. Before this project, I had seen Streamlit mentioned in tutorials but had never built a complete, production-ready application with it.

19.3.1. Design Goals

The application had to serve a non-technical user someone who works in grid operations and needs a forecast, but who has no Python background and no interest in model internals. This constraint shaped every design decision:

- The interface must require zero configuration. The bundled PJME sample dataset loads automatically with one click of button, so the user can run a forecast immediately without uploading anything.
- All interaction must be through the browser. No terminal, no command line, no error messages exposed to the user.
- Feedback must be instant. The LSTM-q70 model produces its 720-step forecast in under five seconds on a standard consumer laptop, so the user never has to wait.
- Outputs must be interpretable by a non-specialist. I added a plain-language **AI Insights** panel that summarises the forecast quality in one paragraph: what the model predicted, whether the MAE is acceptable, and what the pattern of errors looks like. This panel was not part of the original scope – I added it because I felt a number like “MAE = 3 307 MW” means nothing to a user who has never seen a MAE before.

19.4. Cross-Platform Portable Deployment

Building a portable, offline, double-click-to-launch bundle for both macOS (Apple Silicon, arm64) and Windows (x86_64) was the most operationally complex part of the entire project. I had not done anything like this before, and it took considerably more effort than I had estimated.

19.4.1. The Problem

The target user cannot be expected to install Python, manage virtual environments, or run terminal commands. Electricity grid environments often also restrict internet access, which eliminates cloud-hosted solutions and `pip install` at runtime. The application therefore had to be fully self-contained: a folder that the user unzips, double-clicks one file, and the browser opens with the Streamlit GUI running.

The core difficulty is that our dependency stack PyTorch, Streamlit, pandas, scikit-learn, statsmodels, matplotlib totals approximately 2 GB of compiled packages. Furthermore, PyTorch uses different hardware acceleration backends on different platforms: Apple Metal (MPS) on macOS and CUDA or CPU on Windows. The bundle had to handle this correctly without requiring any configuration from the user.

19.4.2. The Solution

I built a `build_portable.sh` shell script that:

1. Downloads a standalone CPython 3.12.13 interpreter for the target platform. This interpreter is entirely self-contained and does not interact with any Python installation already on the user’s machine.
2. Installs all required packages into an isolated `packages/` directory using the standalone interpreter’s `pip`.
3. Copies the application source files (`src/`, `app/`), the trained LSTM-q70 checkpoint (`models/lstm_pjme_q70.pt`), the baked scaler (`models/lstm_pjme_scaler.json`), and the sample dataset into the bundle.

4. Writes a platform-specific launcher: `run.command` on macOS (a shell script that is executable by double-click in Finder) and `run.bat` on Windows (a batch file that opens in the default browser).

The result is a folder that the user unzips and double-clicks. Nothing is installed on their machine. Everything the application needs is inside the folder.

19.5. The Quantile Loss Innovation

The decision to replace the standard Mean Squared Error loss with a quantile (pinball) loss at $\tau = 0.70$ was the main technical innovation in the project and the one I am most proud of. It emerged from a genuine problem observed in the initial LSTM results rather than from a textbook exercise.

After training the first LSTM with standard MSE loss, I plotted the forecast against the actual values and noticed that the model systematically under-predicted the demand peaks at 14:00–17:00. The MAE of 4 996 MW was decent overall, but the errors at peak hours exceeded 8 000 MW. For a grid operator, peak-hour under-prediction is specifically the most costly type of error because it forces emergency purchases from the spot market or activation of expensive reserve generation.

I had read about quantile regression in the context of financial forecasting and remembered that asymmetric loss functions can bias the model toward higher or lower predictions. I implemented the pinball loss:

$$\mathcal{L}_\tau(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N \max(\tau(y_i - \hat{y}_i), (\tau - 1)(y_i - \hat{y}_i))$$

Setting $\tau = 0.70$ means under-predictions are penalised 2.33 times more heavily than over-predictions ($0.70/0.30 \approx 2.33$). After retraining with this loss, the MAE dropped from 4 996 MW to 3 307 MW – a 34 % improvement – and the visual forecast showed noticeably better peak capture.

What I learned from this experience is that the choice of loss function is a *domain* decision, not just a technical one. The right loss function encodes the real-world cost structure of the problem. MSE assumes over- and under-prediction are equally bad. In electricity forecasting, they are not. Aligning the loss function with the actual cost structure is one of the highest-leverage improvements available.

19.6. Reflection

Looking back, the parts of this project I am most proud of are not the ones that produced the highest numbers. The quantile loss improvement was satisfying, but it was technically straightforward once the idea was clear. What I found genuinely difficult – and genuinely educational – was the deployment work: understanding that a model that produces good forecasts is only valuable if it can be run by the people who need it, and that making it runnable on any machine, without any setup, is a hard engineering problem that deserves as much attention as the modelling itself.

The team leadership experience reinforced something I had read but not yet felt: the hardest part of a collaborative technical project is not the technology. It is keeping a group of people aligned on a shared goal when everyone has different backgrounds, different work rhythms, and different levels of familiarity with the tools. The habits I developed here – clear task assignments, explicit interfaces between components, regular check-ins, documented conventions – are habits I intend to carry into professional software development.

20. Personal Findings and Learnings – Aravind Lakshmi Narayanan

The content of this chapter reflects the personal findings, observations, and learnings of the author, Aravind Lakshmi Narayanan. The statements and interpretations presented herein represent the author's own perspective and are intended for reflective and contextual purposes only. This chapter does not form part of the formal system specification, evaluation, or validation presented in the main body of this report.

20.1. Overview of My Contribution

My primary contribution to this project centred on domain problem analysis, dataset selection and validation, data quality assessment, and the academic documentation of the project. I was responsible for the KDD chapters covering the application context, data acquisition, data quantity, data quality, data relevance, and outlier analysis, as well as the comparative literature review that positioned our SARIMA and LSTM models within the existing energy forecasting research. This chapter reflects on the decisions I made, what I learned, and what I would approach differently.

20.2. Dataset Selection

One of the first and most consequential decisions in the project was the choice of dataset. The Kaggle repository for PJM hourly energy consumption contains multiple regional CSV files, and selecting the right one required careful reasoning rather than simply choosing the largest or most convenient option.

I selected `PJME_hourly.csv` for four specific reasons. First, it provides the longest clean continuous span, containing approximately 145,366 hourly observations from January 2002 to August 2018, covering 16 years. This volume is essential for SARIMA, which requires multiple complete seasonal cycles to estimate its parameters reliably, and for LSTM, which benefits from large sequential training sets to learn long-range temporal dependencies. Second, PJME is one of the most widely benchmarked PJM subregions in the academic literature, allowing our MAE and RMSE results to be compared directly with published baselines. Third, the eastern PJM grid exhibits strong and stable daily, weekly, and annual seasonal cycles driven by summer cooling and winter heating demand, which closely match the assumptions underlying SARIMA. Fourth, the dataset contains minimal structural breaks and well-documented daylight saving time (DST) behaviour, making it a clean and transparent benchmark.

Alternative datasets such as PJMW, AEP, COMED, and the combined `est_hourly.parquet` file were rejected for principled reasons, including shorter historical coverage, lower benchmark comparability, or mixed regional characteristics that would obscure the seasonal patterns on which both models depend.

20.3. Data Validation and Quality Assessment

After selecting the dataset, I conducted a systematic quality assessment by analysing the actual CSV file in Python rather than relying solely on external documentation

or prior reports. This process revealed several findings that directly influenced the project’s preprocessing decisions.

The dataset contains exactly 145,366 rows and two columns, `Datetime` and `PJME_MW`, with zero missing values and zero fully duplicate rows. However, inspection of the timestamp sequence revealed two DST-related structural artefacts: four duplicate timestamps caused by the autumn clock-back transition and 30 two-hour gaps caused by the spring clock-forward transition. These are well-known characteristics of hourly energy datasets rather than data collection errors, but they must be handled explicitly to ensure the regular hourly index required by both SARIMA and LSTM.

An interquartile range (IQR)-based outlier analysis identified 3,455 observations (2.38%) outside the bounds of 15,457.5 MW and 47,765.5 MW. After examining their temporal distribution, I concluded that these observations represent genuine physical events, such as extreme summer cooling peaks and winter heating surges, rather than measurement errors. The decision to retain them was deliberate, since removing real demand extremes would deprive both models of the very events that are most important to forecast accurately from an operational perspective.

This process taught me that data quality assessment is not merely a procedural step. By executing code and inspecting the actual dataset, I uncovered structural issues, particularly the DST timestamp duplicates, that would not have been evident from summary statistics alone.

20.4. Literature Review and Citation Integrity

A significant part of my contribution involved identifying and verifying the academic references used throughout the domain problem chapters. This task proved to be more demanding than I initially expected.

My approach was to locate papers that directly supported the specific claims being made, rather than attaching any broadly relevant reference to a statement. This led me to reject several initially promising citations after carefully reviewing their content. For example, one paper on ARIMA and artificial neural networks applied to Turkish smart meter data was initially considered to support the SARIMA baseline discussion, but it used neither SARIMA nor PJM data. Likewise, a study on generalized additive models (GAMs) for European load forecasting was excluded because, despite its quality, it was not directly relevant to the SARIMA versus LSTM comparison that forms the core of our project.

The references ultimately selected, including Hong and Fan (2016), Park and Yang (2024), Gasparin, Lukovic, and Alippi (2022), Dubey et al. (2021), and Bilgili and Pinar (2023), were chosen because each directly supports the statements to which it is attached. I verified this by reading the full papers rather than relying only on abstracts or secondary descriptions.

The most important lesson I learned is that citation integrity is not a formality. A reference that appears plausible but does not actually support the claim being made constitutes a misrepresentation, and such inaccuracies are likely to be detected during careful academic review.

20.5. Reflection

Looking back, the most challenging aspect of this project was not the technical analysis itself, but the discipline of remaining faithful to what the data and the literature genuinely demonstrate rather than what might be convenient to claim.

The dataset investigation reinforced the importance of assumption-free analysis: working directly with the raw data and verifying every conclusion through code leads to more reliable findings than relying solely on documentation or prior summaries. The

literature review demonstrated that the value of a citation lies not in the prestige of the journal but in the precision with which it supports a specific statement.

I entered this project with more than two years of professional experience as a Data Engineer, and that background strongly influenced my approach to data validation. I treated the dataset as a production data source that had to be carefully interrogated before it could be trusted. The academic context added an additional level of discipline: every claim must be traceable to a verifiable source, and every source must accurately support the associated claim.

The habits developed through this project, namely rigorous validation, citation integrity, and the willingness to reject convenient conclusions in favour of correct ones, are principles that I intend to carry forward into both my academic and professional work.

21. Personal Findings and Learnings – Abhilash Nayak and Pranoti Patil

The content of this chapter reflects the personal findings, observations, and learnings of the authors, Abhilash Nayak and Pranoti Patil. The statements and interpretations presented herein represent the authors' own perspectives and are intended for reflective and contextual purposes only. This chapter does not form part of the formal system specification, evaluation, or validation presented in the main body of this report.

21.1. Personal Findings and Learnings – Abhilash Nayak

21.1.1. Overview of My Contribution

My contribution to this project focused mainly on the KDD and database-related documentation, the SARIMA-related explanation, the evaluation, validation, and conclusion chapter, and the final project poster. I worked on explaining how the `PJME_hourly.csv` dataset is selected, structured, processed, and prepared for forecasting. This included sections such as data selection, data origin, features, data types, data quality, data quantity, fairness and bias, data processing, one database, properties, outliers, anomalies, augmentation, and data version control.

I also contributed to the literature review together with Pranoti. This part helped us understand the academic background of electricity demand forecasting and why SARIMA and LSTM were suitable model choices for our project. In addition, I worked on adapting the report content into a poster format, where the main challenge was to reduce detailed technical writing into a clear visual explanation.

21.2. KDD, Database, and Data Understanding

One of the most important parts of my work was understanding the dataset from a KDD perspective. At first, the dataset looked simple because it mainly contains two columns: `Datetime` and `PJME_MW`. However, I learned that even a simple-looking time series dataset requires careful explanation and validation before it can be used for modelling.

While writing the database chapter, I had to explain why the dataset is suitable for time series forecasting. The PJME dataset contains hourly electricity demand values over many years, which makes it useful for identifying hourly, daily, weekly, and seasonal patterns. This helped me understand that the value of a dataset is not only based on the number of columns, but also on whether it contains the right time-based structure for the project goal.

I also learned the importance of clearly explaining data properties. For example, timestamp handling, chronological order, missing values, duplicate timestamps, outliers, and seasonal behaviour are all important in electricity demand forecasting. If these parts are not explained properly, the modelling result may look correct but the process behind it would be weak.

21.3. SARIMA and Forecast Evaluation

Another major learning for me was understanding SARIMA as a classical time series forecasting model. SARIMA is useful because it can model repeated seasonal patterns, which are clearly present in electricity demand data. Compared with deep learning models, SARIMA is easier to explain because its purpose is linked directly to trend, seasonality, and autocorrelation.

However, I also learned that SARIMA has limitations. It can become too regular when the data changes suddenly, and it may not capture nonlinear behaviour as well as an LSTM model. This made the comparison between SARIMA and LSTM more meaningful. Instead of treating one model as automatically better, I learned to explain what each model contributes to the project.

In the evaluation, validation, and conclusion chapter, I focused on explaining how the results should be interpreted. I learned that metrics such as MAE and RMSE are important, but they are not enough on their own. A model may have acceptable average error but still perform poorly during peak demand periods. Therefore, visual comparison between actual and predicted demand is also necessary.

21.4. Literature Review and Academic Direction

The literature review helped me understand the direction of the project more clearly. Before reading the papers, it was easy to think that SARIMA and LSTM were selected only because they are common forecasting models. After reviewing the literature, I understood that the choice is more logical: SARIMA represents the classical statistical forecasting approach, while LSTM represents a deep learning approach for sequential data.

This also helped me understand the advantages and disadvantages of other models. Simpler models such as Linear Regression may be easier to explain but are weak for seasonal and sequential patterns. Random Forest can handle nonlinear data but does not naturally understand time order. SVM can be useful in some forecasting tasks but may be less suitable for large hourly time series. CNN-based approaches can capture local patterns, but LSTM is more suitable for longer sequence dependencies.

The main lesson I learned is that literature review is not just for adding citations. It helps justify why the project moves in a certain direction and why some alternatives are not selected.

21.5. Poster and Communication

Working on the poster taught me that technical communication changes depending on the format. In the report, detailed explanations are useful because the reader has time to follow the full argument. In a poster, the content must be much shorter and more visual. A person should be able to stand a few metres away and still understand the main idea.

For this reason, I worked on reducing the report content into high-level sections such as problem description, challenges, forecasting pipeline, model comparison, solution, and results. I also learned that diagrams and plots are more effective than long paragraphs in a poster. The TikZ pipeline helped explain the project workflow more clearly than text alone.

21.6. Reflection

This project helped me understand the complete flow of a forecasting project, from data understanding to modelling, evaluation, validation, and presentation. My biggest

learning was that a forecasting project is not only about running models. The dataset must be understood, the assumptions must be explained, and the results must be communicated clearly.

If I continued improving this project, I would focus more on baseline model comparison, rolling forecast validation, and weather or holiday features. These additions would make the evaluation stronger and help the model reflect real electricity demand behaviour more accurately.

21.7. Personal Findings and Learnings – Pranoti Patil

21.7.1. Overview of My Contribution

My contribution focused mainly on the application development data sections, package documentation, requirements documentation, bill of materials, software bill of materials, literature review support, and poster preparation. I worked on explaining the structure and practical use of the project data, including data format, data size, data anomalies, and data origin.

I also contributed to documenting the tools and packages used in the project. This included describing why Python libraries and project dependencies were needed and how they support the forecasting workflow. Together with Abhilash, I also worked on the literature review and the final poster.

21.8. Application Development Data Documentation

A major part of my work was connected to explaining the data from an application development point of view. This means not only describing what the dataset contains, but also how it can be used inside a forecasting system.

I learned that proper data documentation is important because users and developers need to understand what kind of input the system expects. In this project, the input is a CSV file containing hourly electricity load values. The timestamp column must be readable as a datetime value, and the demand column must be numeric. If these basic conditions are not clear, the forecasting workflow can fail during preprocessing or modelling.

The data anomaly section also helped me understand that unusual values are not always errors. In electricity demand forecasting, high demand values can represent real peak-load events. These are important because the purpose of the model is not only to predict normal demand, but also to understand high-demand periods. This taught me that anomalies must be interpreted carefully before deciding whether to remove or keep them.

21.9. Package Documentation and Reproducibility

Another important part of my contribution was documenting the packages, requirements, bill of materials, and software bill of materials. This helped me understand the importance of reproducibility in a software and data science project.

A forecasting project may run correctly on one computer but fail on another system if the required libraries, versions, or dependencies are not clearly documented. By working on the package and requirements sections, I learned that documentation is part of the technical system, not separate from it. It helps other users install, run, and understand the project.

This work also made me more aware of the different roles of the tools used in the project. For example, **pandas** supports data loading and timestamp handling, **NumPy** supports numerical operations, **statsmodels** supports statistical time series modelling, deep learning libraries support LSTM modelling, and plotting libraries support evaluation and visual interpretation.

21.10. Literature Review and Model Understanding

The literature review was useful because it helped me understand why different models are used in energy demand forecasting. SARIMA is suitable when the data has clear seasonal structure, while LSTM is useful when the demand pattern is more complex and depends on previous time steps.

Through the literature review, I also learned why not every model is equally suitable for this project. Linear Regression is simple and interpretable, but it cannot naturally capture complex seasonality. Random Forest can model nonlinear relationships, but it does not directly understand time order. SVM can be useful in smaller forecasting tasks, but large hourly electricity datasets can make it harder to apply efficiently. CNN models can identify local patterns, but LSTM is more directly suitable for sequence learning.

This made me understand that model selection should not be based only on popularity. It should be connected to the data type, forecast horizon, project goal, and practical limitations.

21.11. Poster and Visual Presentation

Working on the poster helped me understand how to present technical work in a simpler and more visual way. The full report contains detailed explanation, but the poster must show only the most important points. This required reducing long sections into clear blocks such as problem description, challenges, pipeline, forecast output, model comparison, and results.

I learned that visual structure is very important in a poster. The forecasting pipeline diagram, forecast plots, and model comparison table make the topic easier to understand than long paragraphs. This was an important learning experience because in real projects, technical results often need to be communicated to people who may not read the full report.

21.12. Reflection

This project showed me that data science work includes much more than model training. Data explanation, anomaly understanding, dependency documentation, literature review, and visual communication are also important parts of the final result.

The biggest learning for me was that reproducibility and clarity are essential. A project should not only work once; it should be understandable and runnable by others. The package documentation and poster work helped me see the project from the perspective of a future user or reviewer.

If I continued improving this project, I would focus on making the documentation even more user-friendly and connecting the package documentation more directly with the final forecasting workflow.

21.13. Shared Collaboration

Although Abhilash and Pranoti had separate responsibilities, several parts of the project were completed collaboratively. We worked together on the green-colour assigned tasks, especially the literature review and poster-related work. The literature review helped us understand the strengths, weaknesses, and limitations of SARIMA, LSTM, and alternative forecasting models. The poster required us to convert detailed report content into a concise visual explanation.

This collaboration taught us that academic project work requires both individual responsibility and shared alignment. Each section must be written separately, but the final report and poster must still feel like one connected project.

Literature

- [Arm01] J. S. Armstrong, ed. *Principles of Forecasting: A Handbook for Researchers and Practitioners*. Norwell, MA: Kluwer Academic Publishers, 2001. DOI: [10.1007/978-0-306-47630-3](https://doi.org/10.1007/978-0-306-47630-3). URL: <https://doi.org/10.1007/978-0-306-47630-3>.
- [BEA21] A. Barrak, E. E. Eghan, and B. Adams. “On the Co-evolution of ML Pipelines and Source Code - Empirical Study of DVC Projects”. In: *Proceedings of the 28th IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. Hawaii, USA, 2021. DOI: [10.1109/SANER50967.2021.9425888](https://doi.org/10.1109/SANER50967.2021.9425888).
- [BG+21] A. Blázquez-García et al. “A review on outlier/anomaly detection in time series data”. In: *ACM Computing Surveys* 54.3 (2021), p. 56. DOI: [10.1145/3444690](https://doi.org/10.1145/3444690). URL: <https://doi.org/10.1145/3444690>.
- [BP23] M. Bilgili and E. Pinar. “Gross electricity consumption forecasting using LSTM and SARIMA approaches: A case study of Türkiye”. In: *Energy* 284 (2023), p. 128575. DOI: [10.1016/j.energy.2023.128575](https://doi.org/10.1016/j.energy.2023.128575). URL: <https://doi.org/10.1016/j.energy.2023.128575>.
- [Bai21] T. Baier. “Evaluation Metrics for Forecasting Models in Machine Learning”. In: *Journal of Forecasting Methods* 40.3 (2021), pp. 245–260.
- [Beb+19] T. Bebensee et al. “Privacy and Data Protection in Machine Learning Systems”. In: *Journal of Data Protection & Privacy* 3.4 (2019), pp. 45–60.
- [C 0] B. P. C. “How to Detect Outliers in Machine Learning – 4 Methods for Outlier Detection”. In: *freeCodeCamp.org* (6-07-2022). URL: <https://www.freecodecamp.org/news/how-to-detect-outliers-in-machine-learning/>.
- [CM19] C. Coronel and S. Morris. *Database Systems: Design, Implementation, and Management*. 13th. Cengage Learning, 2019.
- [CS14] S. Chacon and B. Straub. *Pro Git*. Springer Nature, 2014. ISBN: 9781484200773.
- [Cha13] C. Chase. *Monitoring Machine Learning Systems in Production*. O’Reilly Media, 2013.
- [Dub+21] A. Dubey et al. “Study and analysis of SARIMA and LSTM in forecasting time series data”. In: *Sustainable Energy Technologies and Assessments* 47 (2021), p. 101474. DOI: [10.1016/j.seta.2021.101474](https://doi.org/10.1016/j.seta.2021.101474). URL: <https://doi.org/10.1016/j.seta.2021.101474>.
- [FPSS96a] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. “From Data Mining to Knowledge Discovery in Databases”. In: 17.3 (1996). [pdf], p. 37. DOI: [10.1609/aimag.v17i3.1230](https://www.aaai.org/ojs/index.php/aimagazine/article/view/1230). URL: <https://www.aaai.org/ojs/index.php/aimagazine/article/view/1230>.
- [FPSS96b] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. “The KDD Process for Extracting Useful Knowledge from Volumes of Data”. In: *Communications of the ACM* 39.11 (1996), pp. 27–34. DOI: [10.1145/240455.240464](https://doi.org/10.1145/240455.240464).
- [Fas25] FasterCapital. *Importance of User Interface Design in Machine Learning Applications*. <https://fastercapital.com/content/User-Interface-Design-in-AI-and-Machine-Learning-Applications.html>. Accessed: 2026-05-11. 2025.

- [GG+24] T. González Grandón et al. “Electricity demand forecasting with hybrid classical statistical and machine learning algorithms: Case study of Ukraine”. In: *Applied Energy* 355 (2024), p. 122249.
- [GLA22] A. Gasparin, S. Lukovic, and C. Alippi. “Deep learning for time series forecasting: The electric load case”. In: *CAAI Transactions on Intelligence Technology* 7.1 (2022), pp. 1–25. DOI: [10.1049/cit2.12060](https://doi.org/10.1049/cit2.12060). URL: <https://doi.org/10.1049/cit2.12060>.
- [Gér22] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O’Reilly Media, Inc., 2022. ISBN: 9781492032649.
- [HA18] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 2nd. Accessed: 2026-05-11. OTexts, 2018. URL: <https://otexts.com/fpp2/>.
- [HA21a] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 3rd. OTexts, 2021. URL: <https://otexts.com/fpp3/>.
- [HA21b] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 3rd. OTexts, 2021. URL: <https://otexts.com/fpp3/>.
- [HF16] T. Hong and S. Fan. “Probabilistic electric load forecasting: A tutorial review”. In: *International Journal of Forecasting* 32.3 (2016), pp. 914–938. DOI: [10.1016/j.ijforecast.2015.11.011](https://doi.org/10.1016/j.ijforecast.2015.11.011).
- [Haq+21] M. Haque et al. “Augmenting Speech Recognition with Noise: A Study on MFCC Features”. In: *Journal of Signal Processing Systems* 93.3 (2021), pp. 451–465.
- [Har+20] C. R. Harris et al. “Array Programming with NumPy”. In: *Nature* 585.7825 (2020). [pdf], pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [Idr15] I. Idris. *NumPy: Beginner’s Guide*. Packt Publishing Ltd, 2015. DOI: [9781785281969](https://doi.org/9781785281969).
- [Joh 1] J. Johnson. *Anomaly Detection with Machine Learning: An Introduction*. 12-12-2023. URL: <https://www.bmc.com/blogs/machine-learning-anomaly-detection/>.
- [Kum 1] S. Kumar. “5 Anomaly Detection Algorithms to Know”. In: *Built In* (8-11-2023). URL: <https://builtin.com/machine-learning/anomaly-detection-algorithms>.
- [Kum22] K. Kumari. *Time Series Forecasting Using Python*. Accessed Date 05.05.2026. 2022. URL: <https://www.analyticsvidhya.com/blog/2022/06/time-series-forecasting-using-python/>.
- [Li+21] A. Li et al. “A review on data preprocessing techniques toward efficient and reliable knowledge discovery from building operational data”. In: *Frontiers in Energy Research* 9 (2021), p. 652801. DOI: [10.3389/fenrg.2021.652801](https://doi.org/10.3389/fenrg.2021.652801). URL: <https://doi.org/10.3389/fenrg.2021.652801>.
- [Liu+23] Y. Liu et al. “Data quantity governance for machine learning in materials science”. In: *National Science Review* 10 (2023), nwad125. DOI: [10.1093/nsr/nwad125](https://doi.org/10.1093/nsr/nwad125).
- [MSSJ22a] N. S. Md Salleh, A. Suliman, and B. N. Jørgensen. “A Systematic Literature Review of Electricity Load Forecasting using Long Short-Term Memory”. In: *Proceedings of the 8th International Conference on Computational Science and Technology (ICCST 2021)*. Springer. 2022, pp. 765–776.

- [MSSJ22b] N. S. Md Salleh, A. Suliman, and B. N. Jørgensen. “A Systematic Literature Review of Electricity Load Forecasting using Long Short-Term Memory”. In: *Proceedings of the 8th International Conference on Computational Science and Technology*. Springer, 2022, pp. 765–776.
- [MWH18] S. Makridakis, S. C. Wheelwright, and R. J. Hyndman. *Forecasting: Methods and Applications*. Wiley, 2018.
- [NKHZ21] M. Narajewski, J. Kley-Holsteg, and F. Ziel. “tsrobprep — an R package for robust preprocessing of time series data”. In: *SoftwareX* 16 (2021), p. 100766. DOI: [10.1016/j.softx.2021.100766](https://doi.org/10.1016/j.softx.2021.100766). URL: <https://doi.org/10.1016/j.softx.2021.100766>.
- [Nic 0] P. Nichani. “Outliers in Machine Learning - Analytics Vidhya - Medium”. In: *Analytics Vidhya* (22-04-2020). URL: <https://medium.com/analytics-vidhya/outliers-in-machine-learning-e830b2bd8660>.
- [PJM24] PJM Interconnection LLC. *Hourly Energy Consumption*. Accessed: May 2026. 2024. URL: <https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption>.
- [POS23] M. Priestley, F. O'Donnell, and E. Simperl. “A Survey of Data Quality Requirements That Matter in ML Development Pipelines”. In: *ACM Journal of Data and Information Quality* 15.2 (2023). Funded by EU Horizon 2020 research and innovation programme under grants 951771 and 951962., Article 11. DOI: [10.1145/3592616](https://doi.org/10.1145/3592616). URL: <https://doi.org/10.1145/3592616>.
- [PY24] M.-J. Park and H.-S. Yang. “Comparative study of time series analysis algorithms suitable for short-term forecasting in implementing demand response based on AMI”. In: *Sensors* 24.22 (2024), p. 7205. DOI: [10.3390/s24227205](https://doi.org/10.3390/s24227205).
- [Py25] Python Software Foundation. *Python Programming Language*. Python Software Foundation. Delaware, USA, 2025. URL: <https://www.python.org/>.
- [RM+20] D. Rolón-Mérette et al. “The REPRISE Project: A Platform for Reproducible Computational Science”. In: *Computing in Science & Engineering* 22.2 (2020), pp. 63–72. DOI: [10.1109/MCSE.2019.2948701](https://doi.org/10.1109/MCSE.2019.2948701).
- [Rob20] Robikscube. *PJM Hourly Energy Consumption Data*. Kaggle Dataset. 2020. URL: <https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption>.
- [SP10] S. Seabold and J. Perktold. “Statsmodels: Econometric and Statistical Modeling with Python”. In: *Proceedings of the 9th Python in Science Conference (SciPy 2010)*. 2010, pp. 92–96. DOI: [10.25080/Majora-92bf1922-011](https://doi.org/10.25080/Majora-92bf1922-011).
- [Sha16] I. Shah. “Modeling and Forecasting Electricity Market Variables”. PhD thesis. University of Padua, 2016.
- [Sha+24] H. Shahrokhi et al. “PyTond: Efficient Python Data Science on the Shoulders of Databases”. In: *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 423–435. DOI: [10.1109/ICDE60146.2024.00039](https://doi.org/10.1109/ICDE60146.2024.00039). URL: <https://doi.org/10.1109/ICDE60146.2024.00039>.
- [TTL21] S. Taheri, B. Talebjedi, and T. Laukkanen. “Electricity demand time series forecasting based on empirical mode decomposition and long short-term memory”. In: *Energy Engineering* 118.6 (2021), pp. 1577–1594. DOI: [10.32604/EE.2021.017795](https://doi.org/10.32604/EE.2021.017795).

- [Wan+22] Y. Wang et al. “Multi-position wearable human activity recognition dataset”. In: (2022). DOI: [10.21227/z8bc-pt92](https://doi.org/10.21227/z8bc-pt92). URL: <https://dx.doi.org/10.21227/z8bc-pt92>.
- [ZA22] J. P. Zemkoho and A. B. Abdallah. “A Review of Time Series Forecasting Using Python”. In: *International Journal of Advanced Computer Science and Applications* 13.6 (2022). DOI: [10.14569/IJACSA.2022.0130679](https://doi.org/10.14569/IJACSA.2022.0130679).
- [Zho+24] Y. Zhou et al. “A Survey on Data Quality Dimensions and Tools for Machine Learning”. In: *ArXiv* 2406.19614 (2024). Invited Paper. URL: <https://arxiv.org/abs/2406.19614v1>.
- [tea24] T. pandas development team. “pandas: powerful Python data analysis toolkit”. In: (2024). [Accessed Date 05.05.2026](#). URL: <https://pandas.pydata.org/>.